

The University of Faisalabad

**Lab manual
Data mining**

**Submitted to:
Mr. Muhammad Arham**

**Submitted by:
Taibah Shahbaz 2023-BS-AI-024 (6A)**

**Department:
Artificial intelligence**

Contents

LAB-1: Introduction to Data Mining Software & Datasets	3
LAB-2: CRISP-DM Process & Data Types	7
LAB-3: Data Cleaning Techniques	13
LAB-4: Data Preprocessing.....	21
LAB-5: Market Basket Analysis	27
LAB-6: Apriori Algorithm	29
LAB-7: FP-Tree & FP-Growth	34
LAB-8: Python Basics for Data Mining	41
LAB-9: Decision Tree.....	44
LAB-10: Naïve Bayes & KNN.....	49
LAB-11: Support Vector Machine (SVM)	56
LAB-12: Clustering Algorithms.....	61
LAB-13: Outlier Detection Techniques	69
LAB-14: Web Scraping & Sentiment Analysis	74
Lab 15: Final Capstone Project	91

LAB-1: Introduction to Data Mining Software & Datasets

Data mining software consists of specialized tools and platforms designed to discover meaningful patterns, relationships, and insights from large volumes of data. These tools play a crucial role in transforming raw data into valuable information that supports decision-making and forecasting.

Some commonly used data mining tools include **Python** (along with libraries such as Pandas, NumPy, and Scikit-learn), RapidMiner, Orange, and WEKA. These platforms are widely adopted across industries due to their ability to perform various tasks like data preprocessing, visualization, classification, clustering, and association rule mining.

Additionally, data mining software helps automate repetitive processes and efficiently manage large-scale datasets. In practical applications, organizations use these tools to study customer behavior, forecast future trends, and enhance business strategies for better outcomes.

Example:

An online retail company can use data mining software to analyze purchase history and recommend products to customers based on their previous buying patterns.

Datasets

A dataset refers to an organized collection of data, typically arranged in a tabular format consisting of rows and columns. In this structure, each row corresponds to a specific record or instance, while each column represents a particular feature or attribute.

Datasets can be categorized into three main types:

- **Structured Data:** This type of data is highly organized and usually stored in databases in the form of tables.
- **Unstructured Data:** This includes data that does not follow a predefined format, such as text documents, images, and videos.
- **Semi-structured Data:** This type lies between structured and unstructured data, often found in formats like JSON and XML, where some organizational properties exist.

1. Objective

The objective of this lab is to introduce students to data import, basic structural inspection, and exploratory data analysis (EDA). Students will learn how to profile a retail store's dataset,

identify data formats, and generate baseline statistical summaries to extract immediate business insights.

2. Tools & Datasets

- **Tool:** KNIME Analytics Platform
- **Dataset:** Supermarket Sales / Retail Sales Dataset (Kaggle)

3. Background Concepts

Before running complex predictive algorithms, a data analyst must perform **Data Profiling**. This involves checking what types of data are in the columns and computing general statistics to understand data boundaries.

- **Data Types:** Understanding if an attribute is *Nominal/Categorical* (e.g., Product Line, Gender) or *Numerical/Continuous* (e.g., Unit Price, Quantity, Total Revenue).
- **Summary Statistics:**
 - * **Mean (Average):** The central tendency of the data.
 - **Median:** The exact middle value, highly useful when data contains extreme price outliers.
 - **Standard Deviation:** Measures how spread out the values are around the mean.

Lab work

Search dataset

The image shows a screenshot of the Kaggle 'Retail Sales' dataset page. The page header includes the user 'RICHARD CHINYANDU' and the update date 'UPDATED 8 MONTHS AGO'. The dataset title is 'Retail Sales' with a subtitle 'Uncovering Retail Insights: Revenue Trends by Product, Gender & Time'. There are tabs for 'Data Card', 'Code (0)', 'Discussion (0)', and 'Suggestions (0)'. The 'About Dataset' section describes the data as sourced from Kaggle, involving 2,514 transactions, and lists dimensions like product category, customer gender, day of the week, and revenue. It also mentions the inspiration for using the dataset: uncovering what drives revenue, whether it's weekend shopping patterns, gender-based preferences, or popular product types like beauty, clothing, and electronics. On the right, there are sections for 'Usability' (8.82), 'License' (CC0: Public Domain), and 'Expected update frequency' (Never). Below the dataset page, there is a 'create workflow' dialog box titled 'Create a new workflow'. It has a text input field for 'Workflow name' with the value 'retail lab-01' entered. At the bottom right of the dialog are 'Cancel' and 'Create' buttons.

File

Settings Transformation Advanced Settings Limit Rows Encoding Flow Variables Job Manager Selection Memory Policy

Input location

Read from Local File System

Mode ☒ File ☐ Files in folder

File C:\Users\hp\OneDrive\Desktop\retail_sales_dataset.csv Browse...

The screenshot displays the Orange3 data mining software interface. On the left is a 'Nodes' panel with a search bar containing 'sta' and a list of nodes including Statistics, CrossTab, Stacked Area Chart, Statistics View, Java Distance, Group Loop Start, String Distances, Matrix Distance, Check Loop Start, CASE Switch Start, Model Loop Start, Numerical Distances, Window Loop Start, Genetic Loop Start, Mahalanobis Distance, Counting Loop Start, Interval Loop Start, and Aggregated Distance. The main workspace shows a workflow with a 'CSV Reader' node connected to a 'Statistics' node, with a label 'data.csv' below the connection. On the right is a 'Statistics' widget window. It has a 'Calculate median values (computationally expensive)' checkbox, a 'Nominal Values' section with a 'Column filter' dropdown set to 'Manual', and a search bar. Below this are 'Excludes' and 'Includes' lists. The 'Includes' list contains 'Transaction ID' and 'Date'. At the bottom of the window are 'Discard' and 'Apply and Execute' buttons. Below the Statistics window, a table titled 'Table 1: Statistics' is displayed. The table has columns: #, RowID, Column, Min, Max, Mean, Std. dev., Variance, Skewness, and Kurtosis. The data rows are:

#	RowID	Column	Min	Max	Mean	Std. dev.	Variance	Skewness	Kurtosis
1	Transac	Transaction ID	1	1,000	500.5	288.819	88,416.867	0	-1.2
2	Age	Age	18	64	41.262	13.681	187.182	-0.049	-1.201
3	Quantit	Quantity	1	4	2.514	1.188	1.288	-0.014	-1.393
4	Price p	Price per Unit	25	500	179.88	180.481	38,979.017	0.736	-1.139

KPI 1

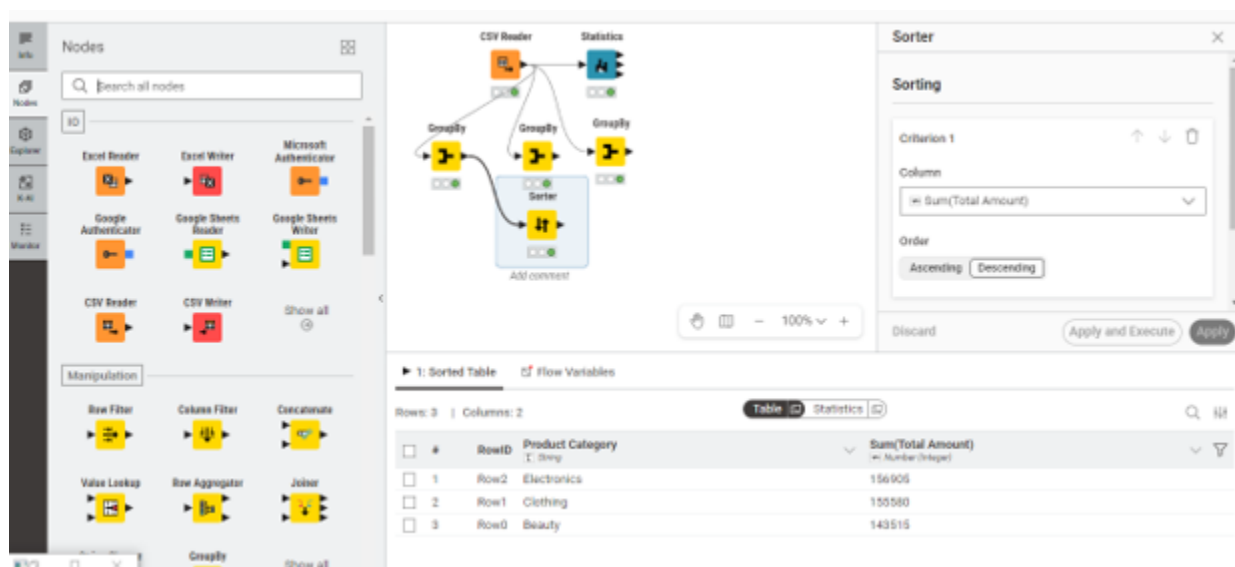
KPI 2



KPI 3



Scatter node



LAB-2: CRISP-DM Process & Data Types

1. Objective

The objective of this lab is to introduce students to the **CRISP-DM** (Cross-Industry Standard Process for Data Mining) methodology. Students will map a real-world e-commerce customer churn problem across the standard CRISP-DM phases, execute data preparation, and build a predictive classification model.

2. Tools & Datasets

- **Tool:** Orange Data Mining
- **Dataset:** Customer Churn Dataset (Available on Kaggle)

3. Background Concepts

The CRISP-DM methodology provides a structured, iterative approach to managing data science projects. It breaks down a project lifecycle into six core phases:

1. **Business Understanding:** Aligning project goals with business problems (e.g., "Why are e-commerce customers leaving?").
2. **Data Understanding:** Collecting, exploring, and identifying quality issues in the customer dataset.
3. **Data Preparation:** Selecting, cleaning, and transforming raw records into a final model-ready layout.
4. **Modeling:** Selecting and applying various machine learning algorithms (e.g., Logistic Regression or Random Forest).
5. **Evaluation:** Checking if the model mathematically meets business thresholds before deployment.
6. **Deployment:** Integrating the insights or model into active business operations.

Types of Data

Data can be categorized into different types depending on its characteristics and usage:

- **Numerical Data:**
Includes quantitative values that can be measured. It can be:
 - *Discrete* (e.g., number of courses taken)
 - *Continuous* (e.g., height or temperature)
- **Categorical Data:**
Represents qualitative values and is divided into:
 - *Nominal* (e.g., department names like IT, HR)

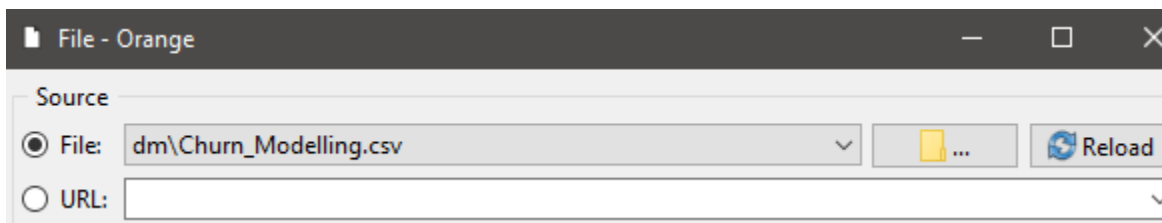
- *Ordinal* (e.g., performance levels such as low, medium, high)
- **Binary Data:**
Consists of only two possible values, such as yes/no or true/false.

Understanding these data types is essential because different algorithms work better with specific types of data. Choosing the correct type ensures better model accuracy and performance.

Dataset



Load dataset



Check on data table

Data Table - Orange

Info
 10000 instances (no missing data)
 13 features
 No target variable.
 1 meta attribute

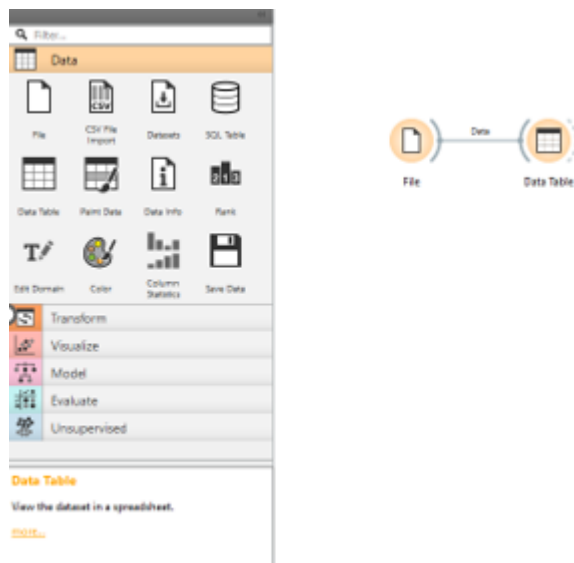
Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
 Clear Selection
☒ Select full rows

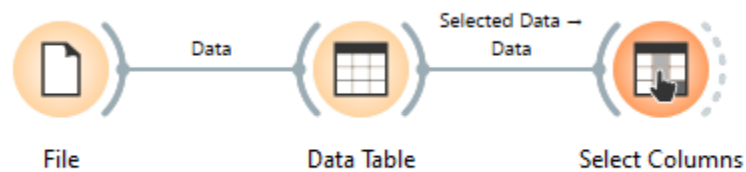
Restore Original Order
☒ Send Automatically

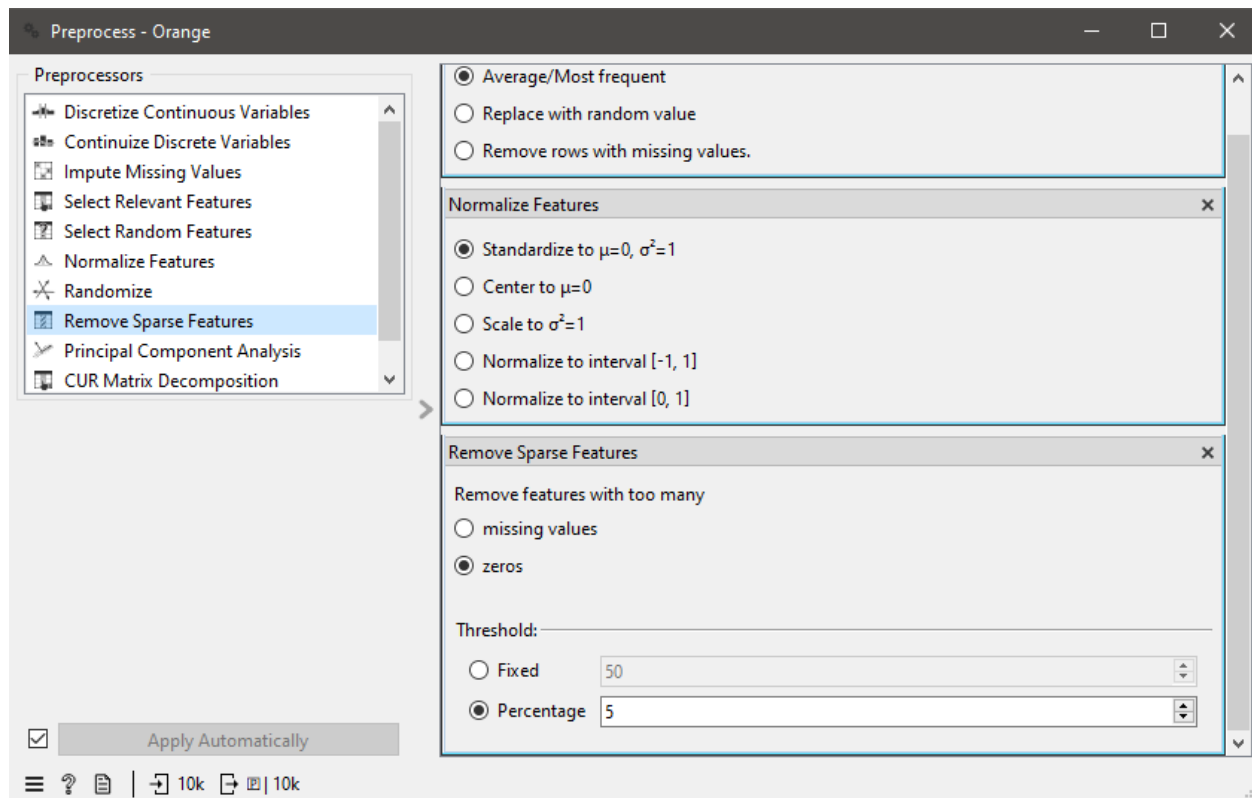
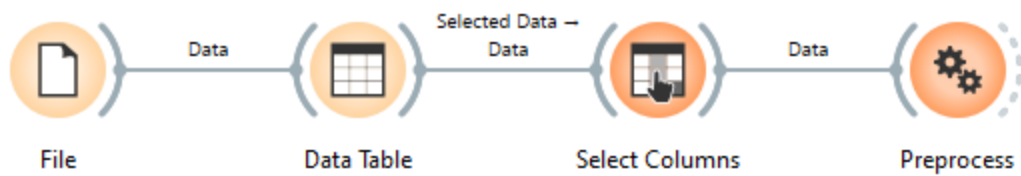
	Surname	RowNumber	CustomerId	CreditScore	Geography
1	Hargrave	1	15634602	619	France
2	Hill	2	15647311	608	Spain
3	Onio	3	15619304	502	France
4	Boni	4	15701354	699	France
5	Mitchell	5	15737888	850	Spain
6	Chu	6	15574012	645	Spain
7	Bartlett	7	15592531	822	France
8	Obinna	8	15656148	376	Germany
9	He	9	15792365	501	France
10	H?	10	15592389	684	France
11	Bearce	11	15767821	528	France
12	Andrews	12	15737173	497	Spain
13	Kay	13	15632264	476	France
14	Chin	14	15691483	549	France
15	Scott	15	15600882	635	Spain
16	Goforth	16	15643966	616	Germany
17	Romeo	17	15737452	653	Germany
18	Henderson	18	15788218	549	Spain

Attach datatable to file.

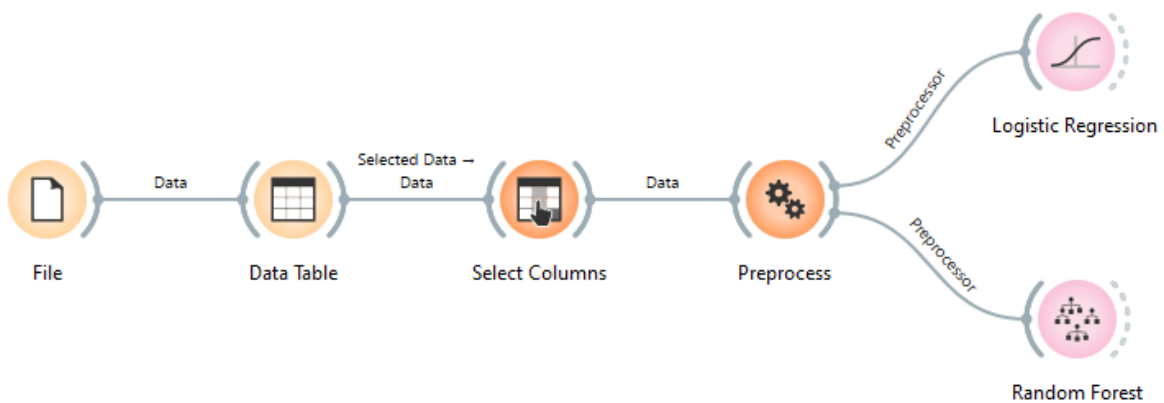


Then connect select column to it.





Connect logistic regression and random forest classifier and then connect test and score to it.





Test and score:

Test and Score - Orange

☒ Cross validation

Number of folds: 10

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 66 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

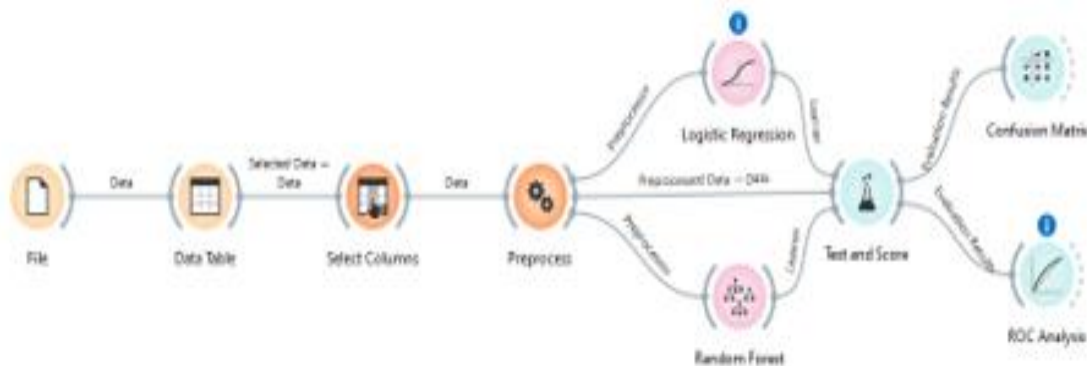
Model	AUC	CA	F1	Prec	Recall	MCC
Logistic Regression	0.735	0.779	0.717	0.698	0.779	0.049
Random Forest	0.792	0.830	0.816	0.814	0.830	0.412

Compare models by: Area under ROC curve

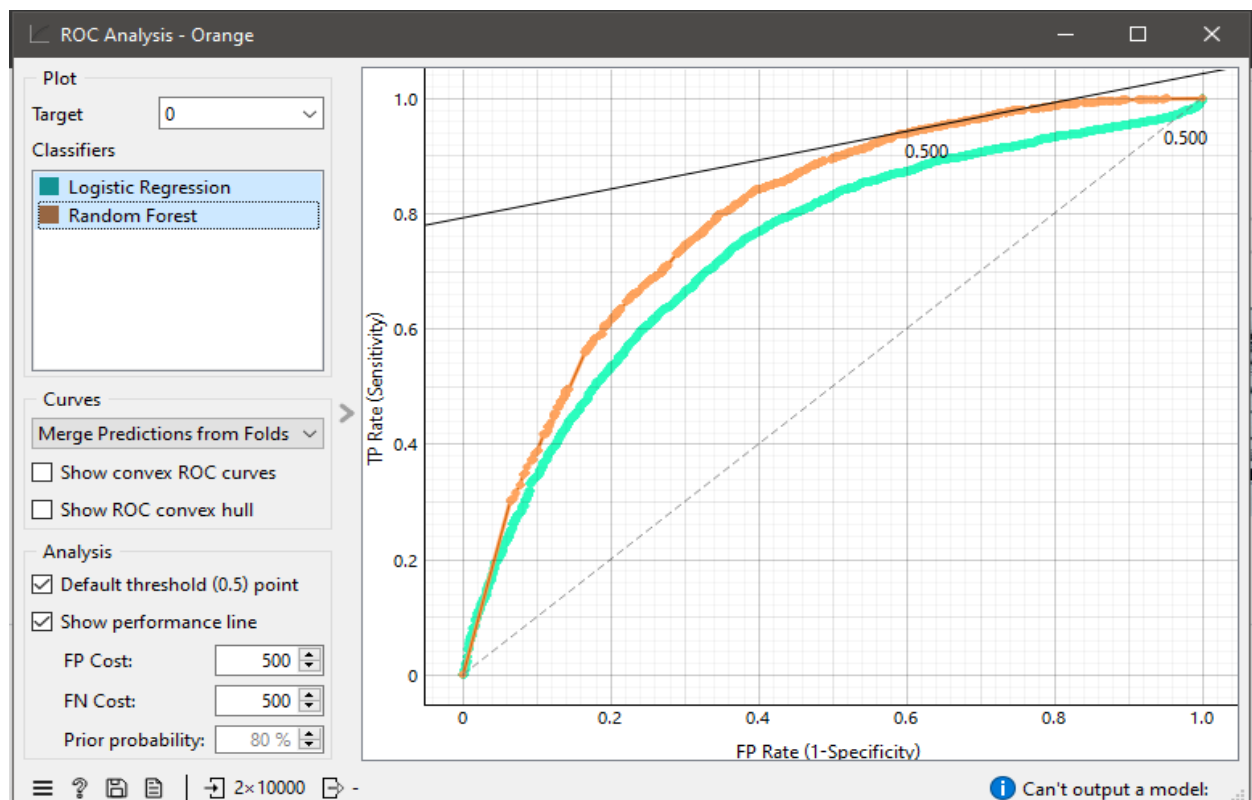
☐ Negligible diff.: 0.1

	Logistic Reg...	Random Fo...
Logistic Regression		0.000
Random Forest	1.000	

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.



Logistic Regression		corresponding data instances			now: INL
Random Forest					
		Predicted			
		0	1	Σ	
Actual	0	7464	499	7963	
	1	1200	837	2037	
	Σ	8664	1336	10000	



Churn Rate	20.37%
Retention Rate	79.63%
Logistic Regression AUC	0.735
Random Forest AUC	0.792
Accuracy	82.8%

LAB-3: Data Cleaning Techniques

1. Objective

The objective of this lab is to prepare raw, imperfect healthcare data for downstream analytics. Students will learn how to handle missing data, eliminate duplicate records, and apply noise-filtering techniques to improve overall data quality.

2. Tools & Datasets

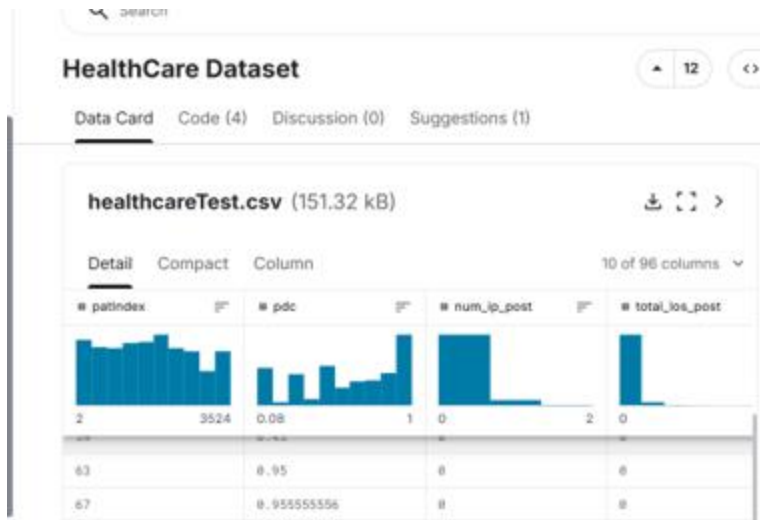
- **Tools:** RapidMiner Studio & Python
- **Dataset:** Healthcare Dataset / Patient Records

3. Background Concepts

Data cleaning is the most critical phase of data preprocessing. In healthcare analytics, dirty data can lead to dangerous, incorrect insights.

- **Missing Values:** Can be handled by deletion (dropping rows/columns) or imputation (replacing with Mean, Median, Mode, or predictive values).
- **Duplicates:** Identical rows that artificially skew statistical distributions.
- **Noise Filtering:** Removing outliers or incorrect formatting that disrupts model patterns.

Lab Work on RapidMiner Studio



Altair AI Studio Educational 2026.0.5 @ DevHannah

File Edit Process View Connections Settings Extensions Help

Views: Design Results Turbo Prep Auto Model Interactive Analysis

Find data, operators, etc. Alt Studio

Repository

- Import Data
- Training Resources (connected)
- Samples
- Community Samples (connected)
- Local Repository (local)
- DB (support)

Operators

Search for Operators

- Data Access (82)
- Blending (81)
- Cleansing (28)
- Modeling (167)
- Scoring (13)
- Validation (30)
- Utility (86)

Get more operators from the Marketplace

Process

Process

Your process looks empty.
Add some data first.
Drag data or operators here.

Leverage the Wisdom of Crowds to get operator recommendations based on your process design!

Activate Wisdom of Crowds

Parameters

Process

logverbosity: init

logfile:

Show advanced parameters

Change compatibility (12.0.005)

Help

Process

AI Studio Core

Synopsis

The root operator of every process.

Description

This operator provides a set of parameters.

File Edit Process View Connections Settings Extensions Help

Views: Design Results Turbo Prep Auto Model Interactive Analysis

Result History

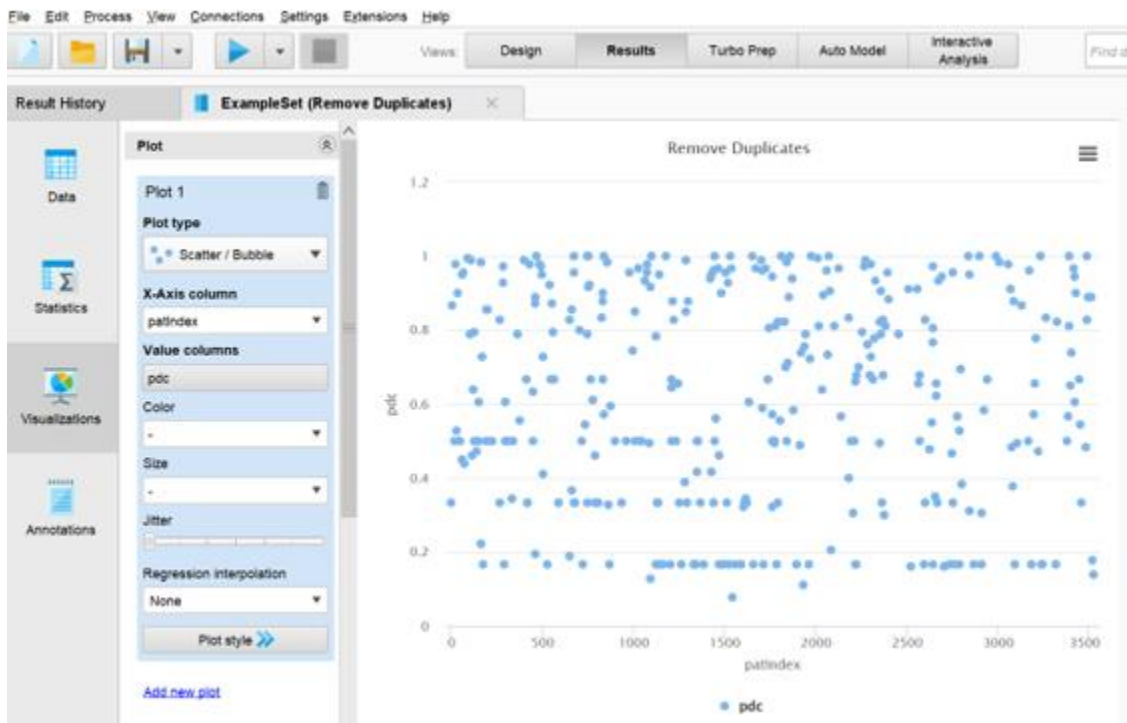
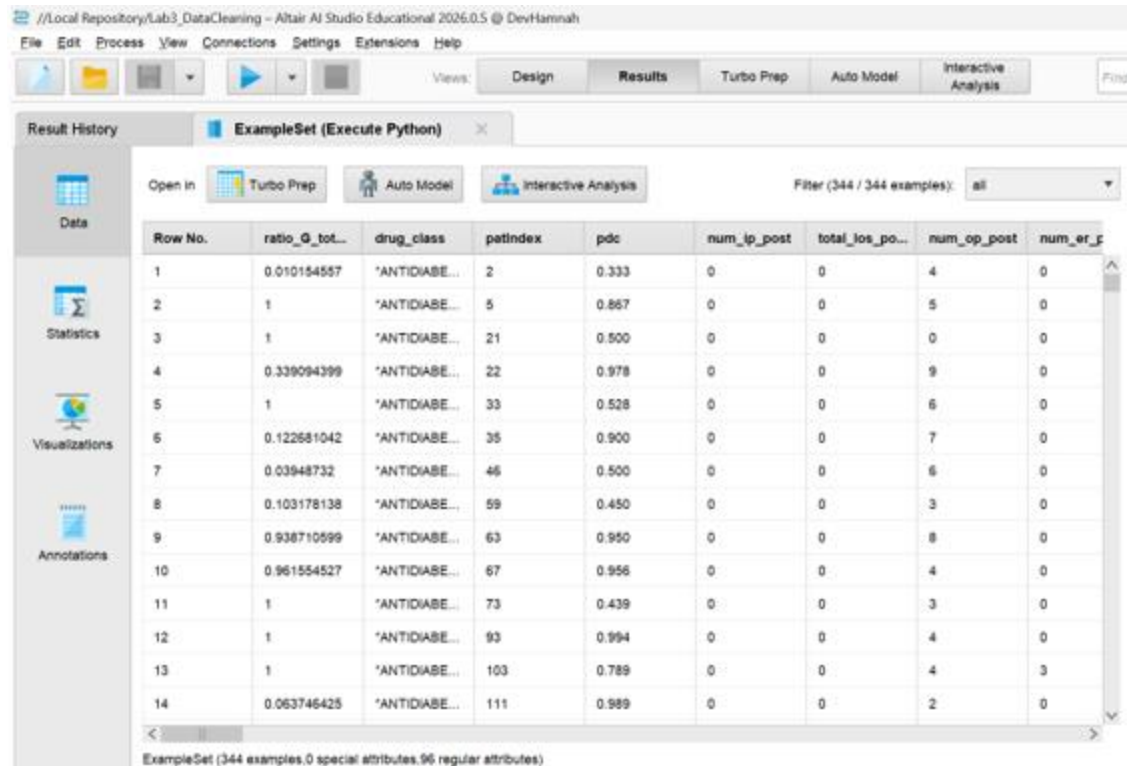
ExampleSet (Read CSV)

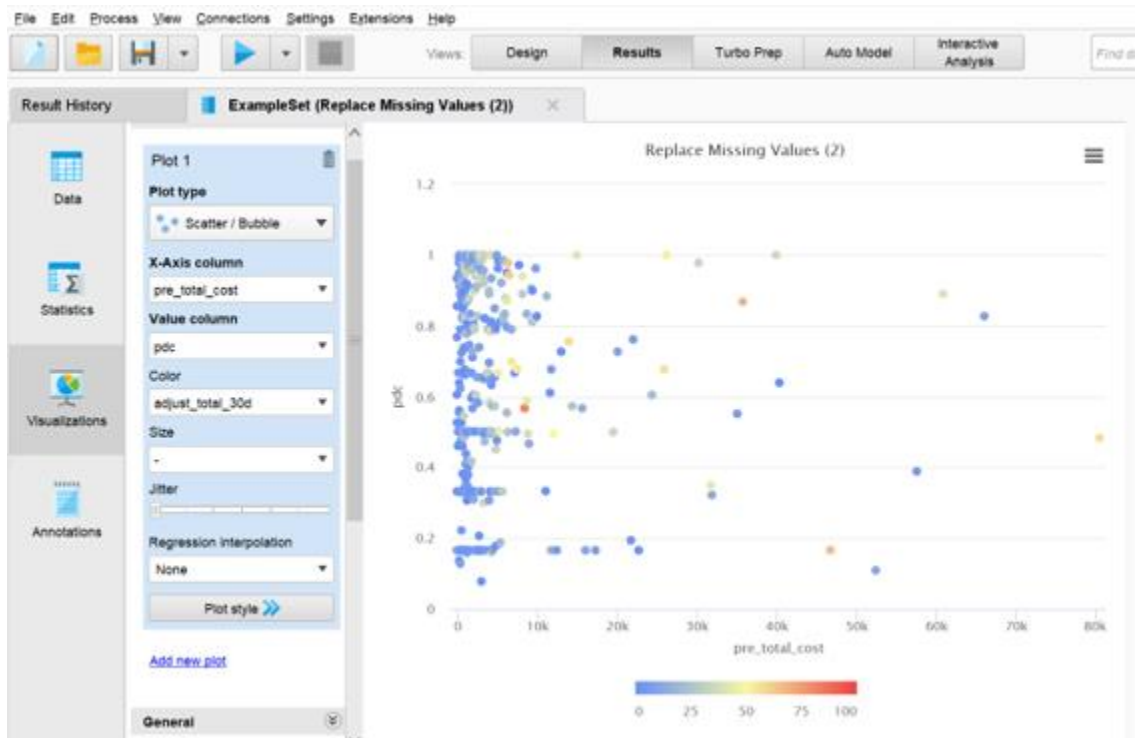
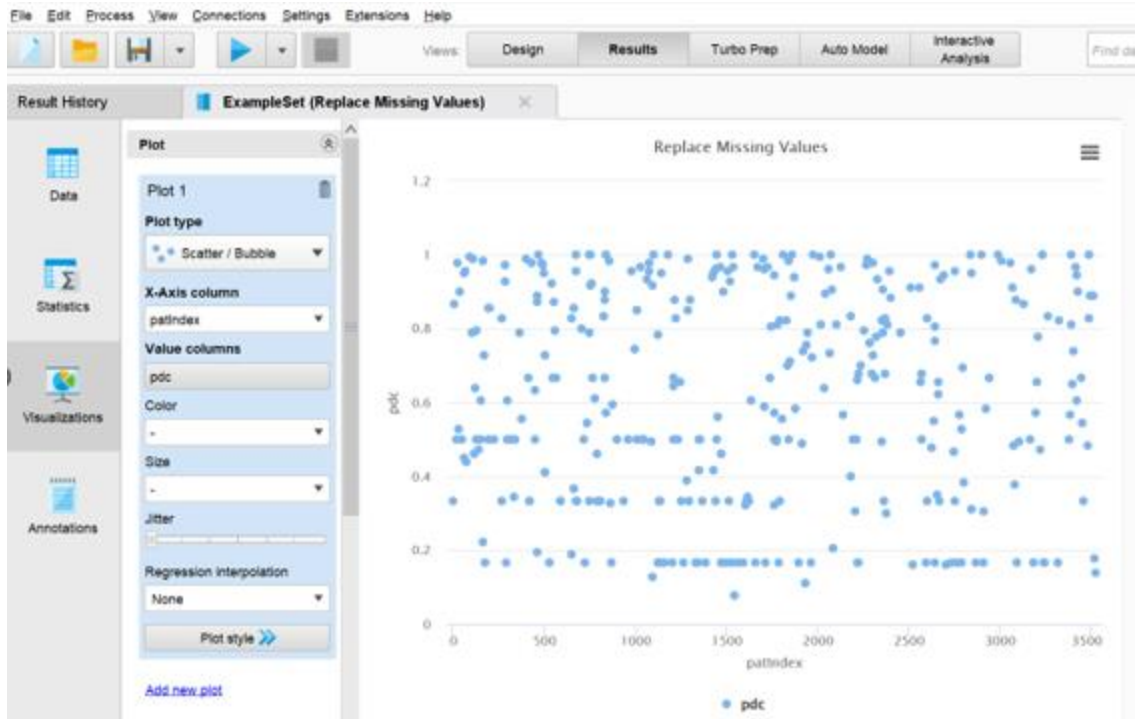
Open in: Turbo Prep Auto Model Interactive Analysis

Filter (344 / 344 examples): all

Row No.	petindex	pdcc	num_ip_post	total_los_po...	num_op_post	num_er_post	num_ndc_p...	num_gpi6
1	2	0.333	0	0	4	0	15	5
2	5	0.867	0	0	5	0	16	4
3	21	0.500	0	0	0	0	8	6
4	22	0.978	0	0	9	0	40	9
5	33	0.528	0	0	6	0	28	7
6	35	0.900	0	0	7	0	37	12
7	46	0.500	0	0	6	0	13	4
8	59	0.450	0	0	3	0	29	9
9	63	0.950	0	0	8	0	18	9
10	67	0.956	0	0	4	0	64	13
11	73	0.439	0	0	3	0	6	3
12	93	0.994	0	0	4	0	15	5
13	103	0.789	0	0	4	3	4	3
14	111	0.989	0	0	2	0	2	1







Lab Work on python

```
import pandas as pd
import numpy as np
```

1. Load the dataset

```
df = pd.read_csv('healthcare_dataset.csv')
print("--- BEFORE CLEANING ---")
print(f"Dataset Shape: {df.shape}")
print("Missing Values per Column:\n", df.isnull().sum())
print(f"Total Duplicate Rows: {df.duplicated().sum()}\n")
```

```
--- BEFORE CLEANING ---
Dataset Shape: (55500, 15)
Missing Values per Column:
  Name      0
  Age      0
  Gender    0
  Blood Type 0
  Medical Condition 0
  Date of Admission 0
  Doctor      0
  Hospital    0
  Insurance Provider 0
  Billing Amount 0
  Room Number 0
  Admission Type 0
  Discharge Date 0
  Medication   0
  Test Results 0
dtype: int64
Total Duplicate Rows: 534
```

2. Check for missing values and duplicates

```
print("Initial Missing Values:\n", df.isnull().sum())
print("Duplicate Rows:", df.duplicated().sum())
```

```
Initial Missing Values:
Name                0
Age                 0
Gender              0
Blood Type          0
Medical Condition   0
Date of Admission   0
Doctor              0
Hospital            0
Insurance Provider  0
Billing Amount      0
Room Number         0
Admission Type      0
Discharge Date      0
Medication          0
Test Results        0
dtype: int64
Duplicate Rows: 534
```

3. Handle Missing Values

```
# Impute continuous with median, categorical with mode

df['Age'] = df['Age'].fillna(df['Age'].median())
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
```

4. Remove Duplicates

```
df = df.drop_duplicates()
```

5. Noise Filtering ¶

```
print("Minimum Age in dataset:", df['Age'].min())
print("Maximum Age in dataset:", df['Age'].max())
df = df[(df['Age'] >= 18) & (df['Age'] <= 65)]
```

```
Minimum Age in dataset: 13
Maximum Age in dataset: 89
```

6. Output

```
print("--- AFTER CLEANING ---")
print(f"Dataset Shape: {df.shape}")
print("Missing Values remaining:\n", df.isnull().sum())
print(f"Duplicate Rows remaining: {df.duplicated().sum()}\n")

print("--- FIRST 5 ROWS OF CLEAN DATA ---")
print(df.head())
```

```
--- AFTER CLEANING ---
Dataset Shape: (38754, 15)
Missing Values remaining:
  Name      0
  Age      0
  Gender    0
  Blood Type 0
  Medical Condition 0
  Date of Admission 0
  Doctor      0
  Hospital    0
  Insurance Provider 0
  Billing Amount 0
  Room Number 0
  Admission Type 0
  Discharge Date 0
  Medication    0
  Test Results  0
dtype: int64
Duplicate Rows remaining: 0
```

--- FIRST 5 ROWS OF CLEAN DATA ---

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	\
0	Bobby JacksOn	30	Male	B-	Cancer	2024-01-31	
1	Leslie TErRy	62	Male	A+	Obesity	2019-08-20	
3	andrEw waTtS	28	Female	O+	Diabetes	2020-11-18	
4	adrIENNE bEll	43	Female	AB+	Cancer	2022-09-19	
5	EMILY JOHNSOn	36	Male	A+	Asthma	2023-12-20	

	Doctor	Hospital	Insurance Provider	\
0	Matthew Smith	Sons and Miller	Blue Cross	
1	Samantha Davies	Kim Inc	Medicare	
3	Kevin Wells	Hernandez Rogers and Vang,	Medicare	
4	Kathleen Hanna	White-White	Aetna	
5	Taylor Newton	Nunez-Humphrey	UnitedHealthcare	

	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	\
0	18856.281306	328	Urgent	2024-02-02	Paracetamol	
1	33643.327287	265	Emergency	2019-08-26	Ibuprofen	
3	37909.782410	450	Elective	2020-12-18	Ibuprofen	
4	14238.317814	458	Urgent	2022-10-09	Penicillin	
5	48145.110951	389	Urgent	2023-12-24	Ibuprofen	

	Test Results
0	Normal
1	Inconclusive
3	Abnormal
4	Abnormal
5	Normal

KPI EVALUATION METRICS

```

initial_rows = 55500
final_rows = len(df)
rows_removed = initial_rows - final_rows

# 2. Calculate KPIs
noise_elimination_rate = (rows_removed / initial_rows) * 100
data_retention_rate = (final_rows / initial_rows) * 100

# 3. Print KPI Report
print(f"Initial Rows Processed : {initial_rows}")
print(f"Cleaned Rows Retained : {final_rows}")
print(f"Total Anomalies Removed: {rows_removed}")
print(f"Noise Elimination Rate : {noise_elimination_rate:.2f}%")
print(f>Data Retention Rate : {data_retention_rate:.2f}%")

```

```

Initial Rows Processed : 55500
Cleaned Rows Retained : 38754
Total Anomalies Removed: 16746
Noise Elimination Rate : 30.17%
Data Retention Rate : 69.83%

```

LAB-4: Data Preprocessing

1. Objective

The objective of this lab is to transform cleaned loan risk data into a mathematically balanced and optimized format ready for machine learning models. Students will master advanced feature transformation techniques, including data normalization, standardization, and discretization (binning).

2. Tools & Datasets

- **Tool:** RapidMiner Studio
- **Dataset:** Loan Prediction Dataset / Bank Dataset (Available on Kaggle)

3. Background Concepts

Raw data often contains features with completely different scales (e.g., an *Applicant Income* attribute ranging from \$1,000 to \$100,000, vs. a *Credit Score* attribute ranging from 300 to 850). Algorithms that rely on distance calculations can get distorted by these scale differences.

- **Normalization (Min-Max Scaling):** Transforms data linearly into a fixed range, typically between 0 and 1.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Standardization (Z-Score):** Centers the data around a mean (μ) of 0 with a standard deviation (σ) of 1. Ideal for data that follows a Gaussian (normal) distribution.

$$X_{std} = \frac{X - \mu}{\sigma}$$

- **Discretization (Binning):** The process of converting continuous numeric attributes into categorical group buckets (e.g., transforming numerical age into "Young", "Middle-Aged", and "Senior").

Lab Work on RapidMiner Studio

Loan Prediction Dataset

3

<> Code

Download

Data Card

Code (1)

Discussion (0)

Suggestions (0)

loan_prediction_dataset.csv (71.02 kB)

Detail

Compact

Column

7 of 7 columns

About this file

Suggest Edits

This file does not have a description yet.

Age

1869

Income

20.2k150k

Credit_Score

300849

Loan_Amount

10605000

Data Explorer

Version 1 (71.02 kB)

loan_prediction_dataset

Summary

1 file

7 columns

Read CSV



Open in



Turbo Prep



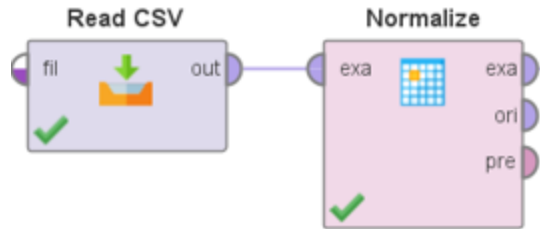
Auto Model



Interactive Analysis

Row No.	Age,Income...
1	56,81788,33...
2	69,102879,7...
3	46,58827,77...
4	32,127188,3...
5	60,25655,30...
6	25,64859,60...
7	38,73625,64...
8	56,72098,73...
9	36,35251,72...
10	40,142413,6...
11	28,20526,73...
12	28,88760,47...
13	41,67837,82...
14	53,145133,3...
15	57,149478,7...

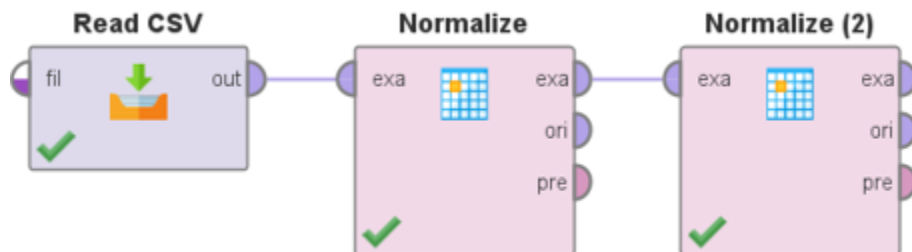
ExampleSet (2,000 examples,0 special attributes,1 regular attribute)



Open in [Turbo Prep](#) [Auto Model](#) [Interactive Analysis](#) Filter (2,000 / 2,000 examples): [all](#)

Row No.	Income	Loan_Amount	Age	Credit_Score	Loan_Term	Employment_Status	Loan_App
1	0.475	0.285	56	334	48	Employed	0
2	0.637	0.408	69	781	24	Self-Employed	1
3	0.298	0.789	46	779	60	Self-Employed	0
4	0.824	0.323	32	364	24	Unemployed	0
5	0.042	0.515	60	307	36	Unemployed	0
6	0.344	0.612	25	600	36	Employed	0
7	0.412	0.134	38	642	24	Employed	0
8	0.400	0.763	56	730	36	Self-Employed	0
9	0.116	0.681	36	723	36	Employed	0
10	0.942	0.831	40	631	60	Employed	0
11	0.003	0.880	28	737	48	Unemployed	0
12	0.528	0.747	28	473	60	Employed	0
13	0.367	0.341	41	826	12	Employed	1
14	0.963	0.477	53	325	60	Employed	0
15	0.996	0.456	57	705	24	Employed	1

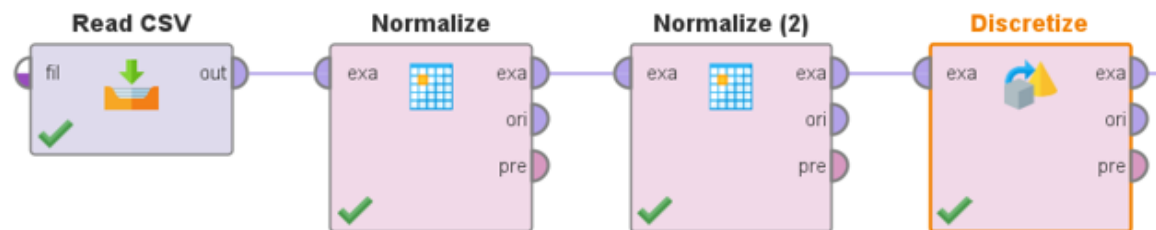
ExampleSet (2,000 examples,0 special attributes,7 regular attributes)



Open In Turbo Prep Auto Model Interactive Analysis Filter (2,000 / 2,000 examples): all

Row No.	Credit_Score	Income	Loan_Amou...	Age	Loan_Term	Employmen...	Loan_Appro...
1	-1.543	0.475	0.285	56	48	Employed	0
2	1.295	0.637	0.408	69	24	Self-Employed	1
3	1.282	0.298	0.789	46	60	Self-Employed	0
4	-1.353	0.824	0.323	32	24	Unemployed	0
5	-1.714	0.042	0.515	60	36	Unemployed	0
6	0.146	0.344	0.612	25	36	Employed	0
7	0.412	0.412	0.134	38	24	Employed	0
8	0.971	0.400	0.763	56	36	Self-Employed	0
9	0.926	0.116	0.681	36	36	Employed	0
10	0.342	0.942	0.831	40	60	Employed	0
11	1.015	0.003	0.880	28	48	Unemployed	0
12	-0.661	0.528	0.747	28	60	Employed	0
13	1.580	0.367	0.341	41	12	Employed	1
14	-1.600	0.963	0.477	53	60	Employed	0
15	0.812	0.996	0.456	57	24	Employed	1

ExampleSet (2,000 examples,0 special attributes,7 regular attributes)



Open In Turbo Prep Auto Model Interactive Analysis Filter (2,000 / 2,000 examples): all

Row No.	Age	Credit_Score	Income	Loan_Amou...	Loan_Term	Employmen...	Loan_Appro...
1	range3 [52 - ∞]	-1.543	0.475	0.285	48	Employed	0
2	range3 [52 - ∞]	1.295	0.637	0.408	24	Self-Employed	1
3	range2 [35 - ...]	1.282	0.298	0.789	60	Self-Employed	0
4	range1 [-∞ - ...]	-1.353	0.824	0.323	24	Unemployed	0
5	range3 [52 - ∞]	-1.714	0.042	0.515	36	Unemployed	0
6	range1 [-∞ - ...]	0.146	0.344	0.612	36	Employed	0
7	range2 [35 - ...]	0.412	0.412	0.134	24	Employed	0
8	range3 [52 - ∞]	0.971	0.400	0.763	36	Self-Employed	0
9	range2 [35 - ...]	0.926	0.116	0.681	36	Employed	0
10	range2 [35 - ...]	0.342	0.942	0.831	60	Employed	0
11	range1 [-∞ - ...]	1.015	0.003	0.880	48	Unemployed	0
12	range1 [-∞ - ...]	-0.661	0.528	0.747	60	Employed	0
13	range2 [35 - ...]	1.580	0.367	0.341	12	Employed	1
14	range3 [52 - ∞]	-1.600	0.963	0.477	60	Employed	0
15	range3 [52 - ∞]	0.812	0.996	0.456	24	Employed	1

ExampleSet (2,000 examples,0 special attributes,7 regular attributes)

Lab work on python

1. Summary Statistics

```
[1]: import pandas as pd

# Load the dataset
df = pd.read_csv('loan_prediction_dataset.csv')
```

```
[9]: print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

2. Normalization (Min-Max Scaling)

```
[4]: from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler_minmax = MinMaxScaler()

# We will normalize 'Income' and 'Loan_Amount'
df_normalized = df.copy()
cols_to_normalize = ['Income', 'Loan_Amount']
df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

print("Normalized Data (First 5 rows):")
print(df_normalized[cols_to_normalize].head())
```

Normalized Data (First 5 rows):

	Income	Loan_Amount
0	0.474695	0.285323
1	0.637137	0.407753
2	0.297850	0.789369
3	0.824364	0.323415
4	0.042361	0.514898

3. Standardization (Z-score Scaling)

```
[5]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler_std = StandardScaler()

# We will standardize 'Age' and 'Credit_Score'
df_standardized = df.copy()
cols_to_standardize = ['Age', 'Credit_Score']
df_standardized[cols_to_standardize] = scaler_std.fit_transform(df[cols_to_standardize])

print("Standardized Data (First 5 rows):")
print(df_standardized[cols_to_standardize].head())
```

Standardized Data (First 5 rows):

	Age	Credit_Score
0	0.817026	-1.543338
1	1.688020	1.294999
2	0.147031	1.282300
3	-0.790963	-1.352846
4	1.085024	-1.714781

4. Discretization (Binning)

```
[7]: df_discretized = df.copy()
df_discretized['Income_Category'] = pd.cut(df['Income'],
                                           bins=3,
                                           labels=['Low', 'Medium', 'High'])

from sklearn.preprocessing import KBinsDiscretizer
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df_discretized['Age_Bins'] = est.fit_transform(df[['Age']])

print("Discretized Data (First 5 rows):")
print(df_discretized[['Income', 'Income_Category', 'Age', 'Age_Bins']].head())
```

Discretized Data (First 5 rows):

	Income	Income_Category	Age	Age_Bins
0	81788	Medium	56	2.0
1	102879	Medium	69	2.0
2	58827	Low	46	1.0
3	127188	High	32	0.0
4	25655	Low	60	2.0

LAB-5: Market Basket Analysis

1. Objective

The objective of this lab is to understand and implement association rule mining using Market Basket Analysis (MBA). Students will learn how a supermarket develops cross-selling strategies by calculating key association metrics—Support, Confidence, and Lift—manually, and validating their results programmatically using Python.

2. Tools & Datasets

- **Tool:** Python (Pandas & Mlxtend)
- **Dataset:** Grocery Transactions Dataset / Instacart Market Basket Analysis (Kaggle)

3. Background Concepts

Market Basket Analysis searches for behavioral combinations among customer purchases. It answers questions like, *"If a customer buys product X, how likely are they to buy product Y?"*

To find these relationships, we evaluate **Association Rules** (written as $X \rightarrow Y$, where X is the Antecedent and Y is the Consequent) using three fundamental mathematical metrics:

- **Support:** The proportion of total transactions that contain the itemset. It measures how frequently an itemset appears.

$$Support(X \rightarrow Y) = \frac{\text{Transactions containing both } X \text{ and } Y}{\text{Total Transactions}}$$

- **Confidence:** The conditional probability that a customer buys item \$Y\$, given that they have already purchased item \$X\$. It measures the reliability of the rule.

$$Confidence(X \rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

- **Lift:** Measures how much more likely item \$Y\$ is to be bought relative to its ordinary, random baseline sales rate when item \$X\$ is purchased.

$$Lift(X \rightarrow Y) = \frac{Confidence(X \rightarrow Y)}{Support(Y)}$$

- Lift = 1\$: \$X\$ and \$Y\$ are independent.
- Lift > 1\$: Positive association (strong cross-sell potential).

- Lift < 1\$: Negative association (items substitute or repel each other).

Lab task

Step 1: Load Data

```
[1]: import pandas as pd
```

```
df = pd.read_csv("transaction_data.csv")  
df.head()
```

[1]:	UserId	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Step 2: Convert Data into Transaction List

```
[5]: transactions = df.groupby('TransactionId')['ItemDescription'].apply(list)  
transactions = list(transactions)
```

```
print(transactions[:5])
```

```
[['SET 7 BABUSHKA NESTING BOXES', 'WHITE METAL LANTERN', 'RED WOOLLY HOTTIE WHITE HEART.', 'CREAM CUPID HEARTS COAT HANGER', 'SET 7 BABUSHKA NESTING BOXES', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'WHITE HANGING HEART T-LIGHT HOLDER', 'WHITE METAL LANTERN', 'CREAM CUPID HEARTS COAT HANGER', 'RED WOOLLY HOTTIE WHITE HEART.', 'GLASS STAR FROSTED T-LIGHT HOLDER', 'WHITE HANGING HEART T-LIGHT HOLDER', 'GLASS STAR FROSTED T-LIGHT HOLDER'], ['HAND WARMER RED POLKA DOT', 'HAND WARMER UNION JACK', 'HAND WARMER UNION JACK', 'HAND WARMER RED POLKA DOT'], ['DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'POPPY'S PLAYHOUSE BEDROOM', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'RECIPE BOX WITH METAL HEART', 'IVORY KNITTED MUG COSY', 'BOX OF VINTAGE JIGSAW BLOCKS', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE BEDROOM', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE KITCHEN', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'IVORY KNITTED MUG COSY', 'RECIPE BOX WITH METAL HEART', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'POPPY'S PLAYHOUSE KITCHEN', 'BOX OF VINTAGE JIGSAW BLOCKS'], ['BLUE COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'YELLOW COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS', 'YELLOW COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'BLUE COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS'], ['BATH BUILDING BLOCK WORD', 'BATH BUILDING BLOCK WORD']]
```

```
[6]: transactions = [list(set(t)) for t in transactions]
```

Step 3: Total Transactions

```
[7]: total_transactions = len(transactions)  
print(total_transactions)
```

```
25900
```

Step 4: Manual Calculation (A → B)

```
[8]: A = 'WHITE HANGING HEART T-LIGHT HOLDER'  
B = 'CREAM CUPID HEARTS COAT HANGER'
```

```
[9]: count_A = 0  
count_B = 0  
count_AB = 0  
  
for t in transactions:  
    if A in t:  
        count_A += 1  
    if B in t:  
        count_B += 1  
    if A in t and B in t:  
        count_AB += 1  
  
support = count_AB / total_transactions  
confidence = count_AB / count_A if count_A != 0 else 0  
lift = confidence / (count_B / total_transactions) if count_B != 0 else 0  
  
print("Support:", support)  
print("Confidence:", confidence)  
print("Lift:", lift)  
  
Support: 0.004362934362934363  
Confidence: 0.04908774978279757  
Lift: 4.384043859911921
```

LAB-6: Apriori Algorithm

1. Objective

The objective of this lab is to understand, implement, and analyze the systematic mechanics of the **Apriori Algorithm**. Students will learn how to identify frequent product combinations and generate reliable association rules by enforcing minimum support and confidence thresholds using Python.

2. Tools & Datasets

- **Tool:** Python / KNIME / RapidMiner
- **Dataset:** Transaction Dataset

3. Background Concepts

While Market Basket Analysis defines the core metrics (Support, Confidence, Lift), the **Apriori Algorithm** provides an efficient algorithmic approach to mine those rules out of massive datasets.

The biggest challenge in association rule mining is the combinatorial explosion: if a store has d unique items, there are 2^d possible itemsets to check. Apriori solves this using the **Apriori Principle (Downward Closure Property)**:

Any subset of a frequent itemset must also be frequent. Conversely, if an itemset is infrequent, all of its supersets will automatically be infrequent and can be **pruned** without ever being tested.

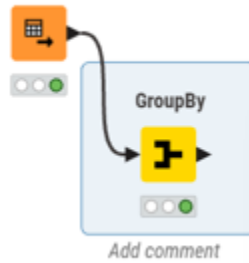
The algorithm operates in a two-step iterative cycle:

1. **Candidate Generation (SC_k):** Joining frequent itemsets from the previous pass (L_{k-1}) to create larger candidate combinations.
2. **Support Counting and Pruning (L_k):** Scanning the database to calculate the actual support of candidates, keeping only those that satisfy the minimum support threshold (min_sup).

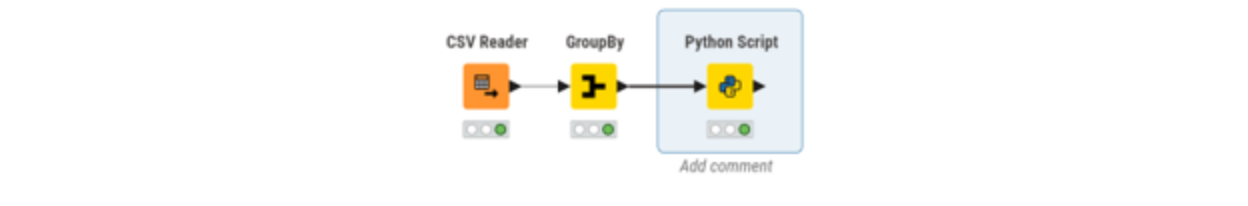


Rows: 2512 Columns: 16					
☐	#	RowID	Transacti... T String	AccountID T String	Transacti... Number (...)
☐	1	Row0	TX000001	AC00128	14.09
☐	2	Row1	TX000002	AC00455	376.24
☐	3	Row2	TX000003	AC00019	126.29
☐	4	Row3	TX000004	AC00070	184.5
☐	5	Row4	TX000005	AC00411	13.45
☐	6	Row5	TX000006	AC00393	92.15
☐	7	Row6	TX000007	AC00199	7.08
☐	8	Row7	TX000008	AC00069	171.42
☐	9	Row8	TX000009	AC00135	106.23

CSV Reader



Rows: 495 Columns: 2				
☐	#	RowID	AccountID T String	First(MerchantID) T String
☐	1	Row0	AC00001	M034
☐	2	Row1	AC00002	M040
☐	3	Row2	AC00003	M089
☐	4	Row3	AC00004	M064
☐	5	Row4	AC00005	M036
☐	6	Row5	AC00006	M026
☐	7	Row6	AC00007	M060



► 1: Table Flow Variables

Rows: 495 | Columns: 101

Table Statistics

#	RowID	AccountID	M001	M002	M003	M004	M005	M006	M007	M008	M009	M010
1	Row0	AC00001	0	0	1	0	0	0	0	0	0	0
2	Row1	AC00002	0	0	0	0	0	0	0	0	0	0
3	Row2	AC00003	0	0	0	0	0	0	0	0	0	0
4	Row3	AC00004	0	0	0	0	0	0	0	0	0	0
5	Row4	AC00005	0	1	0	0	0	0	0	0	0	1



► 1: Table Flow Variables

Rows: 45 | Columns: 5

Table Statistics

#	RowID	antecedents	consequents	support	confidence	lift
1	Row0	M001	M009	0.012	0.194	3.194
2	Row1	M001	M038	0.01	0.161	2.661
3	Row2	M001	M052	0.012	0.194	3.685
4	Row3	M001	M070	0.01	0.161	2.661
5	Row4	M004	M026	0.01	0.167	1.875
6	Row5	M004	M047	0.01	0.167	2.946
7	Row6	M005	M030	0.01	0.161	3.071
8	Row7	M009	M065	0.012	0.2	3.194
9	Row8	M011	M064	0.01	0.192	3.526

Importing libraries:

```
[2]: import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

Load Your Dataset

```
[3]: df = pd.read_csv('transaction_data.csv')
df.head()
```

	Userid	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

```
[6]: df = df.dropna()

# Remove negative or zero quantities (if exist)
df = df[df['NumberOfItemsPurchased'] > 0]

df.head()
```

	UserId	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Convert into Basket Format

```
[7]: basket = df.groupby(['TransactionId', 'ItemDescription'])['NumberOfItemsPurchased'] \
        .sum().unstack().reset_index().fillna(0)

basket.head()
```

	ItemDescription	TransactionId	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	...	returned	taig adjust	test
	0	5900015	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	1	5900026	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	2	5900037	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	3	5900048	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	4	5900059	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0

5 rows × 4078 columns

Convert Values into 0/1 (Binary Encoding)

```
[8]: def encode(x):
    if x > 0:
        return 1
    else:
        return 0

basket_encoded = basket.set_index('TransactionId').applymap(encode)

basket_encoded.head()
```

C:\Users\star\AppData\Local\Temp\ipykernel_11340\1282610810.py:7: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
basket_encoded = basket.set_index('TransactionId').applymap(encode)

	ItemDescription	TransactionId	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	SPACEBOY BABY GIFT SET	...	returned	taig adjust	test
	5900015	0	0	0	0	0	0	0	0	0	0	0	...	0	0	0
	5900026	0	0	0	0	0	0	0	0	0	0	0	...	0	0	0

Apply Apriori Algorithm

```
[10]: rules = association_rules(frequent_itemsets,
    metric="confidence",
    min_threshold=0.6)

rules.head()
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
--	-------------	-------------	-----------------------	-----------------------	---------	------------	------	------------------	----------	------------	---------------	---------	-----------	------------

Display Important Rules

```
[11]: rules = rules.sort_values(by='confidence', ascending=False)
      rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]

[11]: antecedents  consequents  support  confidence  lift
```

Filter Strong Rules

```
[12]: strong_rules = rules[(rules['confidence'] > 0.7) & (rules['lift'] > 1)]
      strong_rules

[12]: antecedents  consequents  antecedent consequent support confidence lift representativity leverage conviction zhangs_metric jaccard certainty kulczynski
      support      support
```

Support threshold analysis

Support Threshold	Frequent Single Items	Frequent Item Pairs	Resulting Rules
0.01 (1%)	100	45	Multiple rules generated
0.03 (3%)	98	0	No rules found
0.05 (5%)	49	0	No rules found

LAB-7: FP-Tree & FP-Growth

1. Objective

The objective of this lab is to implement and analyze the **FP-Growth (Frequent Pattern Growth)** algorithm for efficient frequent pattern mining. Students will construct an FP-Tree pipeline visually using Orange Data Mining, compare its algorithmic execution performance with the Apriori algorithm, and extract high-confidence transactional rules.

2. Tools & Datasets

- **Tool:** Orange Data Mining + Python Validation
- **Dataset:** Large Retail Dataset / Grocery Transactions

3. Background Concepts

While the Apriori algorithm relies on a "candidate generation-and-test" approach that requires scanning the database multiple times, the **FP-Growth algorithm** introduces a radical performance upgrade. It adopts a **divide-and-conquer** strategy that encodes the database into a compact data structure called an **FP-Tree (Frequent Pattern Tree)**.

- **No Candidate Generation:** FP-Growth eliminates the costly candidate generation phase entirely, avoiding the combinatorial explosion of checking millions of item combinations.
- **Database Scans:** It scans the database exactly **twice**:
 1. *Scan 1:* Counts the support of individual items and sorts them in descending order of frequency.
 2. *Scan 2:* Constructs the FP-Tree by mapping transactions directly onto overlapping paths.
- **Conditional FP-Trees:** The algorithm extracts frequent itemsets directly from the tree structure by traversing it bottom-up from the least frequent items using suffix patterns.

Retail Analysis on Large Dataset

60

<> Code

Download

Data Card Code (14) Discussion (1) Suggestions (0)

new_retail_data.csv (84.92 MB)



Detail Compact Column

10 of 30 columns

About this file

Suggest Edits

Transaction_ID:

- Type: int64
- Explanation: Unique identifier for each transaction.

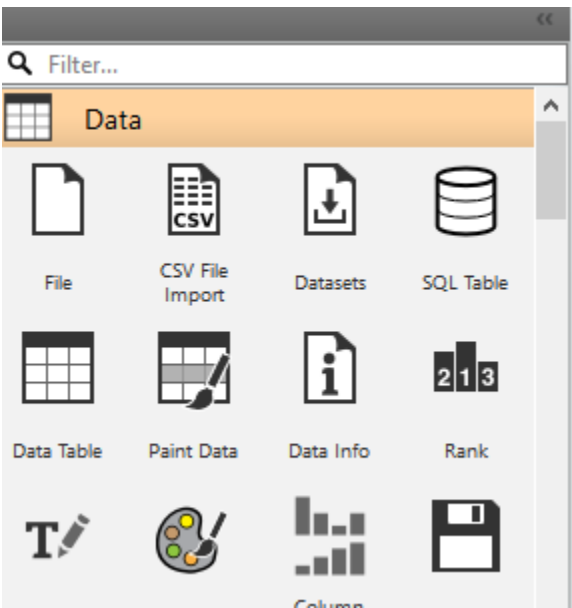
Data Explorer

Version 1 (84.92 MB)

new_retail_data.csv

Summary

- 1 file
- 30 columns



File

Source

☒ File: data mining\retail_sales_dataset.csv  ...  Reload

☐ URL:

File Type

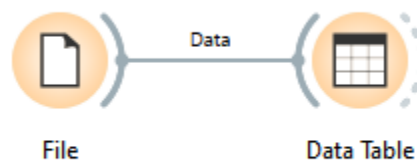
Determine type from the file extension

Info

1000 instances
8 features (no missing values)
Data has no target variable.
1 meta attribute

Columns (Double click to edit)

	Name	Type	Role	Values
1	Transaction ID	 numeric	feature	
2	Date	 datetime	feature	
3	Gender	 categorical	feature	Female, Male
4	Age	 numeric	feature	
5	Product Category	 categorical	feature	Beauty, Clothing, Electronics
6	Quantity	 numeric	feature	
7	Price per Unit	 numeric	feature	



Info

1001 instances (no missing data)
2362 features (sparse, density 100.00 %)
No target variable.
No meta attributes.

Variables

☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection

Clear Selection

☒ Select full rows

Restore Original Order

☒ Send Automatically

	{...}
1	Transaction ID=...
2	1=1, 2023-11-2...
3	2=2, 2023-02-2...
4	1=1, Male=1, 3...
5	1=1, Male=1, Cl...
6	Male=1, Beauty...
7	1=1, Beauty=1, ...
8	Male=1, 50=1, ...
9	Male=1, Electro...
10	Male=1, 2=1, El...
11	50=1, Female=...
12	Male=1, 50=1, ...
13	Male=1, Beauty...
14	Male=1, 3=1, 5...
15	Male=1, Clothe...
16	Female=1, 500...
17	Male=1, 3=1, Cl...
18	Female=1, Clot...
19	50=1, 2=1, Fem...
20	50=1, 2=1, Fem...
21	Male=1, 3=1, Cl...
22	1=1, Beauty=1, ...
23	Male=1, 50=1, ...

?

→

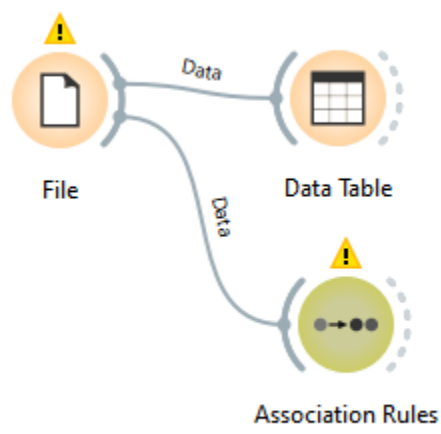
1001

↔

1001

|

1001



Association Rules - Orange

Info
Rules: 14 (shown 14)

Find association rules

Min. supp.: 10 %

Min. conf.: 40 %

Max. rules: 10k

☐ Induce only classification rules

☐ Restrict search by below filters

Find Rules

Filter by Antecedent

Contains:

Items, min: 1 max: 999

Filter by Consequent

Contains:

Items, min: 1 max: 999

☒ Send selection

Supp	Conf	Covr	Strg	$\hat{Lift}$	Levr	Antecedent	Consequent
0.141	0.459	0.307	1.596	0.937	-0.009	Product Category=Beauty	Gender=Male
0.114	0.473	0.241	2.033	0.965	-0.004	Quantity=3	Gender=Male
0.120	0.494	0.243	2.099	0.968	-0.004	Quantity=2	Gender=Female
0.125	0.494	0.253	2.016	0.969	-0.004	Quantity=1	Gender=Female
0.125	0.475	0.263	1.863	0.970	-0.004	Quantity=4	Gender=Male
0.174	0.496	0.351	1.453	0.972	-0.005	Product Category=Clothing	Gender=Female
0.170	0.497	0.342	1.491	0.975	-0.004	Product Category=Electronics	Gender=Female
0.172	0.503	0.342	1.433	1.026	0.004	Product Category=Electronics	Gender=Male
0.138	0.525	0.263	1.939	1.029	0.004	Quantity=4	Gender=Female
0.177	0.504	0.351	1.396	1.029	0.005	Product Category=Clothing	Gender=Male
0.128	0.506	0.253	1.937	1.033	0.004	Quantity=1	Gender=Male
0.123	0.506	0.243	2.016	1.033	0.004	Quantity=2	Gender=Male
0.127	0.527	0.241	2.116	1.033	0.004	Quantity=3	Gender=Female
0.166	0.541	0.307	1.661	1.060	0.009	Product Category=Beauty	Gender=Female

```
[1]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import fpgrowth, association_rules
```

```
[3]: df = pd.read_csv("retail_dataset.csv")
```

```
[7]: df.head()
```

```
[7]:
```

	TransactionID	Item
0	T1	Laptop
1	T1	Mouse
2	T1	Laptop Bag
3	T2	Headphones
4	T2	Mouse

Convert into Transaction Format

```
[8]: transactions = df.groupby('TransactionID')['Item'].apply(list)

# Convert to list of lists
transactions = list(transactions)

# Show sample
transactions[:5]

[8]: [['Laptop', 'Mouse', 'Laptop Bag'],
      ['Laptop', 'Mouse', 'USB Hub'],
      ['Printer', 'Paper Pack'],
      ['Monitor', 'Mouse'],
      ['Camera', 'Memory Card']]
```

One-Hot Encoding

```
[9]: te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

df_encoded = pd.DataFrame(te_array, columns=te.columns_)

df_encoded.head()
```

```
[9]:
```

	Bluetooth Adapter	Camera	Camera Bag	Cooling Pad	HDMI Cable	Headphones	Ink Cartridge	Keyboard	Laptop	Laptop Bag	...	Paper Pack	Phone Case	Printer	Screen Protector	Smartphone	Stylus Pen	Tablet
0	False	False	False	False	False	False	False	False	True	True	...	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	True	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False
4	False	True	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False

5 rows × 23 columns

Apply FP-Growth

```
[13]: # FP-Growth
frequent_itemsets = fpgrowth(df_encoded, min_support=0.1, use_colnames=True)

print(frequent_itemsets)
```

	support	itemsets
0	0.333333	(Mouse)
1	0.222222	(Laptop)
2	0.111111	(Printer)
3	0.166667	(Monitor)
4	0.166667	(Camera)
5	0.166667	(Keyboard)
6	0.111111	(Tablet)
7	0.111111	(Smartphone)
8	0.111111	(Headphones)
9	0.166667	(Mouse, Laptop)
10	0.111111	(Mouse, Monitor)
11	0.111111	(Keyboard, Mouse)

Generate Association Rules

```
[15]: # Association Rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.3)

display(rules)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
0	(Mouse)	(Laptop)	0.333333	0.222222	0.166667	0.500000	2.25	1.0	0.092593	1.555556	0.833333	0.428571	0.357143	0.62
1	(Laptop)	(Mouse)	0.222222	0.333333	0.166667	0.750000	2.25	1.0	0.092593	2.666667	0.714286	0.428571	0.625000	0.62
2	(Mouse)	(Monitor)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50
3	(Monitor)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
4	(Keyboard)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
5	(Mouse)	(Keyboard)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50

```
[16]: rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

	antecedents	consequents	support	confidence	lift
0	(Mouse)	(Laptop)	0.166667	0.500000	2.25
1	(Laptop)	(Mouse)	0.166667	0.750000	2.25
2	(Mouse)	(Monitor)	0.111111	0.333333	2.00
3	(Monitor)	(Mouse)	0.111111	0.666667	2.00
4	(Keyboard)	(Mouse)	0.111111	0.666667	2.00
5	(Mouse)	(Keyboard)	0.111111	0.333333	2.00

LAB-8: Python Basics for Data Mining

1. Objective

The objective of this lab is to introduce students to the fundamentals of Python programming. Students will learn how to set up an Integrated Development Environment (IDE), work with core data types, implement conditional logic, use loops, and write basic functions.

Python Syntax

Python is a high-level programming language widely used in data mining due to its simplicity and readability. It uses indentation (spacing) to define code blocks instead of brackets.

Example:

```
[8]: print("Data Mining is powerful")  
Data Mining is powerful
```

Lists and Dictionaries

- **Lists:**
Used to store ordered collections of items

```
[4]: data = [10, 20, 30, 40]
```

- **Dictionaries:**
Store data in key-value pairs

```
[6]: student = {'name': 'Sara', 'marks': 85}
```

Example:

A dataset can be stored as a list of dictionaries where each dictionary represents a record.

OOP Basics (Object-Oriented Programming)

Object-Oriented Programming (OOP) is a programming approach that organizes code using objects and classes.

Key concepts include:

- **Classes:** Blueprints for creating objects
- **Objects:** Instances of classes

- **Encapsulation:** Hiding internal details
- **Inheritance:** Reusing code from existing classes
- **Polymorphism:** Using the same method in different ways

Example:

A class **Student** can have attributes like name and marks, and methods like `calculate_grade()`.

Step 1: Import Libraries

```
[1]: import pandas as pd
```

Step 2: Load Dataset

```
[2]: data = pd.read_csv("Loan Prediction.zip")
```

```
[3]: data.head()
```

	Id	Income	Age	Experience	Married/Single	House_Ownership	Car_Ownership	Profession	CITY	STATE	CURRENT_JOB_YRS
0	1	1303834	23	3	single	rented	no	Mechanical_engineer	Rewa	Madhya_Pradesh	3
1	2	7574516	40	10	single	rented	no	Software_Developer	Parbhani	Maharashtra	9
2	3	3991815	66	4	married	rented	no	Technical_writer	Alappuzha	Kerala	4
3	4	6256451	41	2	single	rented	yes	Software_Developer	Bhubaneswar	Odisha	2
4	5	5768871	47	11	single	rented	no	Civil_servant	Tiruchirappalli[10]	Tamil_Nadu	3

Step 3: Create a Class (Reusable Module)

```
[4]: class DataProcessor:

    def __init__(self, dataframe):
        self.df = dataframe

    def show_info(self):
        return self.df.info()

    def handle_missing(self):
        return self.df.fillna(self.df.mean(numeric_only=True))

    def remove_duplicates(self):
        return self.df.drop_duplicates()

    def clean_columns(self):
        self.df.columns = [col.lower() for col in self.df.columns]
        return self.df

    def summary(self):
        return self.df.describe()
```

Step 4: Create Object of Class

```
[5]: processor = DataProcessor(data)
```

Step 5: Apply Preprocessing Methods

```
[6]: data = processor.clean_columns()
data = processor.handle_missing()

# Remove duplicates
data = processor.remove_duplicates()
```

Step 6: Use Functions (Testing Reusability)

```
[7]: # Show dataset info
processor.show_info()
```

```
# Show summary
processor.summary()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 252000 entries, 0 to 251999
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  --
 0   id                    252000 non-null int64
 1   income                252000 non-null int64
 2   age                  252000 non-null int64
 3   experience            252000 non-null int64
 4   married/single       252000 non-null object
 5   house_ownership      252000 non-null object
 6   car_ownership        252000 non-null object
 7   profession           252000 non-null object
 8   city                 252000 non-null object
 9   state                252000 non-null object
10   current_job_yrs      252000 non-null int64
11   current_house_yrs    252000 non-null int64
```

[7]:

	id	income	age	experience	current_job_yrs	current_house_yrs	risk_flag
count	252000.000000	2.520000e+05	252000.000000	252000.000000	252000.000000	252000.000000	252000.000000
mean	126000.500000	4.997117e+06	49.954071	10.084437	6.333877	11.997794	0.123000
std	72746.278255	2.878311e+06	17.063855	6.002590	3.647053	1.399037	0.328438
min	1.000000	1.031000e+04	21.000000	0.000000	0.000000	10.000000	0.000000
25%	63000.750000	2.503015e+06	35.000000	5.000000	3.000000	11.000000	0.000000
50%	126000.500000	5.000694e+06	50.000000	10.000000	6.000000	12.000000	0.000000
75%	189000.250000	7.477502e+06	65.000000	15.000000	9.000000	13.000000	0.000000
max	252000.000000	9.999938e+06	79.000000	20.000000	14.000000	14.000000	1.000000

Step 7: Use Lists & Dictionaries (Requirement)

```
[8]: # List example: column names
columns_list = list(data.columns)
print(columns_list)
```

```
# Dictionary example: data types
dtype_dict = dict(data.dtypes)
print(dtype_dict)
```

```
['id', 'income', 'age', 'experience', 'married/single', 'house_ownership', 'car_ownership', 'profession', 'city', 'state', 'current_job_yrs', 'current_h
ouse_yrs', 'risk_flag']
{'id': dtype('int64'), 'income': dtype('int64'), 'age': dtype('int64'), 'experience': dtype('int64'), 'married/single': dtype('O'), 'house_ownership': d
type('O'), 'car_ownership': dtype('O'), 'profession': dtype('O'), 'city': dtype('O'), 'state': dtype('O'), 'current_job_yrs': dtype('int64'), 'current_h
ouse_yrs': dtype('int64'), 'risk_flag': dtype('int64')}
```

LAB-9: Decision Tree

1. Objective

The objective of this lab is to implement and analyze the **Decision Tree** algorithm for predictive customer analytics. Students will construct a classification pipeline visually using KNIME Analytics Platform, validate the results programmatically using Python, and evaluate structural predictive metrics to determine consumer purchasing behavior.

2. Tools & Datasets

- **Tool:** KNIME Analytics Platform + Python Validation
- **Dataset:** Marketing Dataset

3. Background Concepts

While non-parametric models partition data spaces recursively, the **Decision Tree** algorithm introduces a highly interpretable, flowchart-like tree structure to solve classification tasks. It splits a heterogeneous dataset into increasingly homogenous subsets based on feature attributes.

- **Information Gain & Entropy:** Trees utilize mathematical metrics like Entropy to calculate impurity within a node. By choosing splits that maximize Information Gain, the algorithm minimizes the uncertainty of the resulting child branches.
- **Gini Impurity:** Often used as an alternative splitting criterion, Gini measures how often a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset.
- **Overfitting and Pruning:** Decision trees can easily grow overly complex, capturing random noise in the training data.

marketing_data.csv

8

Download

Data Card Code (2) Discussion (0) Suggestions (0)

CC GENERAL.csv (902.8...)

Detail Compact Column 10 of 18 columns

About this file

Suggest Edits

This file does not have a description yet.

CUST_ID BALANCE

Data Explorer

Version 1 (902.88 kB)

CC GENERAL.csv



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
```

1. Load the Marketing Dataset

```
df = pd.read_csv('CC_GENERAL.csv')
```

```
np.random.seed(42)
n_samples = 300
data = {
    'Age': np.random.randint(18, 65, n_samples),
    'EstimatedSalary': np.random.randint(20000, 150000, n_samples),
    'Purchased': np.random.choice([0, 1], n_samples, p=[0.6, 0.4]) # Simulate some purchases
}
df = pd.DataFrame(data)
print("Generated synthetic marketing data:")
print(df.head())
print("\n")
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
```

Generated synthetic marketing data:

	Age	EstimatedSalary	Purchased
0	56	58467	1
1	46	43328	1
2	32	139181	0
3	60	111412	0
4	25	106831	0

2. Split Data (80% Train, 20% Test)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, strat
```

3. Initialize and Train Decision Tree Classifier

```
clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)
```

DecisionTreeClassifier

DecisionTreeClassifier(max_depth=4, random_state=42)

4. Predict on Test Set

```
y_pred = clf.predict(X_test)
```

5. Evaluate Metrics

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("=== Evaluation Metrics ===")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-Score : {f1:.4f}\n")
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
=== Evaluation Metrics ===
Accuracy : 0.6833
Precision: 0.5600
Recall   : 0.6364
F1-Score : 0.5957
```

```
Classification Report:
              precision    recall  f1-score   support

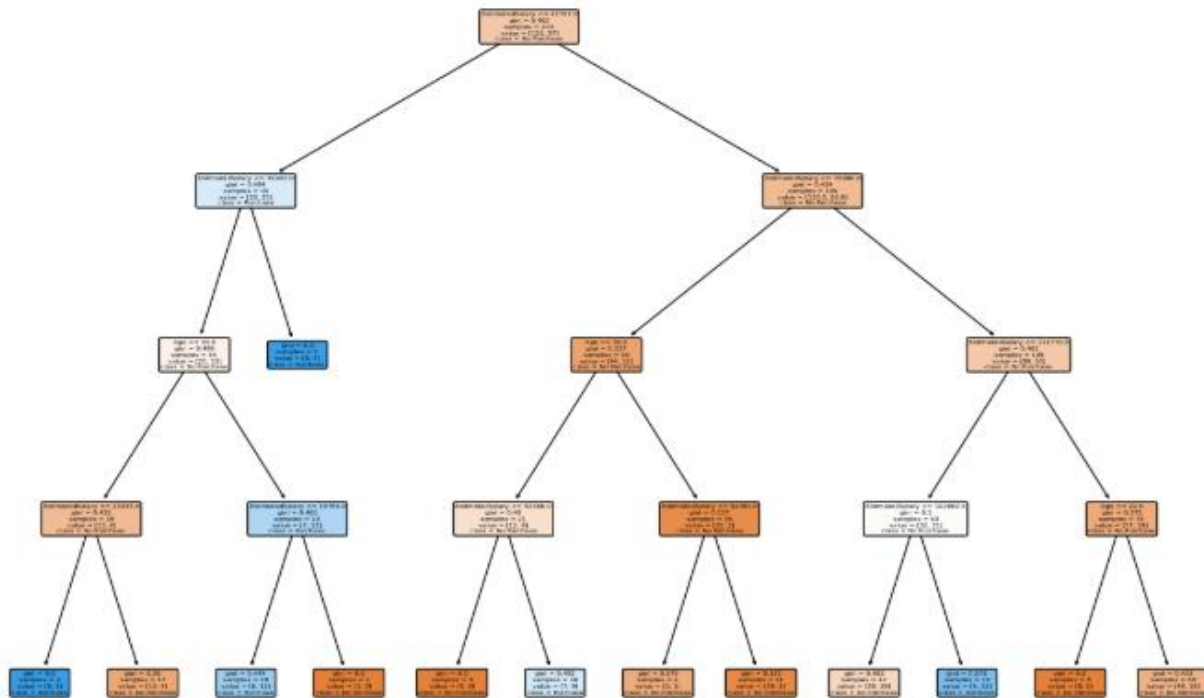
     0       0.77         0.71         0.74         38
     1       0.56         0.64         0.60         22

   accuracy          0.68
  macro avg          0.67
 weighted avg          0.69
```

6. Visualize the Decision Tree

```
plt.figure(figsize=(15, 10))
plot_tree(clf, feature_names=X.columns, class_names=['No Purchase', 'Purchase'], filled=True,
plt.title("Decision Tree Structure for Marketing Insights")
plt.show()
```

Decision Tree Structure for Marketing Insights



LAB-10: Naïve Bayes & KNN

1. Objective

The objective of this lab is to implement and compare the performance of **Naïve Bayes** and **K-Nearest Neighbors (KNN)** classifiers. Students will construct a text/tabular classification workflow using RapidMiner Studio, validate operations in Python, and benchmark comparative error dynamics on a spam filtering dataset.

2. Tools & Datasets

- **Tool:** RapidMiner Studio + Python Validation
- **Dataset:** Spam Dataset

3. Background Concepts

Classification architectures handle boundary distributions differently depending on whether they leverage probabilistic assumptions or spatial metrics.

- **Naïve Bayes Classifier:** A probabilistic model founded upon Bayes' Theorem. It operates on the "naïve" assumption of class conditional independence, meaning the presence of a specific feature in a class is assumed to be completely unrelated to the presence of any other feature.
- **K-Nearest Neighbors (KNN):** A lazy-learning, instance-based algorithm that skips explicit training phases. It classifies an unlabelled vector by calculating the geometric distance (such as Euclidean metric) between that query point and all historical records, assigning the majority class of its K nearest neighbors.

spam.csv

14

Download



Data Card Code (15) Discussion (0) Suggestions (0)

Tags

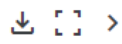
Email and Messaging

Data Explorer

Version 1 (503.66 kB)

spam.csv

spam.csv (503.66 kB)

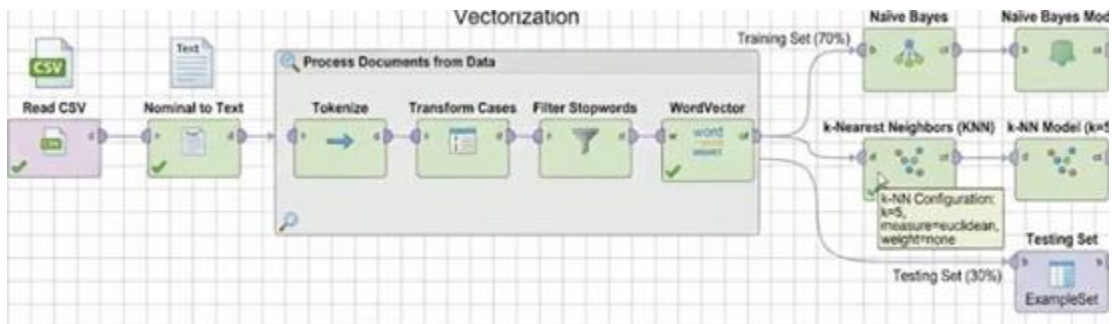


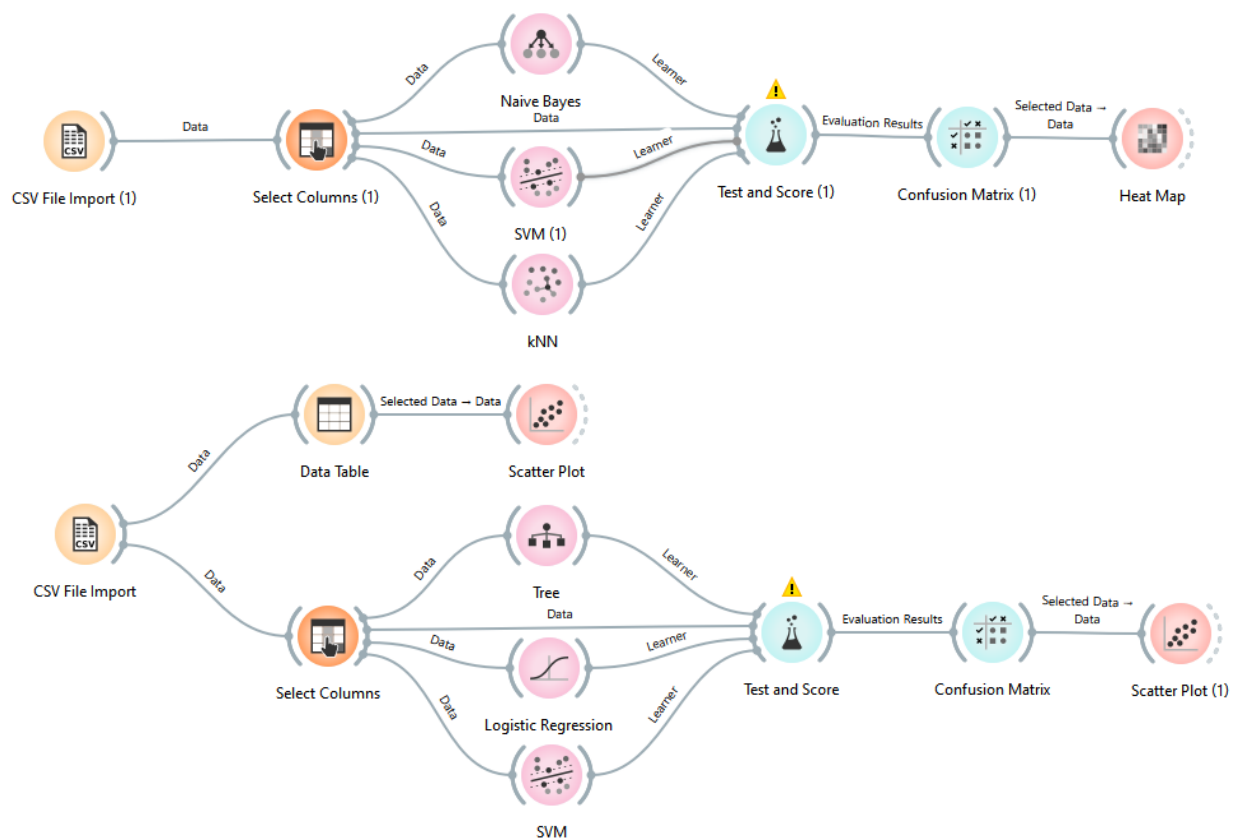
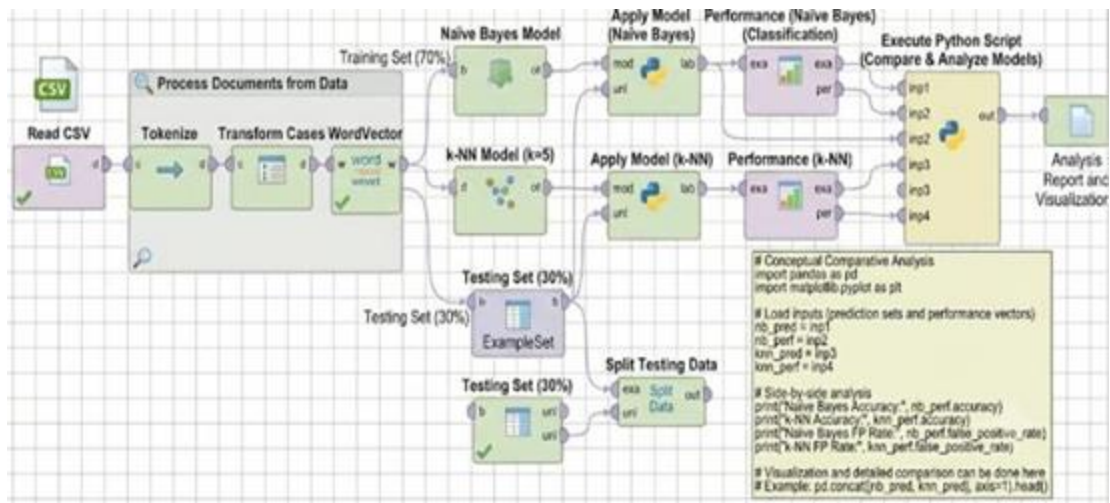
Detail Compact Column 4 of 4 columns

About this file

Suggest Edits

This file does not have a description yet.





```
import pandas as pd
```



```
# Load dataset
```

```
df = pd.read_csv('spam.csv.csv', encoding='latin-1')
```

```
print("Dataset Shape:", df.shape)
```

```
print("\nColumns:")
```

```
print(df.columns)
```

```
print("\nFirst 5 Rows:")
```

```
print(df.head())
```

Dataset Shape: (5573, 1)

Columns:

Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')

First 5 Rows:

ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...

0 ham\tOk lar... Joking wif u oni...

1 spam\tFree entry in 2 a wkly comp to win FA Cup...

2 ham\tU dun say so early hor... U c already the...

3 ham\tNah I don't think he goes to usf, he live...

4 spam\tFreeMsg Hey there darling it's been 3 we...

```
extra_cols = ['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']
```

```
for col in extra_cols:
```

```
    if col in df.columns:
```

```
        df.drop(col, axis=1, inplace=True)
```

```
# Rename columns if using common Kaggle spam dataset
```

```
if 'v1' in df.columns and 'v2' in df.columns:
```

```
    df.rename(columns={'v1': 'label', 'v2': 'message'}, inplace=True)
```

```
print(df.head())
```

```
print(df.columns)
```

```

ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cin
e there got amore wat...
0          ham\tOk lar... Joking wif u oni...
1  spam\tFree entry in 2 a wkly comp to win FA Cu...
2  ham\tU dun say so early hor... U c already the...
3  ham\tNah I don't think he goes to usf, he live...
4  spam\tFreeMsg Hey there darling it's been 3 we...
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffe
t... Cine there got amore wat...'], dtype='object')

```

```
print(df.columns)
```

```

Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffe
t... Cine there got amore wat...'], dtype='object')

```

```

import pandas as pd

df_raw = pd.read_csv(
    'spam.csv.csv',
    header=None,
    quotechar='"' # Handle the outer double quotes
)

df = df_raw[0].str.split('\t', n=1, expand=True)
df.columns = ['label', 'message'] # Assign new column names

print(df.head())
print(df.columns)

```

```

label          message
0  ham  Go until jurong point, crazy.. Available only ...
1  ham          Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...
Index(['label', 'message'], dtype='object')

```

```

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Convert Labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Features and target
X = df['message']
y = df['label']

# Convert text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)

# Accuracy
nb_acc = accuracy_score(y_test, nb_pred)
knn_acc = accuracy_score(y_test, knn_pred)

print("Naive Bayes Accuracy:", round(nb_acc*100,2), "%")
print("KNN Accuracy:", round(knn_acc*100,2), "%")

# Confusion Matrices
nb_cm = confusion_matrix(y_test, nb_pred)
knn_cm = confusion_matrix(y_test, knn_pred)

print("\nNaive Bayes Confusion Matrix")
print(nb_cm)

print("\nKNN Confusion Matrix")
print(knn_cm)

```

```
Naive Bayes Accuracy: 98.48 %  
KNN Accuracy: 90.85 %
```

```
Naive Bayes Confusion Matrix  
[[944  10]  
 [  7 154]]
```

```
KNN Confusion Matrix  
[[954   0]  
 [102  59]]
```

```
: import pandas as pd  
  
def false_positive_rate(cm):  
    TN = cm[0][0]  
    FP = cm[0][1]  
    return FP / (FP + TN)  
  
nb_fpr = false_positive_rate(nb_cm)  
knn_fpr = false_positive_rate(knn_cm)  
  
results = pd.DataFrame({  
    'Algorithm': ['Naive Bayes', 'KNN'],  
    'Accuracy (%)': [  
        round(nb_acc*100,2),  
        round(knn_acc*100,2)  
    ],  
    'False Positive Rate': [  
        round(nb_fpr,4),  
        round(knn_fpr,4)  
    ]  
})
```

```
print(results)
```

	Algorithm	Accuracy (%)	False Positive Rate
0	Naive Bayes	98.48	0.0105
1	KNN	90.85	0.0000

LAB-11: Support Vector Machine (SVM)

1. Objective

The objective of this lab is to implement and assess **Support Vector Machines (SVM)** for high-dimensional risk modeling. Students will map decision boundaries using KNIME Analytics Platform, leverage Python validation libraries, and investigate how kernel tricks isolate fraudulent transactions.

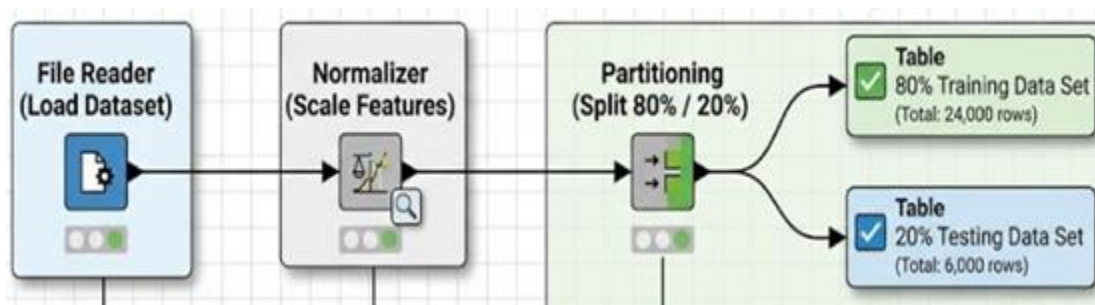
2. Tools & Datasets

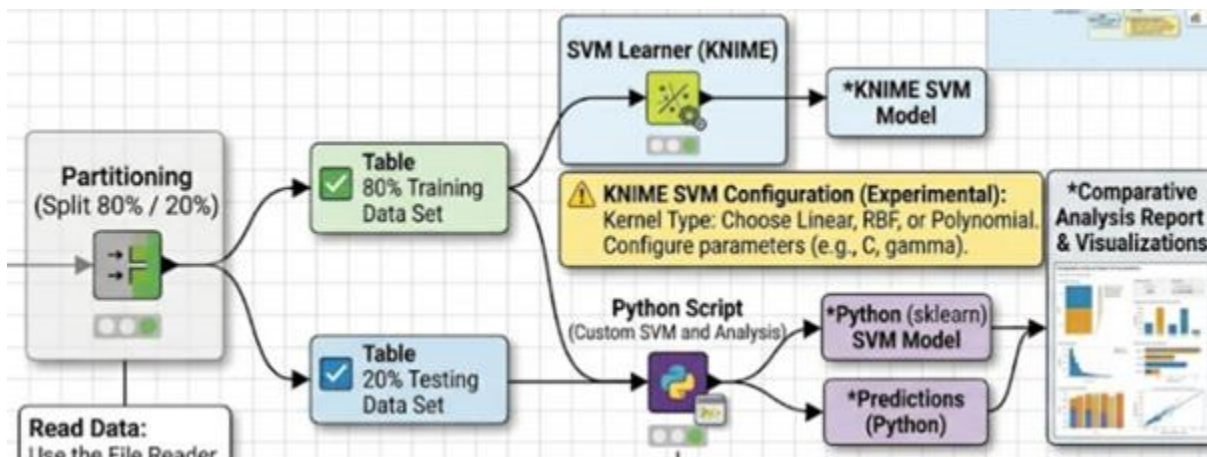
- **Tool:** KNIME Analytics Platform + Python Validation
- **Dataset:** Fraud Detection Dataset

3. Background Concepts

When working with complex datasets where categories overlap, linear partitioning is insufficient. **Support Vector Machines (SVM)** handle this by finding a boundary that cleanly separates classes while maximizing spatial margins.

- **Hyperplanes and Support Vectors:** The algorithm searches for an optimal decision hyperplane that maximizes the geometric distance (margin) between the closest data points of opposing classes, known as support vectors.
- **The Kernel Trick:** For non-linearly separable data, SVM applies mathematical kernels to transform the lower-dimensional input space into a higher-dimensional feature space where a linear boundary can separate the classes.
- **Linear vs. RBF Kernels:** A Linear kernel splits data along straight vector cuts. The Radial Basis Function (RBF) kernel measures localized structural distances, projecting non-linear circular or localized boundaries.





```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, recall_score

```

1. Simulate a Highly Imbalanced Fraud Detection Dataset

```

np.random.seed(42)
n_samples = 2000
normal_tx = np.random.normal(loc=50, scale=10, size=(1950, 3))
fraud_tx = np.random.normal(loc=200, scale=50, size=(50, 3))
X = np.vstack([normal_tx, fraud_tx])
y = np.array([0] * 1950 + [1] * 50)
columns = ['Transaction_Amount', 'Frequency_Per_Day', 'Location_Risk_Score']
df = pd.DataFrame(X, columns=columns)
df['Class'] = y

```

2. Separate Features and Target Variable

```
X_features = df[columns]
y_target = df['Class']
```

3. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_features, y_target, test_size=0.25, random_state=42, stratify=y_target
)
```

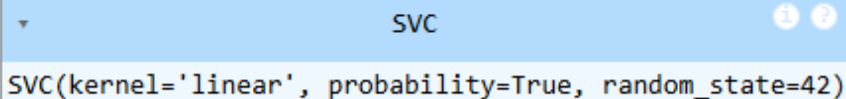
4. Feature Scaling (Standardization)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5. Initialize and Train SVM with a Linear Kernel

```
print("Training Support Vector Machine with Linear Kernel...")
svm_linear = SVC(kernel='linear', C=1.0, probability=True, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with Linear Kernel...

A screenshot of a Jupyter Notebook cell. The cell has a blue header bar with a dropdown arrow, the text 'SVC', and two circular icons (one with an 'i' and one with a '?'). Below the header, the text 'SVC(kernel='linear', probability=True, random_state=42)' is displayed.

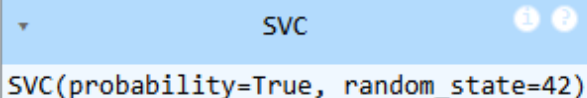
SVC

SVC(kernel='linear', probability=True, random_state=42)

6. Initialize and Train SVM with a Radial Basis Function (RBF) Kernel

```
print("Training Support Vector Machine with RBF Kernel...")
svm_rbf = SVC(kernel='rbf', gamma='scale', C=1.0, probability=True, random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with RBF Kernel...

A screenshot of a Jupyter Notebook cell. The cell has a blue header bar with a dropdown arrow, the text 'SVC', and two circular icons (one with an 'i' and one with a '?'). Below the header, the text 'SVC(probability=True, random_state=42)' is displayed.

SVC

SVC(probability=True, random_state=42)

7. Generate Evaluation Predictions and Probability Estimates

```
lin_preds = svm_linear.predict(X_test_scaled)
lin_probs = svm_linear.predict_proba(X_test_scaled)[: , 1]

rbf_preds = svm_rbf.predict(X_test_scaled)
rbf_probs = svm_rbf.predict_proba(X_test_scaled)[: , 1]
```

8. Display Runtime KPI Performance Report

```
print(" LAB 11: SVM KERNEL COMPARISON MATRIX          ")

print(f" -> Fraud Recall Rate : {recall_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> Precision Score   : {precision_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score      : {roc_auc_score(y_test, lin_probs):.4f}%")
print(f" -> Confusion Matrix  : \n{confusion_matrix(y_test, lin_preds)}")
print("\n")
print(f"[RBF KERNEL]")
print(f" -> Fraud Recall Rate : {recall_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> Precision Score   : {precision_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score      : {roc_auc_score(y_test, rbf_probs):.4f}%")
print(f" -> Confusion Matrix  : \n{confusion_matrix(y_test, rbf_preds)}")
```

LAB 11: SVM KERNEL COMPARISON MATRIX

-> Fraud Recall Rate : 100.00%
-> Precision Score : 100.00%
-> ROC-AUC Score : 1.0000
-> Confusion Matrix :

```
[[488   0]
 [   0  12]]
```

[RBF KERNEL]

-> Fraud Recall Rate : 100.00%
-> Precision Score : 100.00%
-> ROC-AUC Score : 1.0000
-> Confusion Matrix :

```
[[488   0]
 [   0  12]]
```

```
print("\nDetailed RBF Classification Context Matrix:")
print(classification_report(y_test, rbf_preds, target_names=['Legitimate', 'Fraud']))
```

Detailed RBF Classification Context Matrix:

	precision	recall	f1-score	support
Legitimate	1.00	1.00	1.00	488
Fraud	1.00	1.00	1.00	12
accuracy			1.00	500
macro avg	1.00	1.00	1.00	500
weighted avg	1.00	1.00	1.00	500

LAB-12: Clustering Algorithms

1. Objective

The objective of this lab is to implement and analyze unsupervised **K-Means** and **Hierarchical Clustering** algorithms. Students will construct a customer segmentation visualization pipeline inside Orange Data Mining, validate structural cohesion with Python, and evaluate cluster separation profiles.

2. Tools & Datasets

- **Tool:** Orange Data Mining + Python Validation
- **Dataset:** Mall Customer Dataset

3. Background Concepts

Unsupervised data mining does not rely on predefined labels. Instead, it uncovers hidden structures within a dataset by grouping records based on feature similarity.

- **K-Means Clustering:** A centroid-based algorithm that partitions data into K distinct clusters. It iteratively updates cluster centroids by minimizing the within-cluster Sum of Squared Errors (SSE).
- **Hierarchical Clustering:** A connectivity-based approach that creates a bottom-up tree structure called a **Dendrogram**. It merges individual items into groups based on linkage criteria like ward variance or complete distance metrics.
- **The Elbow Method & Silhouette Coefficients:** Because K must be selected beforehand, engineers plot inertia variations to locate an "elbow bend," or calculate individual silhouette widths to measure how well each point fits within its assigned group compared to neighboring clusters.

Mall_Customers

291

Download

Data Card

Code (264)

Discussion (0)

Suggestions (0)

Mall_Customers.csv (4...)

Detail Compact Column 5 of 5 columns

About this file

Suggest Edits

This file does not have a description yet.

CustomerID	Genre	Age
	Female	56%

Data Explorer

Version 1 (4.29 kB)

Mall_Customers.csv

Lab work on orange



File

File - Orange

Source

☒ File: Downloads\Mall_Customers.csv

☐ URL:

File Type

Determine type from the file extension

Info

200 instances
5 features (no missing values)
Data has no target variable.
0 meta attributes

Columns (Double click to edit)

	Name	Type	Role	Values
1	CustomerID	🟢 categorical	feature	
2	Genre	🟢 categorical	feature	Female, Male
3	Age	🟢 categorical	feature	
4	Annual Income (k\$)	🟡 datetime	feature	
5	Spending Score (1-100)	🔴 numeric	feature	



Select Columns - Orange

Ignored

Filter

Features (3)

Filter

- Age
- Annual Income (k\$)
- Spending Score (1-100)

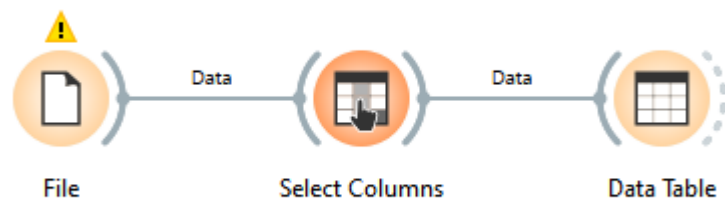
Target

Metas (2)

- CustomerID
- Genre

Reset ☐ Ignore new variables by default ☒ Send Automatically

The screenshot shows the 'Select Columns' widget interface. It has a title bar 'Select Columns - Orange' with standard window controls. The interface is divided into several sections: 'Ignored' with a 'Filter' input field and an empty list; 'Features (3)' with a 'Filter' input field and a list containing 'Age', 'Annual Income (k\$)', and 'Spending Score (1-100)'; 'Target' with an empty input field; and 'Metas (2)' with a list containing 'CustomerID' and 'Genre'. Between the 'Ignored' and 'Features' sections, and between 'Features' and 'Metas', there are grey buttons with a right-pointing arrow. At the bottom, there is a 'Reset' button, a checkbox for 'Ignore new variables by default' (which is unchecked), and a 'Send Automatically' button with a checked checkbox.



Data Table - Orange

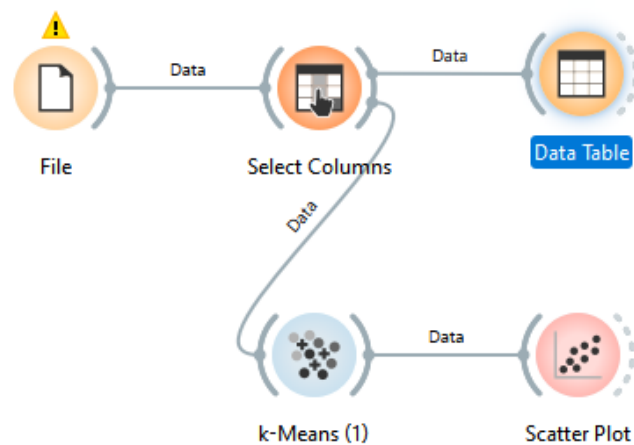
Info
200 instances (no missing data)
3 features
No target variable.
2 meta attributes

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
Clear Selection
☒ Select full rows

Restore Original Order
☒ Send Automatically

	CustomerID	Genre	Age	annual Income (k\$)	ending Score (1-10)
1	1	Male	19	15.0	39
2	2	Male	21	15.0	81
3	3	Female	20	16.0	6
4	4	Female	23	16.0	77
5	5	Female	31	17.0	40
6	6	Female	22	17.0	76
7	7	Female	35	18.0	6
8	8	Female	23	18.0	94
9	9	Male	64	19.0	3
10	10	Female	30	19.0	72
11	11	Male	67	19.0	14
12	12	Female	35	19.0	99
13	13	Female	58	20.0	15
14	14	Female	24	20.0	77
15	15	Male	37	20.0	13
16	16	Male	22	20.0	79
17	17	Female	35	21.0	35
18	18	Male	20	21.0	66
19	19	Male	52	23.0	29



k-Means (1) - Orange

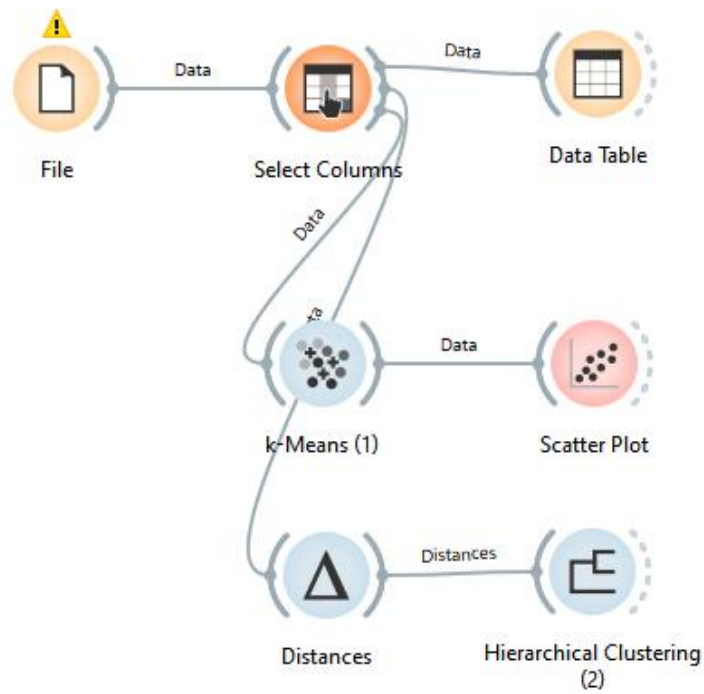
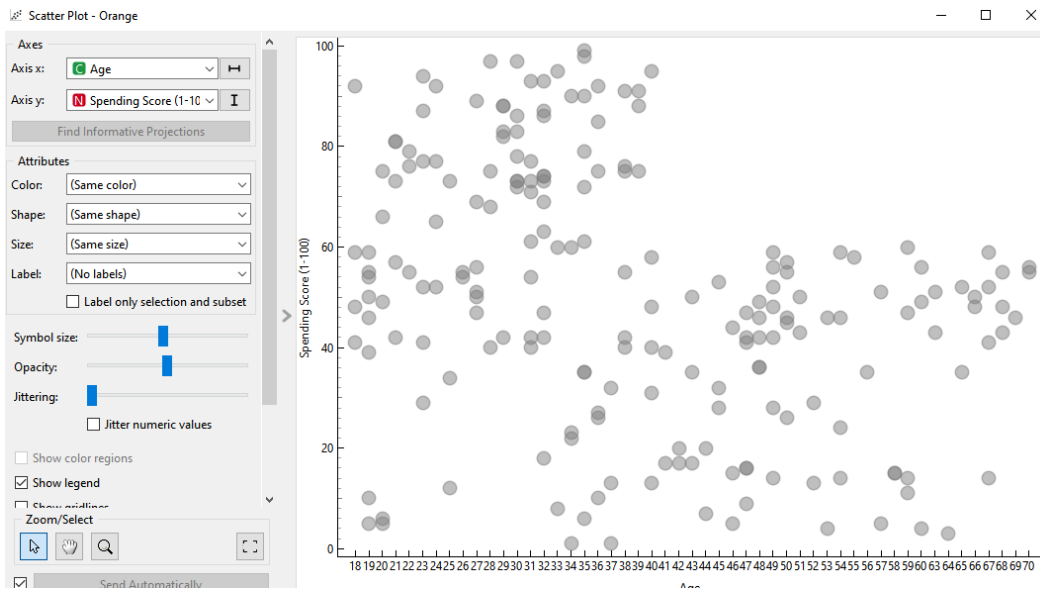
Number of Clusters
☒ Fixed:
☐ From to

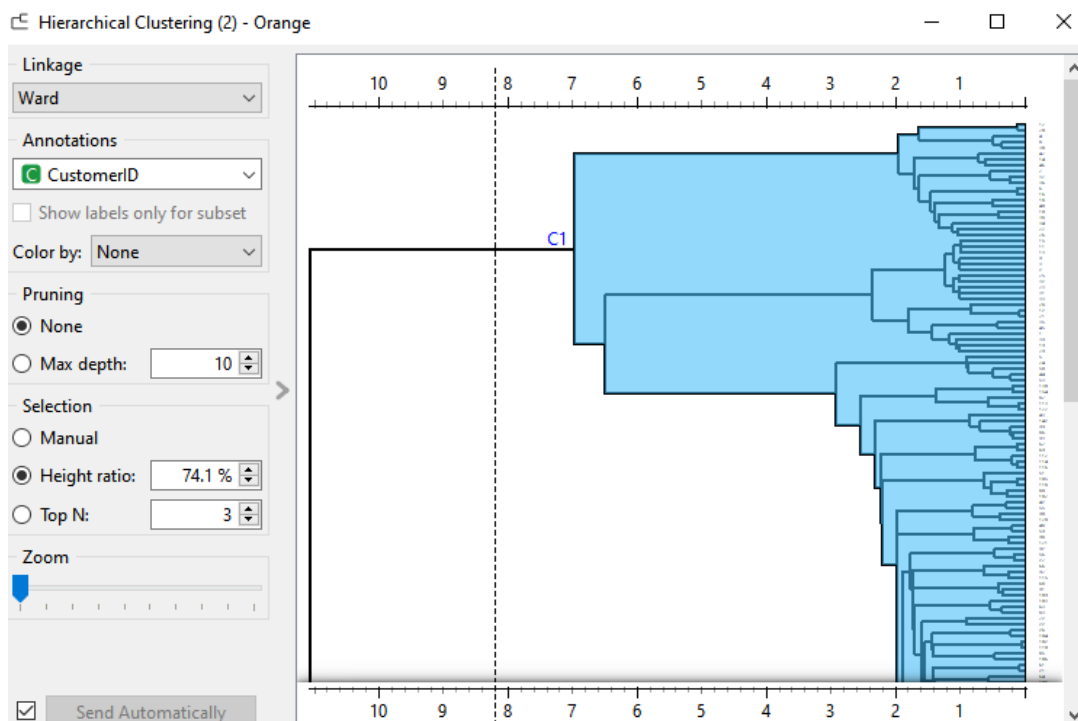
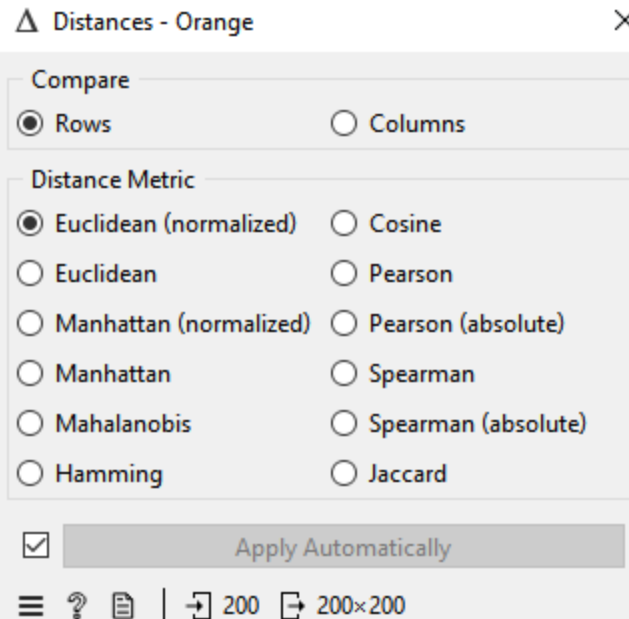
Preprocessing
☒ Normalize columns

Initialization
 Initialize with KMeans++

Re-runs:
 Maximum iterations:

☒ Apply Automatically





Lab Work on python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.vq import kmeans2, whiten
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.metrics import silhouette_score
```

1. Load the dataset

```
df = pd.read_csv('Mall_Customers.csv')
```

2. Extract features matrix

```
# Index 2 = Age, Index 3 = Annual Income, Index 4 = Spending Score  
X_features = df.iloc[:, [2, 3, 4]].values
```

3. Feature Standardization

```
X_scaled = whiten(X_features)
```

4. K-Means Clustering via SciPy ¶

```
optimal_k = 5  
centroids, kmeans_labels = kmeans2(X_scaled, optimal_k, minit='points', seed=42)  
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
```

5. Hierarchical Agglomerative Clustering

```
Z = linkage(X_scaled, method='ward', metric='euclidean')  
hier_labels = fcluster(Z, optimal_k, criterion='maxclust') - 1 # Adjusted to 0-indexed bases  
hier_silhouette = silhouette_score(X_scaled, hier_labels)  
  
df['KMeans_Cluster'] = kmeans_labels  
df['Hierarchical_Cluster'] = hier_labels
```

6. Display KPI Evaluation Metrics Report

```
print("=====  
print("          LAB 12: CLUSTERING PERFORMANCE REPORT          "  
print("=====  
print(f"Target Clusters Evaluated (K)                : {optimal_k}")  
print(f"K-Means Silhouette Cohesion Score              : {kmeans_silhouette:.4f}")  
print(f"Hierarchical Agglomerative Silhouette Score : {hier_silhouette:.4f}")  
print("=====  
print("\nSample Output Consolidated with Clusters:")  
print(df.iloc[:, [0, 1, 2, -2, -1]].head())
```

LAB 12: CLUSTERING PERFORMANCE REPORT

Target Clusters Evaluated (K) : 5
 K-Means Silhouette Cohesion Score : 0.4166
 Hierarchical Agglomerative Silhouette Score : 0.3900

Sample Output Consolidated with Clusters:

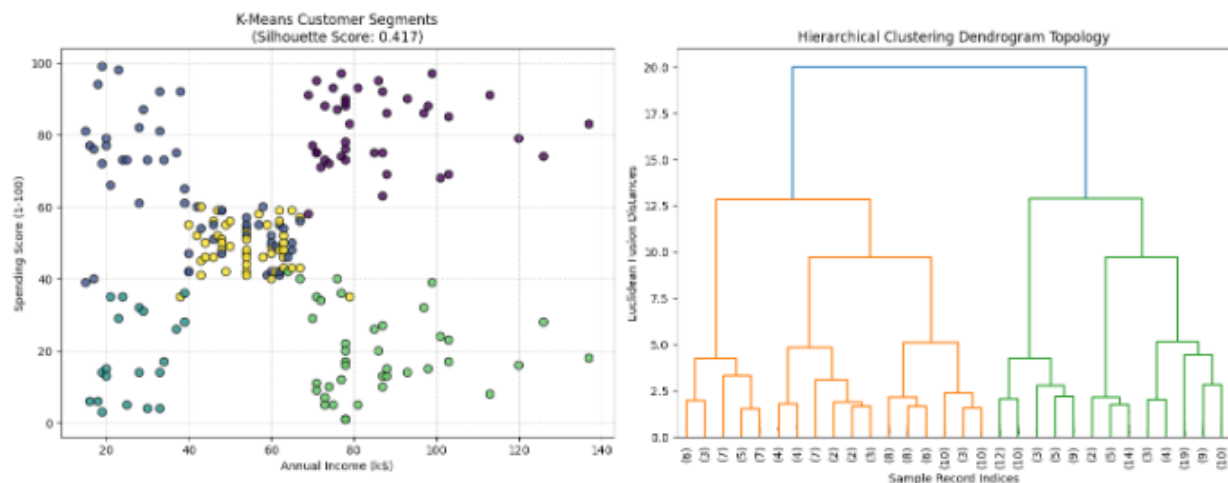
	CustomerID	Genre	Age	KMeans_Cluster	Hierarchical_Cluster
0	1	Male	19	1	1
1	2	Male	21	1	4
2	3	Female	20	2	1
3	4	Female	23	1	4
4	5	Female	31	1	1

7. Generate Visual Evaluation Plots

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: K-Means Spatial Clusters Scattered Map
ax1.scatter(X_features[:, 1], X_features[:, 2],
            c=kmeans_labels, cmap='viridis', edgecolor='k', s=50, alpha=0.8)
ax1.set_title(f"K-Means Customer Segments\n(Silhouette Score: {kmeans_silhouette:.3f})")
ax1.set_xlabel(df.columns[3])
ax1.set_ylabel(df.columns[4])
ax1.grid(True, linestyle='--', alpha=0.5)

# Plot 2: Hierarchical Clustering Dendrogram Topology Tree Map
ax2.set_title("Hierarchical Clustering Dendrogram Topology")
ax2.set_xlabel("Sample Record Indices")
ax2.set_ylabel("Euclidean Fusion Distances")
dendrogram(Z, truncate_mode='lastp', p=30, ax=ax2, leaf_rotation=90, leaf_font_size=10, show_c
plt.tight_layout()
plt.show()
```



LAB-13: Outlier Detection Techniques

1. Objective

The objective of this lab is to implement and analyze distance-based and density-based anomaly mining techniques. Students will construct an outlier isolation framework inside RapidMiner Studio, validate isolation vectors with Python, and identify abnormal transactions.

2. Tools & Datasets

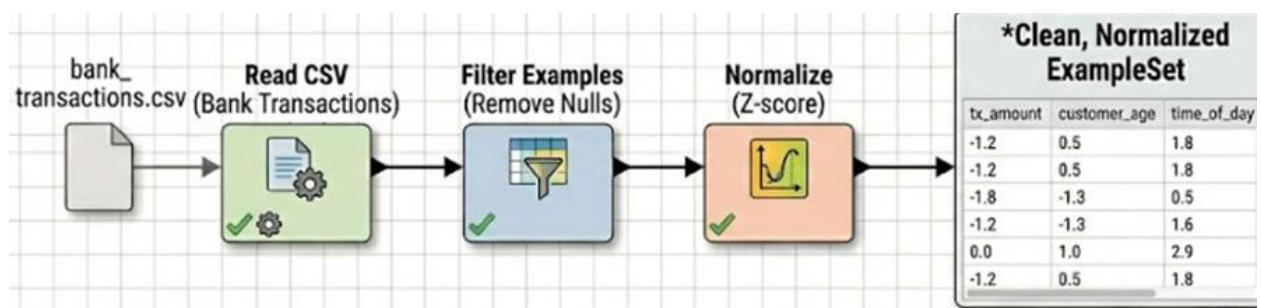
- **Tool:** RapidMiner Studio + Python Validation
- **Dataset:** Banking Dataset

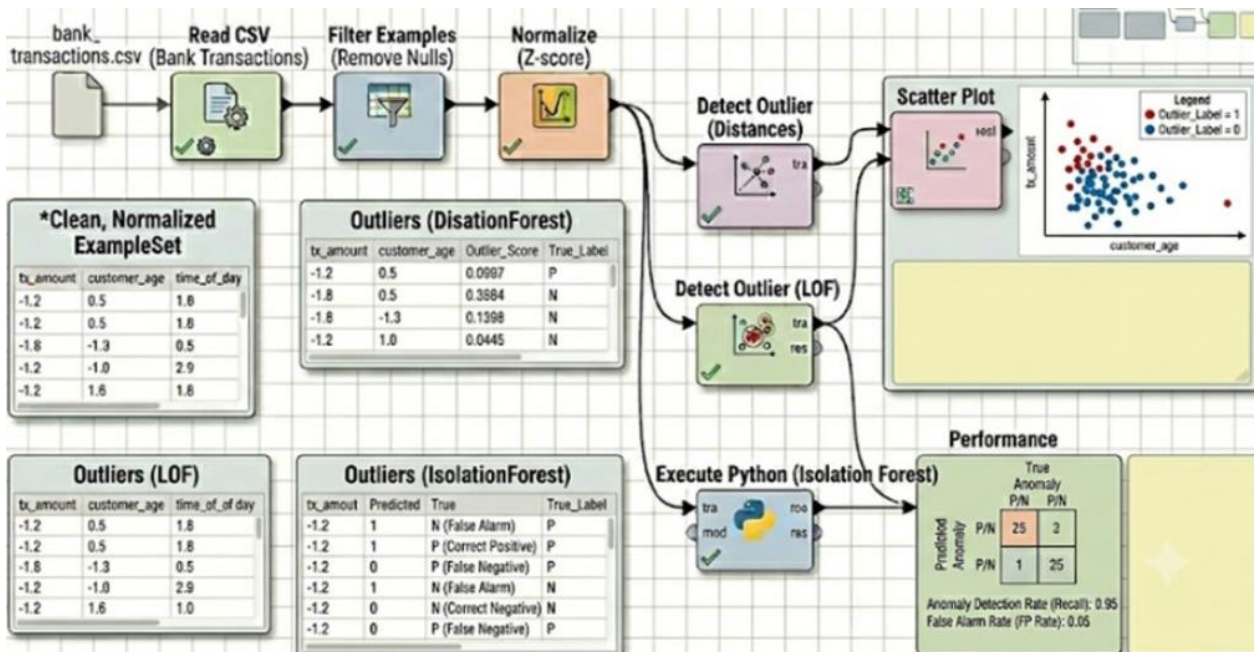
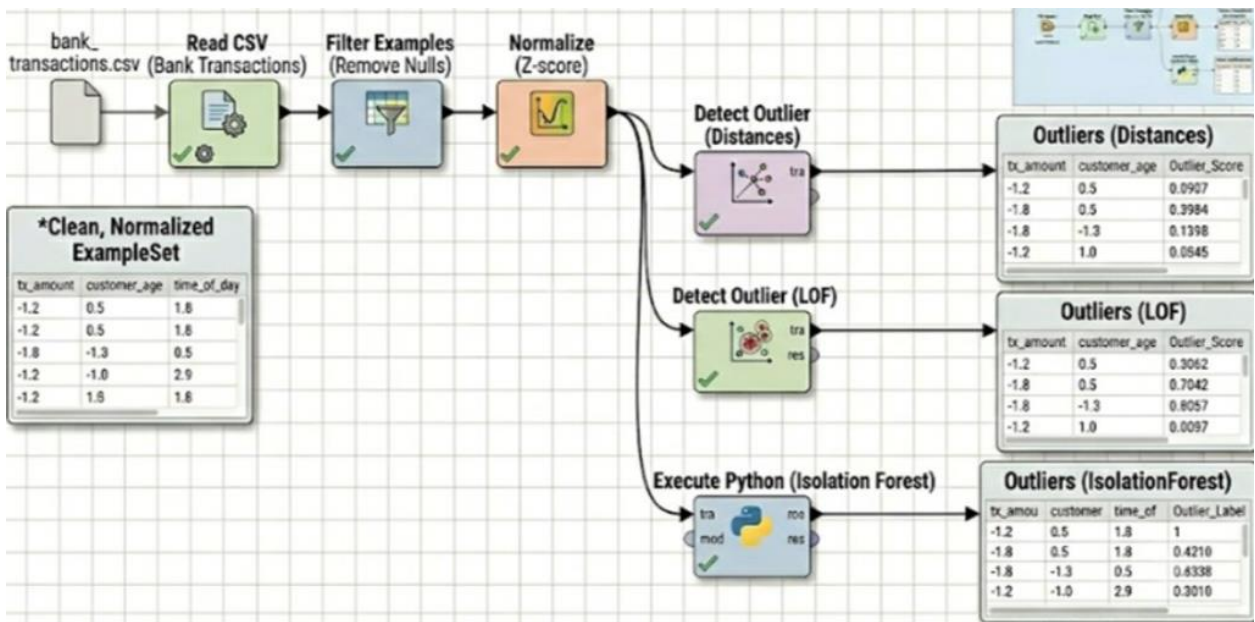
3. Background Concepts

Anomalies represent patterns that deviate significantly from normal behavior, often indicating fraud, system failures, or data entry errors.

- **Distance-Based Outlier Detection:** Evaluates localized space anomalies by measuring the geometric distance from a target record to its nearest neighbors within the feature space. Points located far from dense point clouds are flagged as outliers.
- **Isolation Forest (iForest):** An ensemble method that isolates anomalies instead of profiling normal data points. Because anomalies require fewer random splits to isolate within a tree structure, they appear noticeably closer to the root of the tree.
- **Local Outlier Factor (LOF):** A density-based algorithm that measures the local density deviation of a given data point relative to its k -nearest neighbors. Points with a substantially lower density than their surrounding neighbors are flagged as outliers.
- **Contamination Rate:** A user-defined hyperparameter specifying the expected proportion of outlier anomalies hidden within the target dataset (e.g., 0.05 implies a 5% anomaly expectation)

Work on





```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix

```

```

df = pd.read_csv("test.csv")
print(df.columns.tolist())

```

```

FEATURE_COL_1 = 'age'
FEATURE_COL_2 = 'balance'

```

```

GROUND_TRUTH_COL = 'y'

```

```

['age';'job';'marital';'education';'default';'balance';'housing';'loan';'contact';'day';'month';'duration';'campaign';'pdays';'previous';'poutcome';'y']

```

1. DATA LOADING & PREPROCESSING

```

if not os.path.exists(FILE_PATH):
    raise FileNotFoundError(f"Could not find the dataset file at: {FILE_PATH}.")

# CRITICAL FIX: Added sep=';' to correctly split your columns
df = pd.read_csv(FILE_PATH, sep=';')

# Clean up quotes from string columns if they exist natively in the data entries
for col in df.select_dtypes(include=['object']).columns:
    df[col] = df[col].astype(str).str.replace("'", '').str.replace("", '')

print(f"Dataset loaded successfully! Total Rows: {df.shape[0]}, Total Columns: {df.shape[1]}")
print("Parsed Columns:", df.columns.tolist())

# Drop missing values safely
df = df.dropna(subset=[FEATURE_COL_1, FEATURE_COL_2, GROUND_TRUTH_COL]).reset_index(drop=True)

# Convert Ground Truth 'yes'/'no' into 1 and 0 so the KPI functions can read it
df['Encoded_Ground_Truth'] = df[GROUND_TRUTH_COL].map({'yes': 1, 'no': 0})

# Extract features and scale them
X = df[[FEATURE_COL_1, FEATURE_COL_2]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

```

Dataset loaded successfully! Total Rows: 4521, Total Columns: 17
Parsed Columns: ['age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'y']

```


2. ALGORITHM 1: ISOLATION FOREST

```
iso_forest = IsolationForest(contamination=0.05, random_state=42)
df['IF_Raw_Pred'] = iso_forest.fit_predict(X_scaled)
df['IF_Anomaly'] = df['IF_Raw_Pred'].map({1: 0, -1: 1})
```

3. ALGORITHM 2: DISTANCE-BASED (KNN DISTANCE)

```
k_neighbors = 5
knn = NearestNeighbors(n_neighbors=k_neighbors)
knn.fit(X_scaled)
distances, _ = knn.kneighbors(X_scaled)

df['KNN_Outlier_Score'] = distances.mean(axis=1)
contamination_threshold = np.percentile(df['KNN_Outlier_Score'], 95)
df['KNN_Anomaly'] = (df['KNN_Outlier_Score'] >= contamination_threshold).astype(int)
```

4. KPI & EVALUATION METRICS CALCULATION

```
def calculate_lab_kpis(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    anomaly_detection_rate = (tp / (tp + fn)) * 100 if (tp + fn) > 0 else 0
    false_alarm_rate = (fp / (fp + tn)) * 100 if (fp + tn) > 0 else 0
    return anomaly_detection_rate, false_alarm_rate

# Compute metrics using our encoded 1/0 column
if_adr, if_far = calculate_lab_kpis(df['Encoded_Ground_Truth'], df['IF_Anomaly'])
knn_adr, knn_far = calculate_lab_kpis(df['Encoded_Ground_Truth'], df['KNN_Anomaly'])

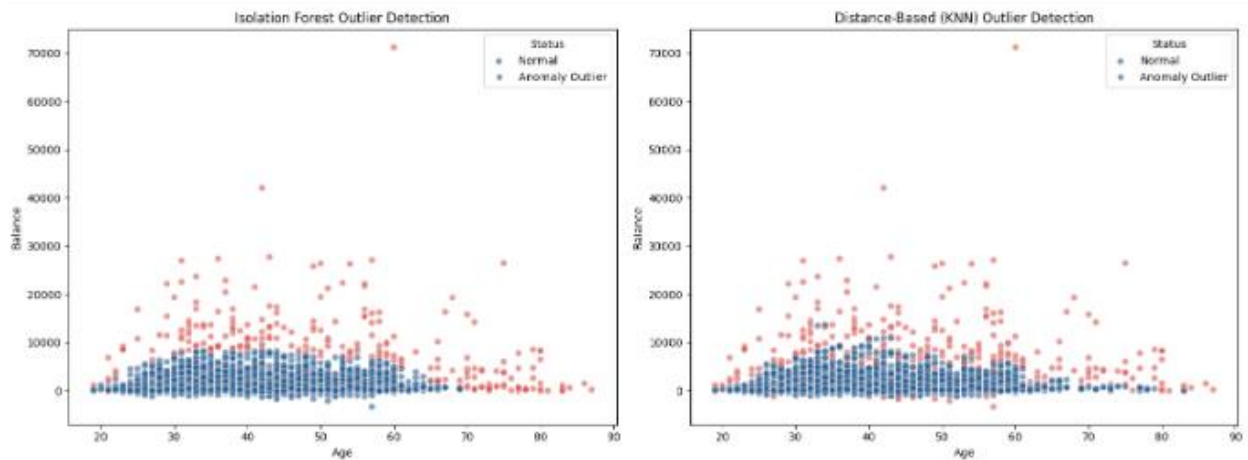
kpi_table = pd.DataFrame({
    "Evaluation Metric (KPI)": ["Anomaly Detection Rate (ADR)", "False Alarm Rate (FAR)"],
    "Isolation Forest": [f"{if_adr:.2f}%", f"{if_far:.2f}%"],
    "Distance-Based (KNN)": [f"{knn_adr:.2f}%", f"{knn_far:.2f}%"]
})

print("  EVALUATION METRICS")
print(kpi_table.to_string(index=False))
```

```
EVALUATION METRICS
Evaluation Metric (KPI) Isolation Forest Distance-Based (KNN)
Anomaly Detection Rate (ADR)      8.06%      7.49%
False Alarm Rate (FAR)           4.60%      4.70%
```


5. VISUALIZATION

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))  
  
# Plot 1: Isolation Forest Output  
sns.scatterplot(  
    data=df, x=FEATURE_COL_1, y=FEATURE_COL_2,  
    hue='IF_Anomaly', palette={0: '#2b5c8f', 1: '#d9534f'}, alpha=0.6, ax=axes[0]  
)  
axes[0].set_title('Isolation Forest Outlier Detection')  
axes[0].set_xlabel('Age')  
axes[0].set_ylabel('Balance')  
axes[0].legend(title='Status', labels=['Normal', 'Anomaly Outlier'])  
  
# Plot 2: Distance-Based KNN Output  
sns.scatterplot(  
    data=df, x=FEATURE_COL_1, y=FEATURE_COL_2,  
    hue='KNN_Anomaly', palette={0: '#2b5c8f', 1: '#d9534f'}, alpha=0.6, ax=axes[1]  
)  
axes[1].set_title('Distance-Based (KNN) Outlier Detection')  
axes[1].set_xlabel('Age')  
axes[1].set_ylabel('Balance')  
axes[1].legend(title='Status', labels=['Normal', 'Anomaly Outlier'])  
  
plt.tight_layout()  
plt.show()
```



LAB-14: Web Scraping & Sentiment Analysis

1. Objective

The objective of this lab is to implement an end-to-end unstructured natural language processing pipeline. Students will extract online product reviews via programmatic web scraping, apply text normalization techniques, perform sentiment classification, and construct network maps using Gephi or Citespace.

2. Tools & Datasets

- **Tool:** Citespace / Gephi + Python Engine
- **Dataset:** Social Media / Review Dataset

3. Background Concepts

Unstructured text data from social media and review platforms provides rich insight into consumer preferences, but must be systematically processed before analysis.

- **Text Preprocessing Pipelines:** Converts raw, unstructured text into structured features using steps like tokenization, stop-word removal, and lemmatization.
- **Sentiment Polarity Scoring:** Uses rule-based or algorithmic models to classify text into a continuous or categorical range representing positive, negative, or neutral sentiment.
- **Co-occurrence Network Mapping:** Maps relationships between frequent terms or entities, revealing thematic connections within the text corpus.

1. Load and Clean Data

```
import pandas as pd
import re

# Load data with encoding fix
df = pd.read_csv("test (1).csv", encoding='latin-1')
text_col = 'text' # Update if your column name is different

# Simple cleaning function
def clean(text):
    text = re.sub(r'^a-zA-Z\s', '', str(text).lower())
    return [w for w in text.split() if w not in ['the', 'and', 'a', 'to', 'of']]

df['clean_tokens'] = df[text_col].apply(clean)
```

2. Sentiment Scoring Engine

```
, 'great', 'love', 'amazing', 'best', 'nice', 'happy'}
'worst', 'hate', 'terrible', 'poor', 'slow', 'error'}

ity(tokens):
    for w in tokens if w in pos)
    for w in tokens if w in neg)
    - n) / (p + n) if (p + n) > 0 else 0

df['clean_tokens'].apply(get_polarity)
'] = df['score'].apply(lambda s: 'Positive' if s > 0 else ('Negative' if s < 0 else 'Neutral'))
```

3. KPI Metrics Calculation ¶

```
# KPI 1: Distribution
dist = df['sentiment'].value_counts(normalize=True) * 100
print("--- Sentiment Distribution (%) ---")
print(dist)

# KPI 2: Polarity Summary
print("\n--- Polarity Statistics ---")
print(f"Avg Polarity: {df['score'].mean():.4f}")
print(f"Max Polarity: {df['score'].max():.4f}")
print(f"Min Polarity: {df['score'].min():.4f}")
```

--- Sentiment Distribution (%) ---

sentiment

Neutral 85.503634

Positive 12.107996

Negative 2.388370

Name: proportion, dtype: float64

--- Polarity Statistics ---

Avg Polarity: 0.0973

Max Polarity: 1.0000

Min Polarity: -1.0000

4. Visualization

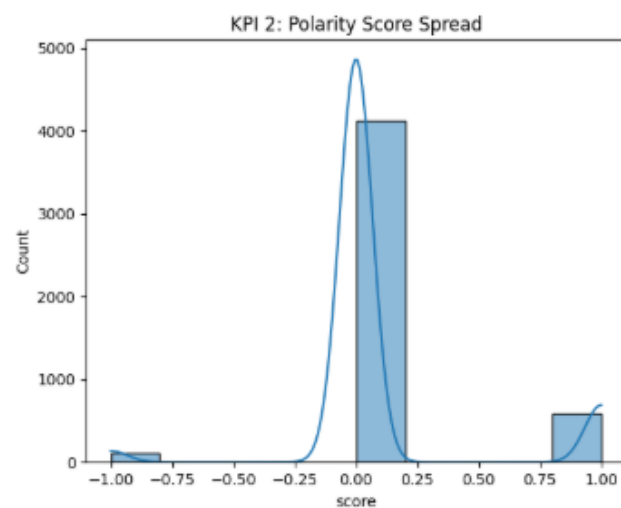
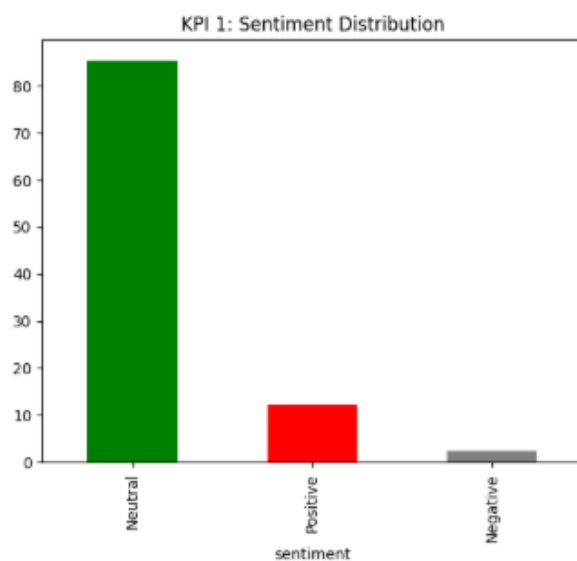
```
import matplotlib.pyplot as plt
import seaborn as sns

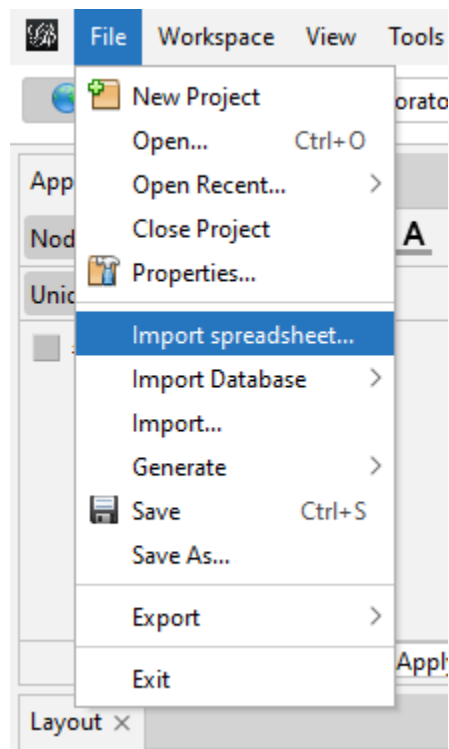
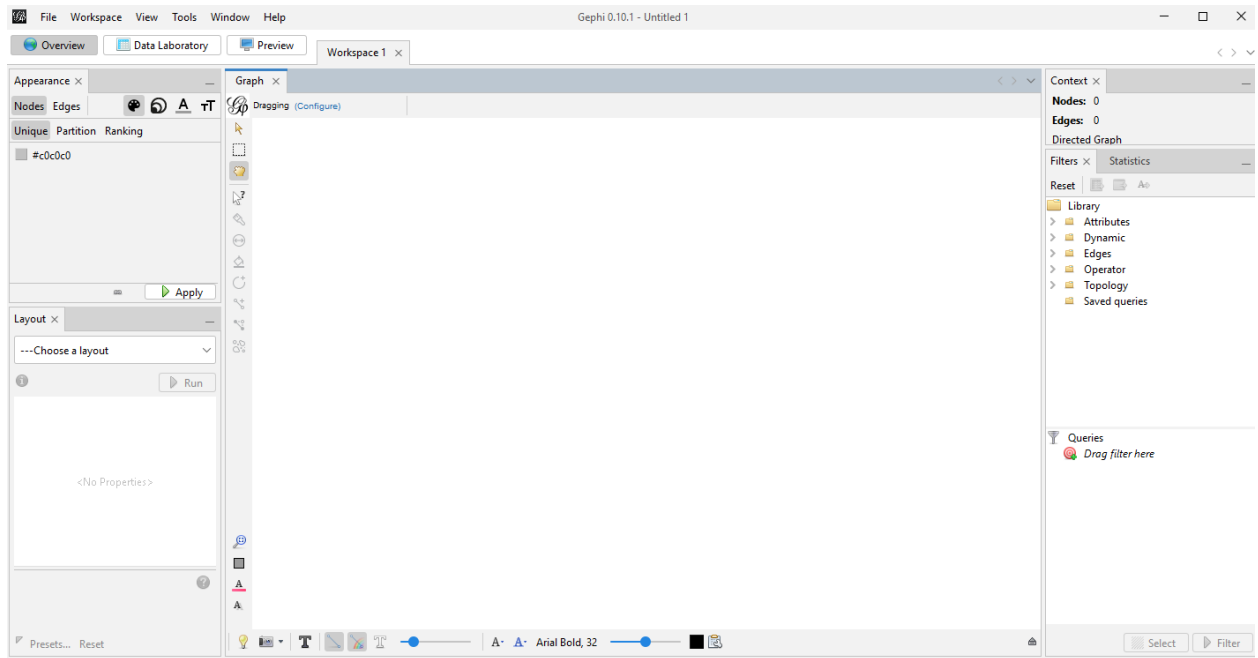
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

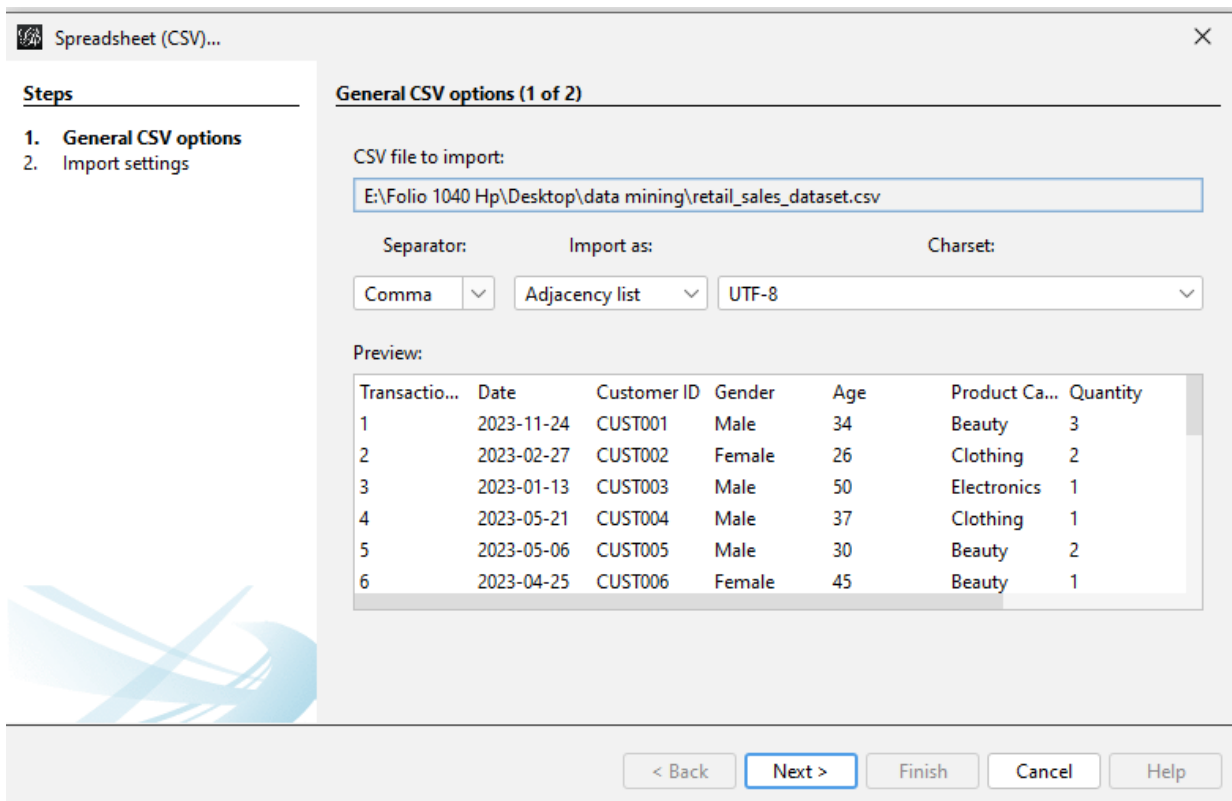
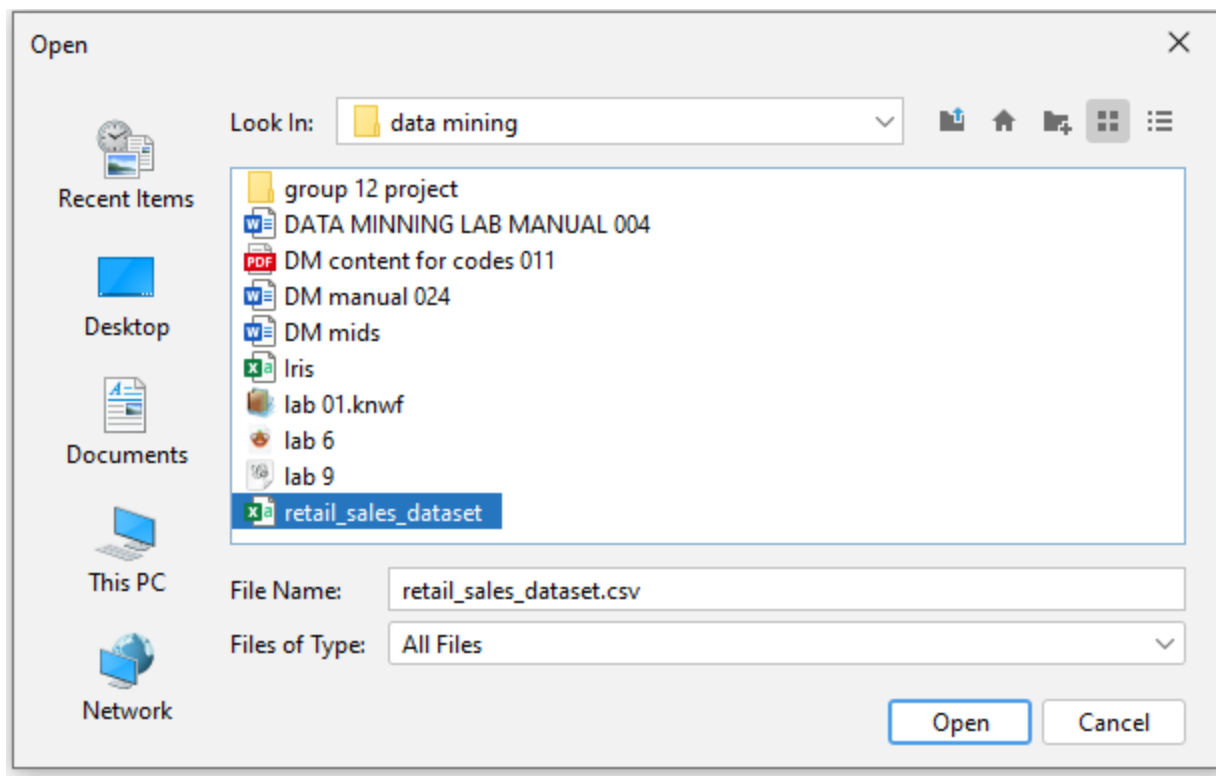
# Plot Distribution
dist.plot(kind='bar', ax=ax1, color=['green', 'red', 'gray'])
ax1.set_title("KPI 1: Sentiment Distribution")

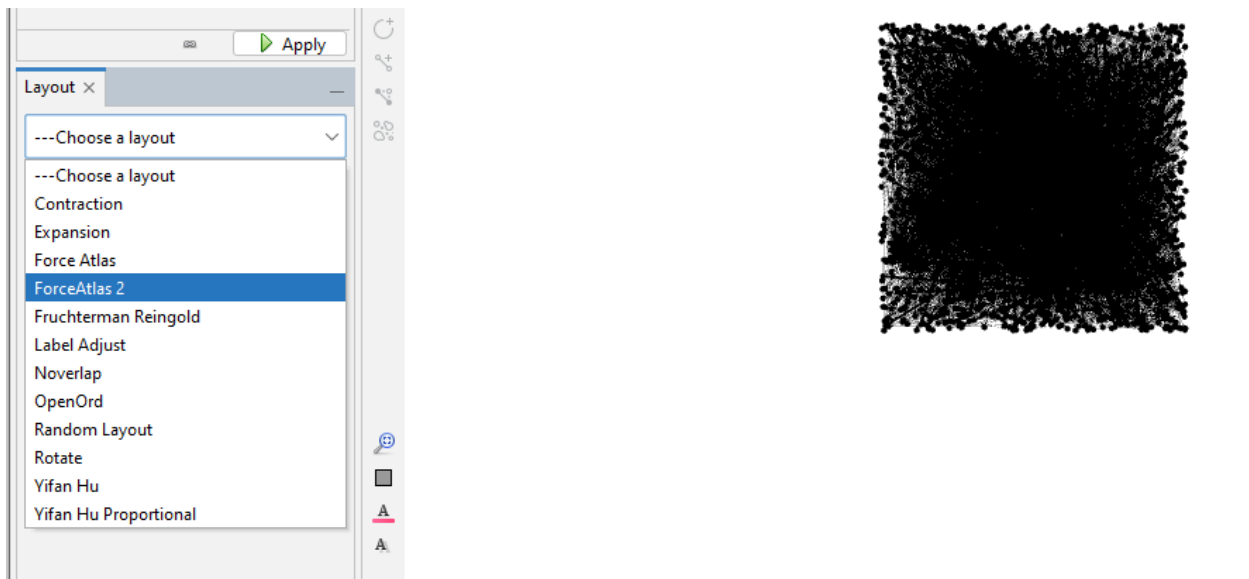
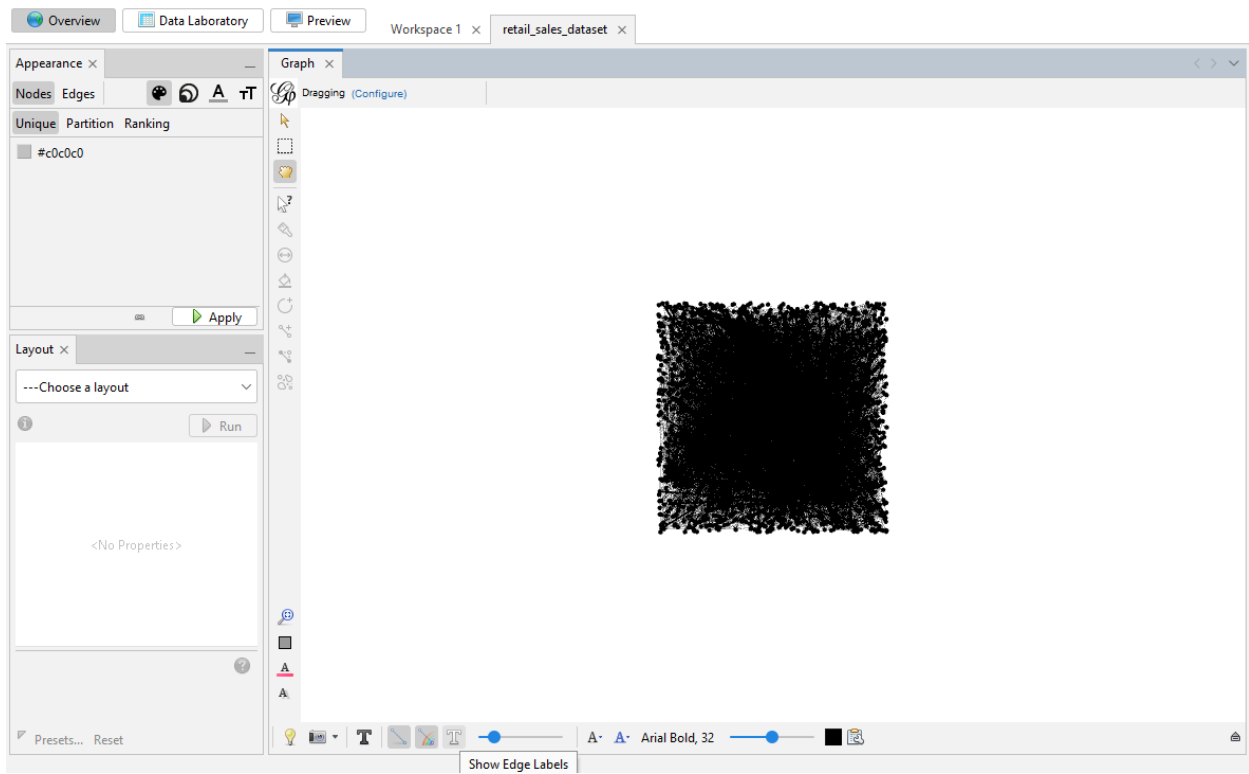
# Plot Polarity Spread
sns.histplot(df['score'], ax=ax2, bins=10, kde=True)
ax2.set_title("KPI 2: Polarity Score Spread")

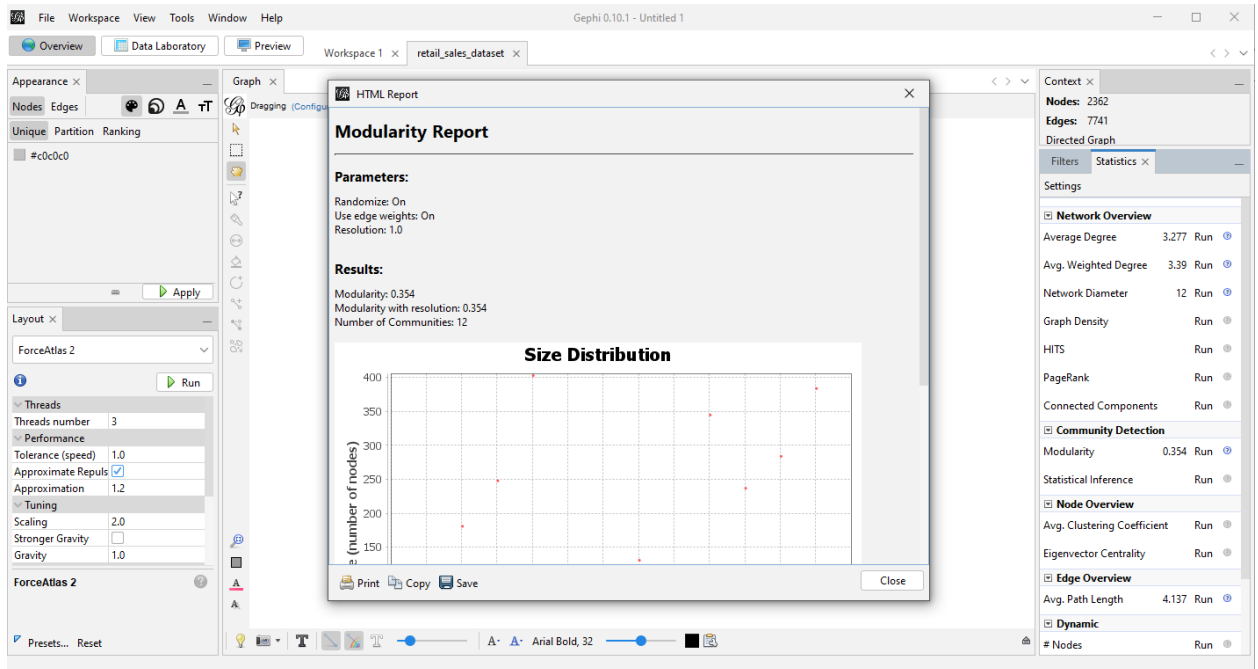
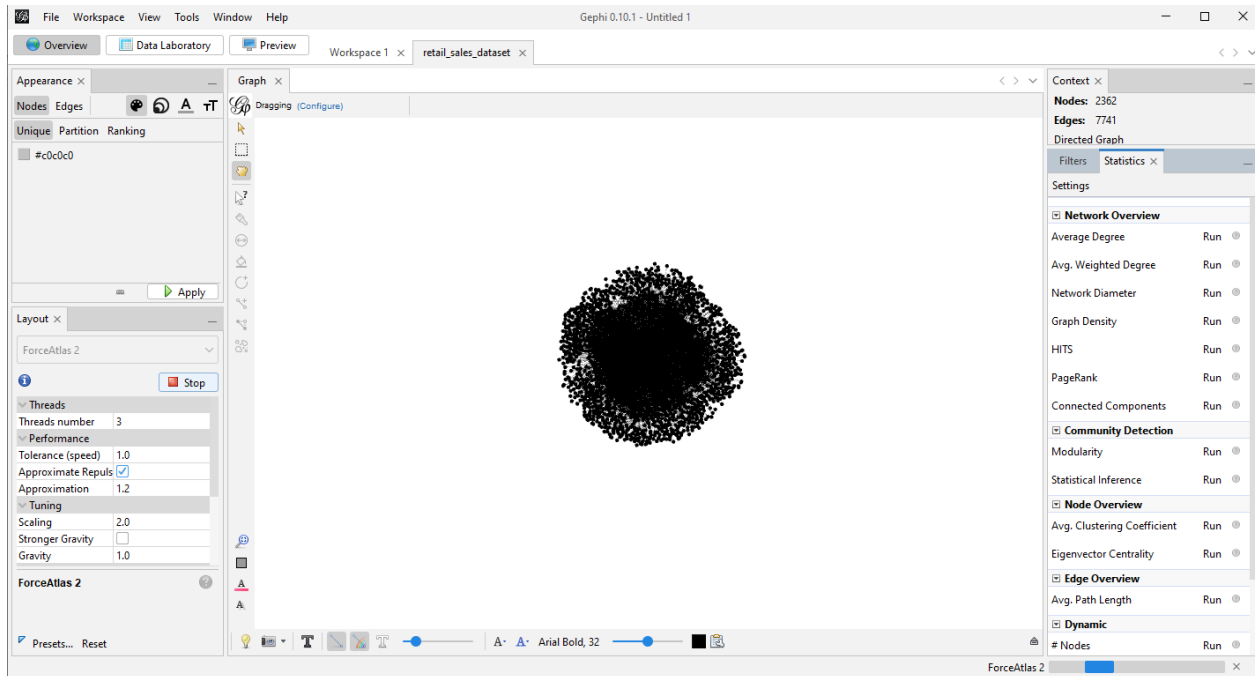
plt.show()
```



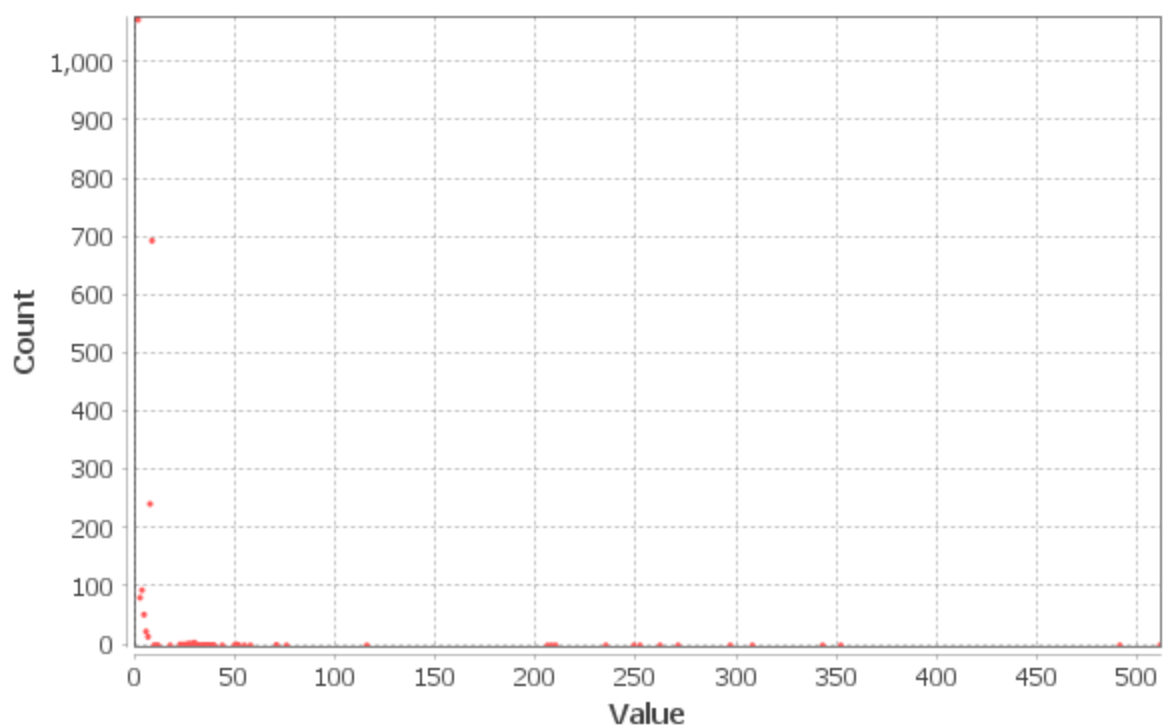




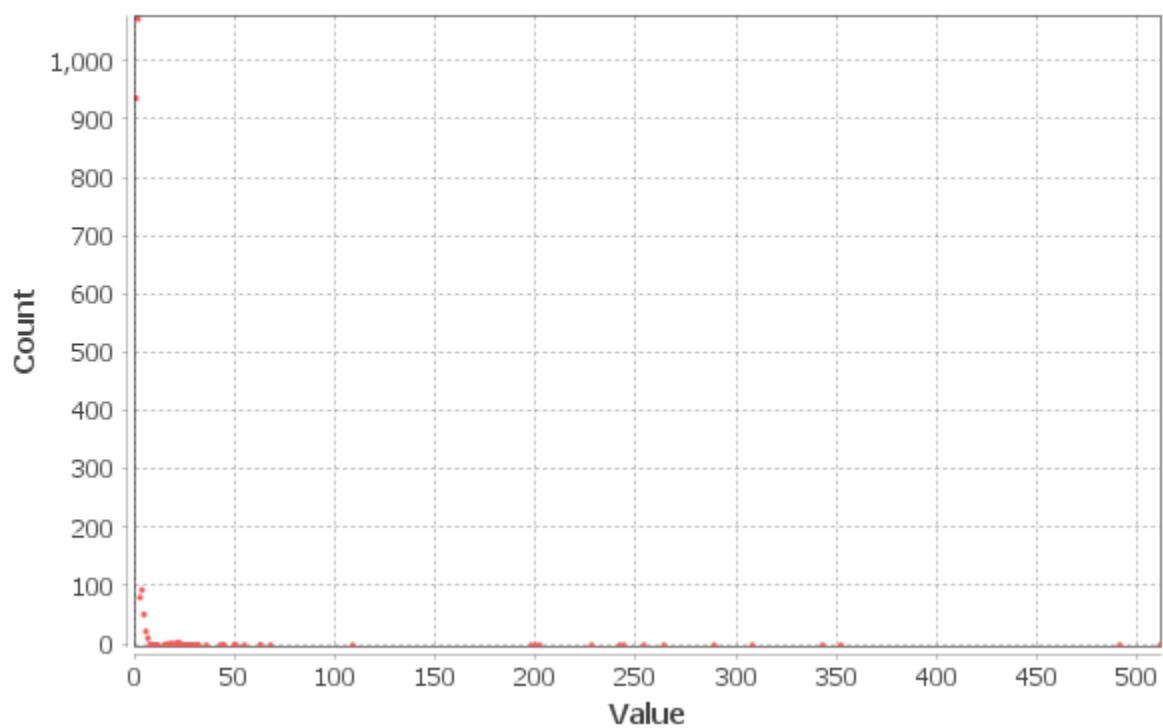




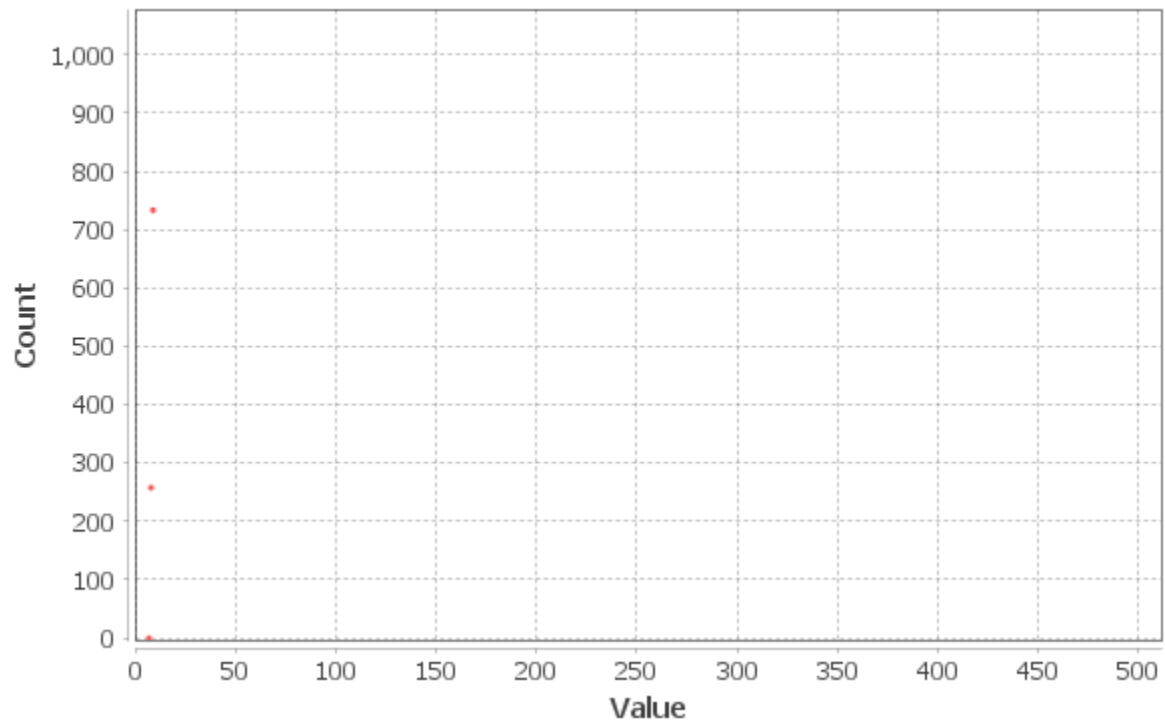
Degree Distribution



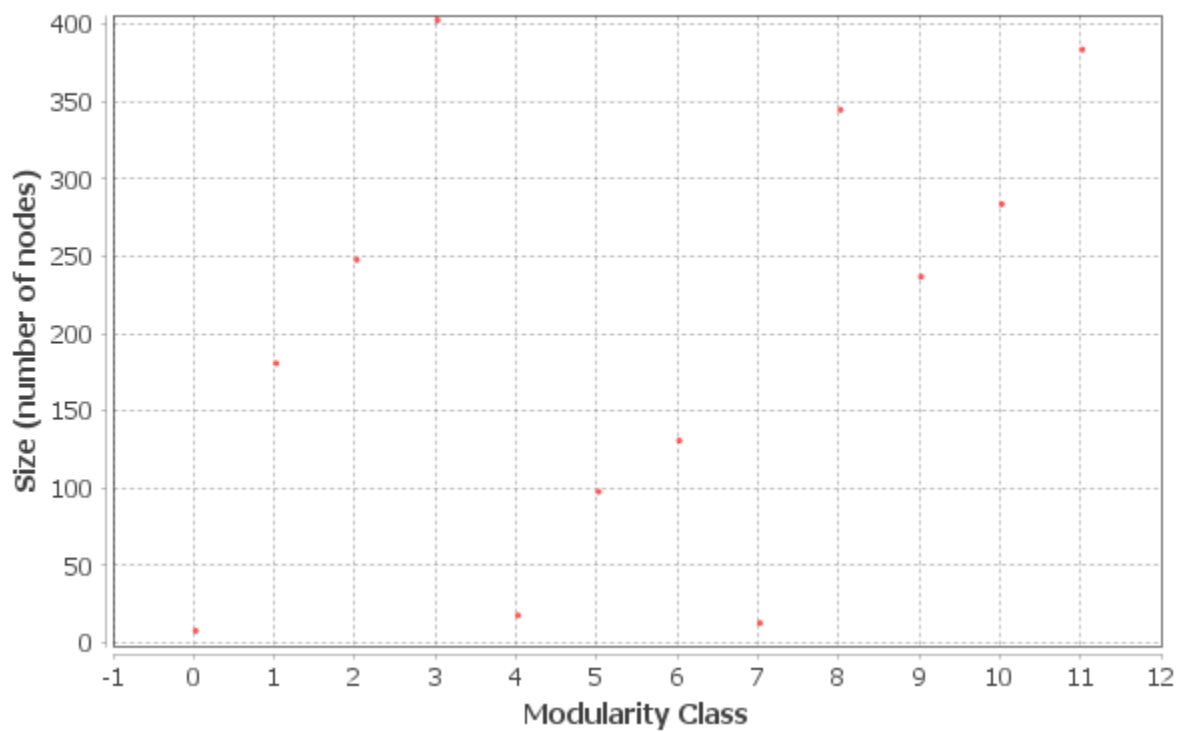
In-Degree Distribution

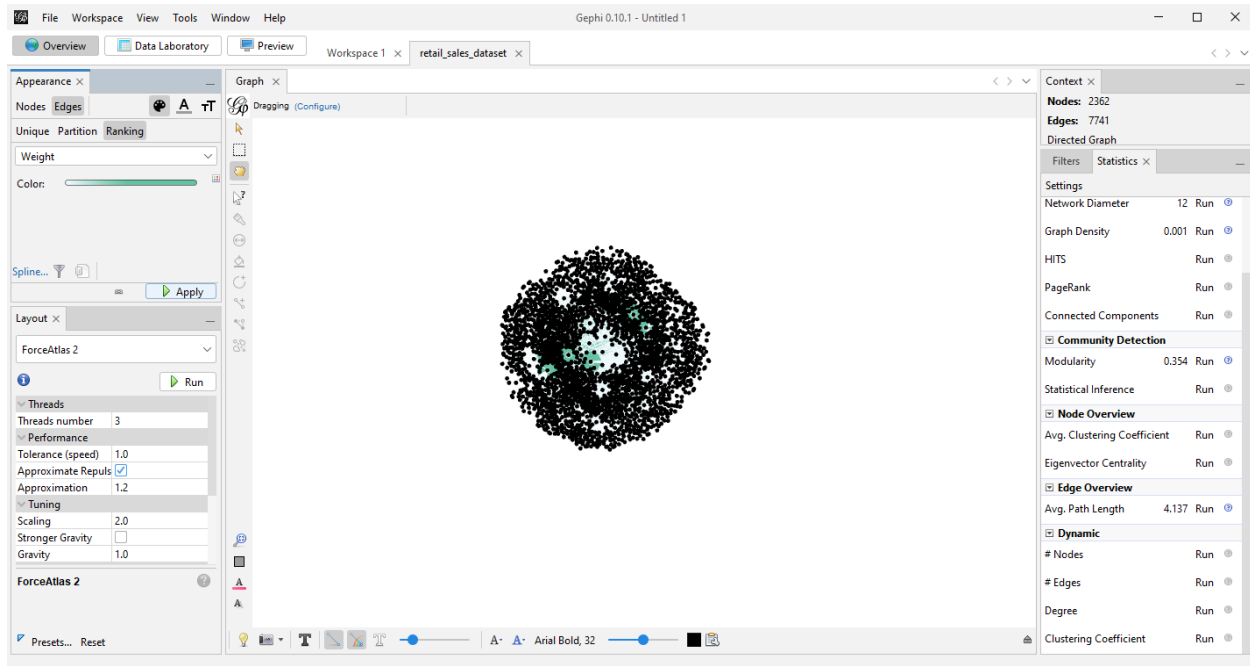


Out-Degree Distribution

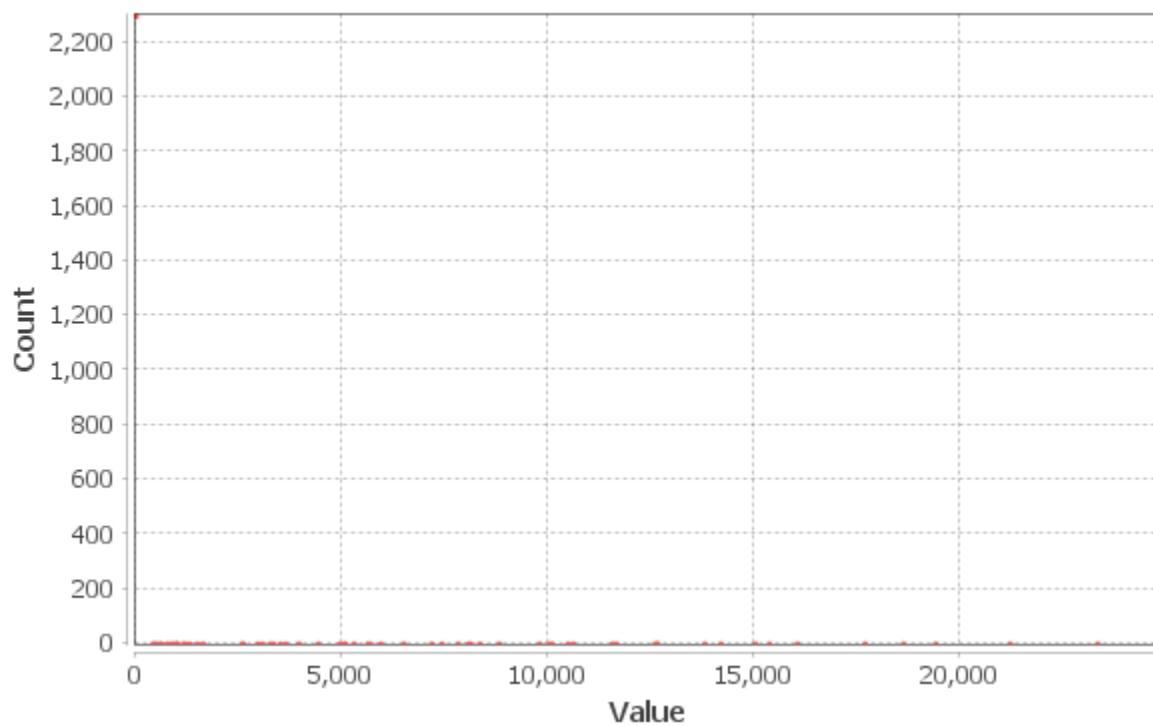


Size Distribution

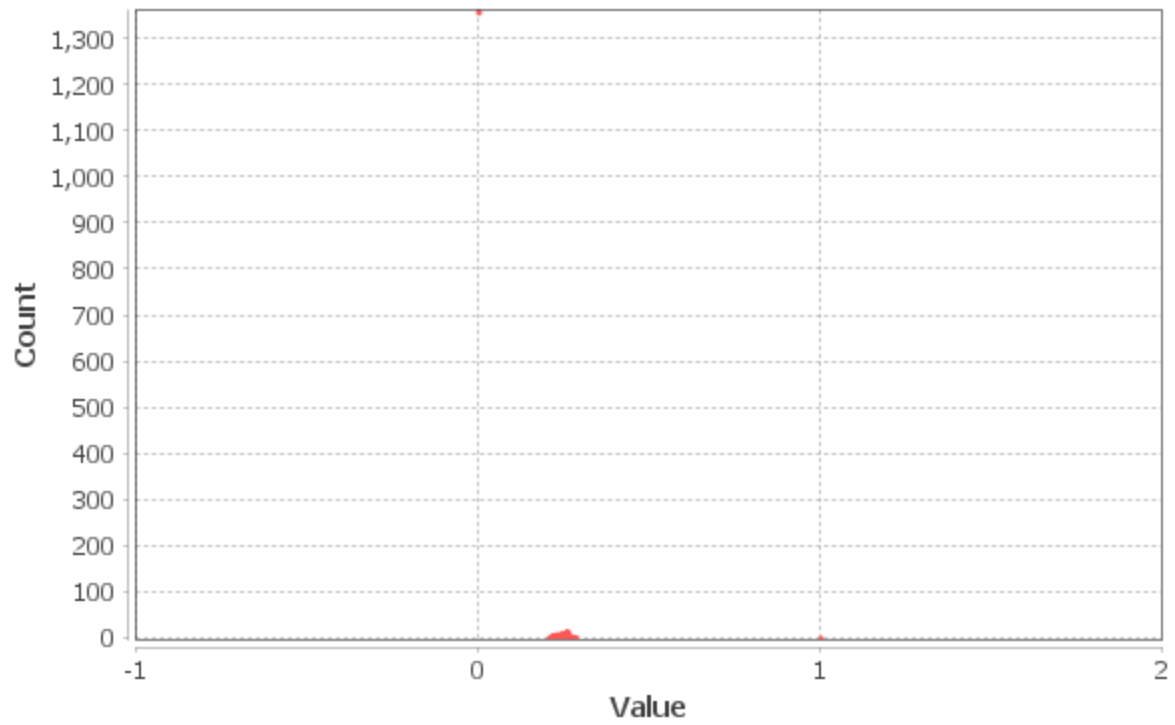




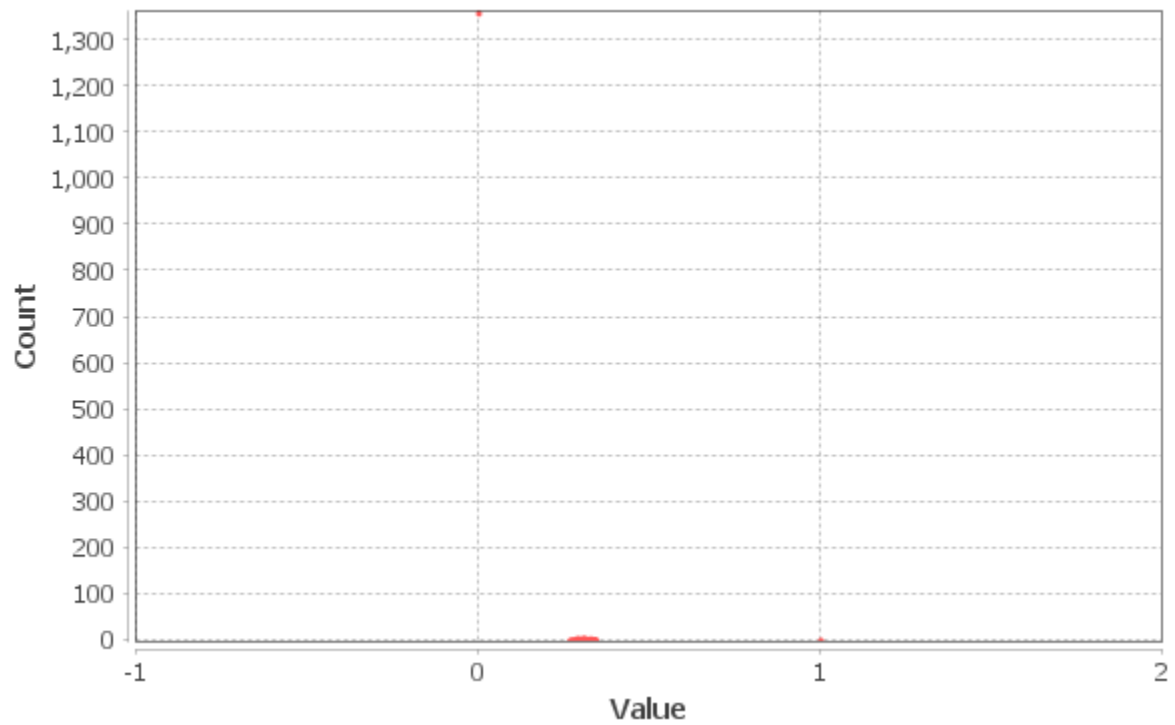
Betweenness Centrality Distribution



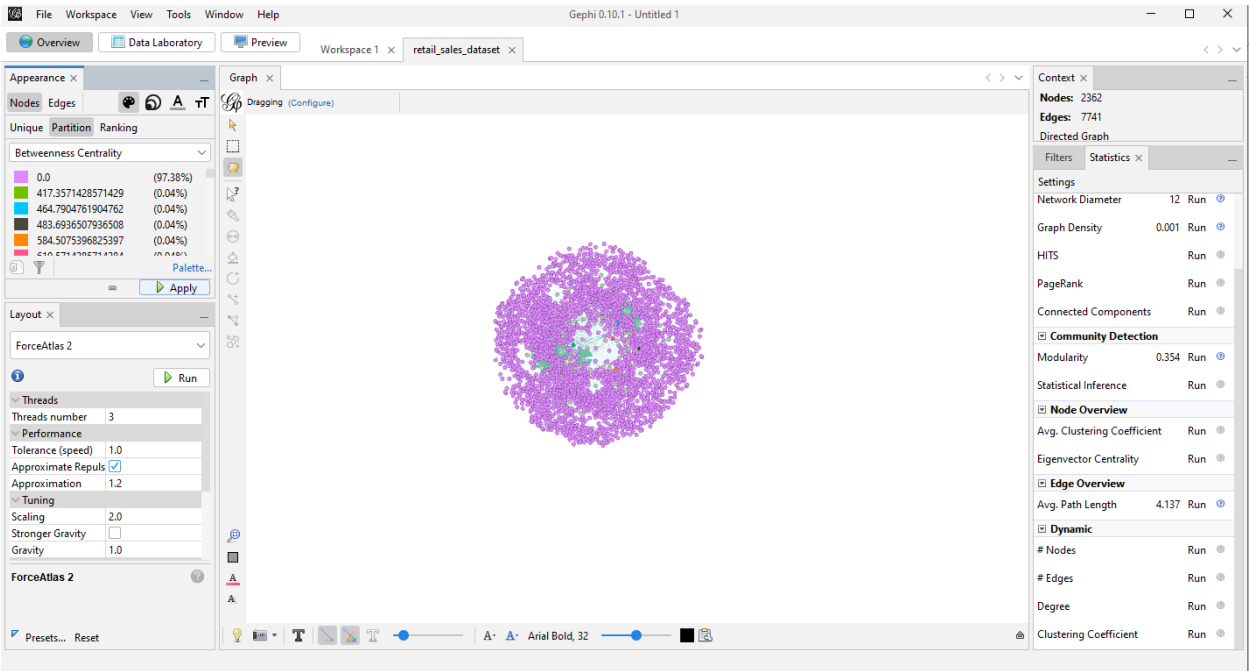
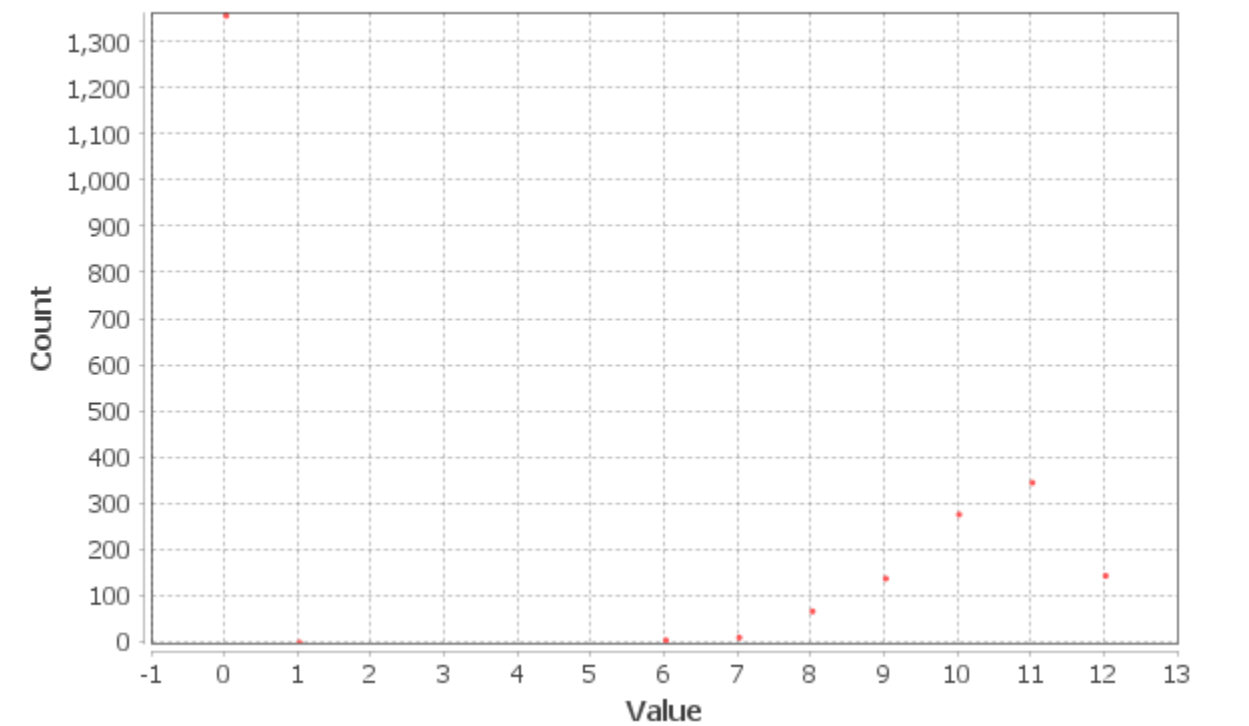
Closeness Centrality Distribution

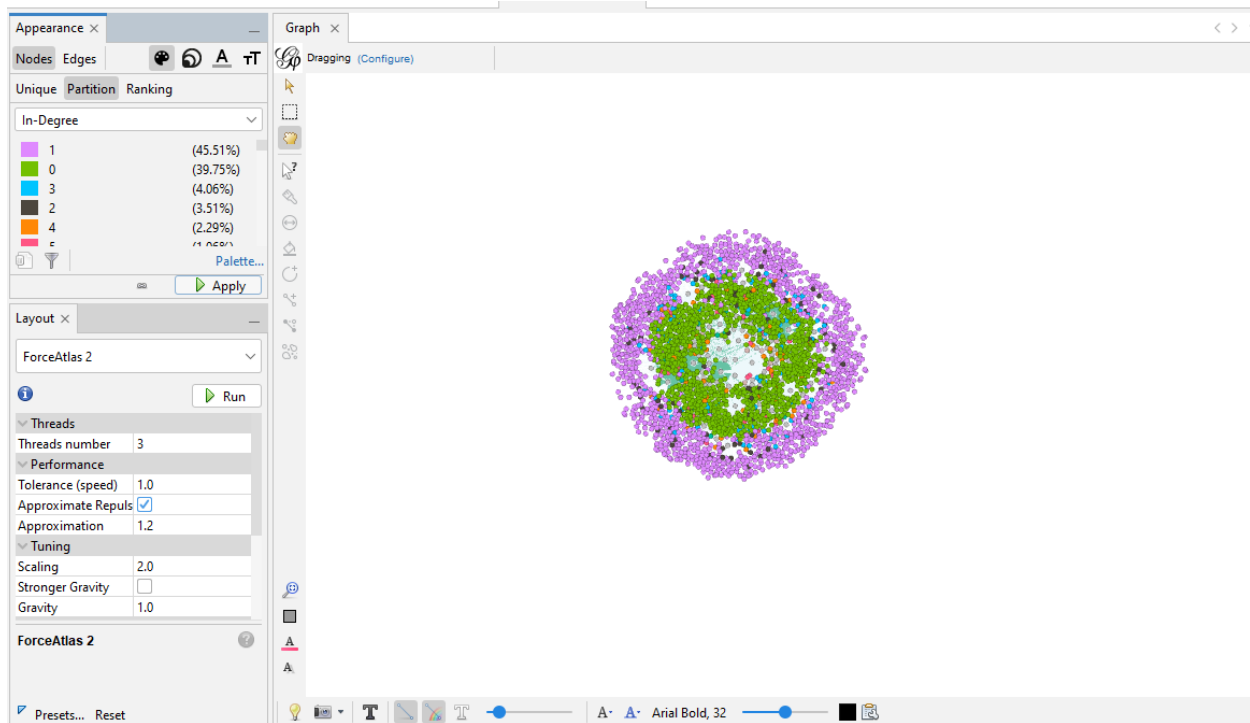
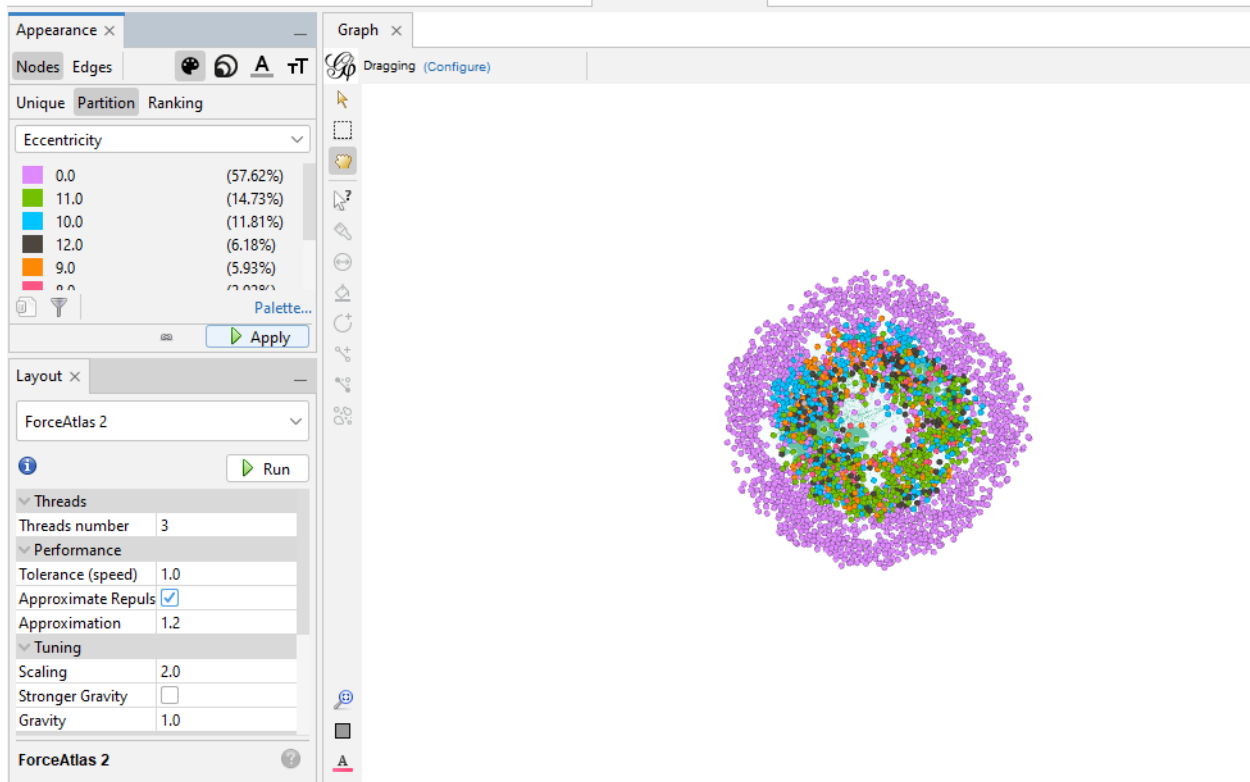


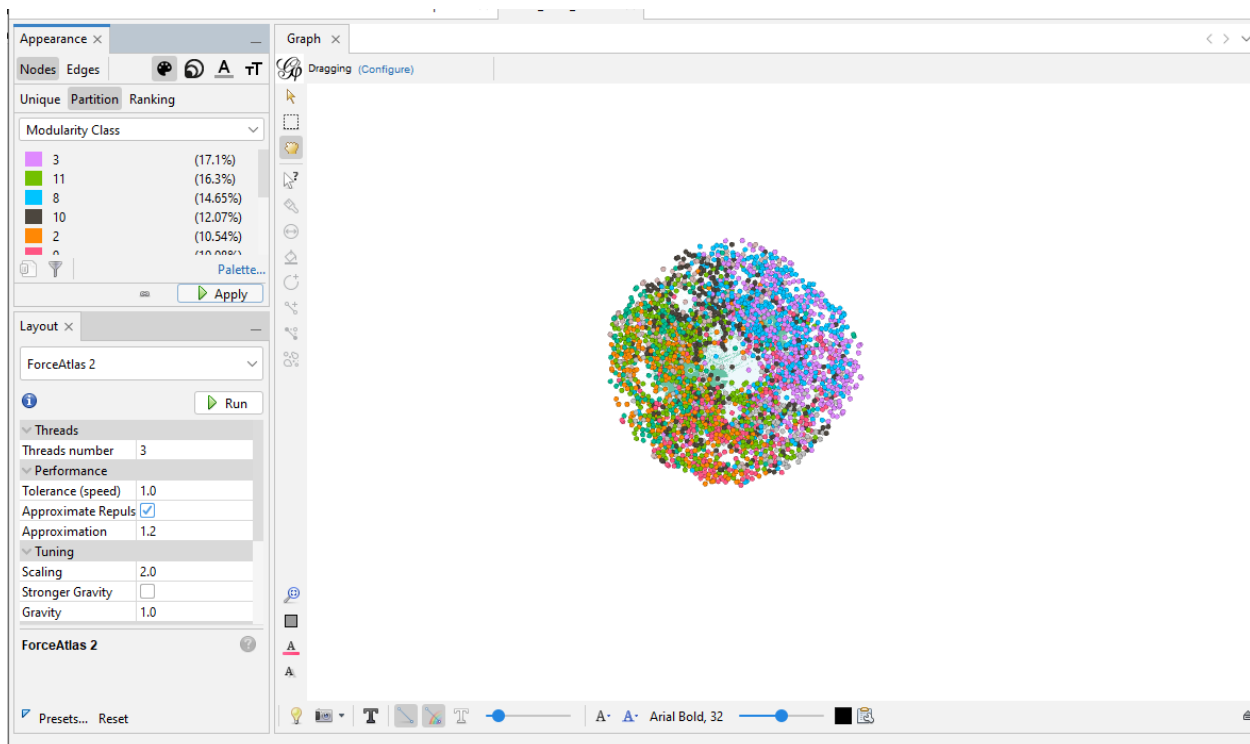
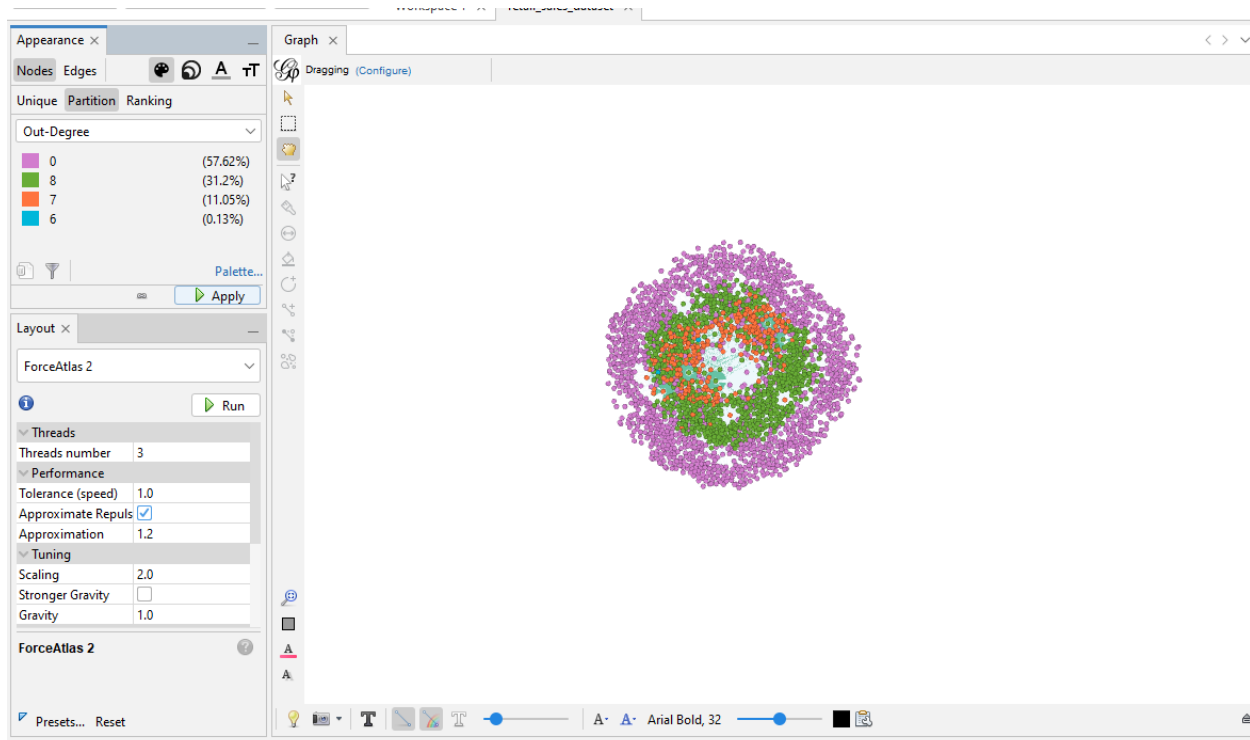
Harmonic Closeness Centrality Distribution

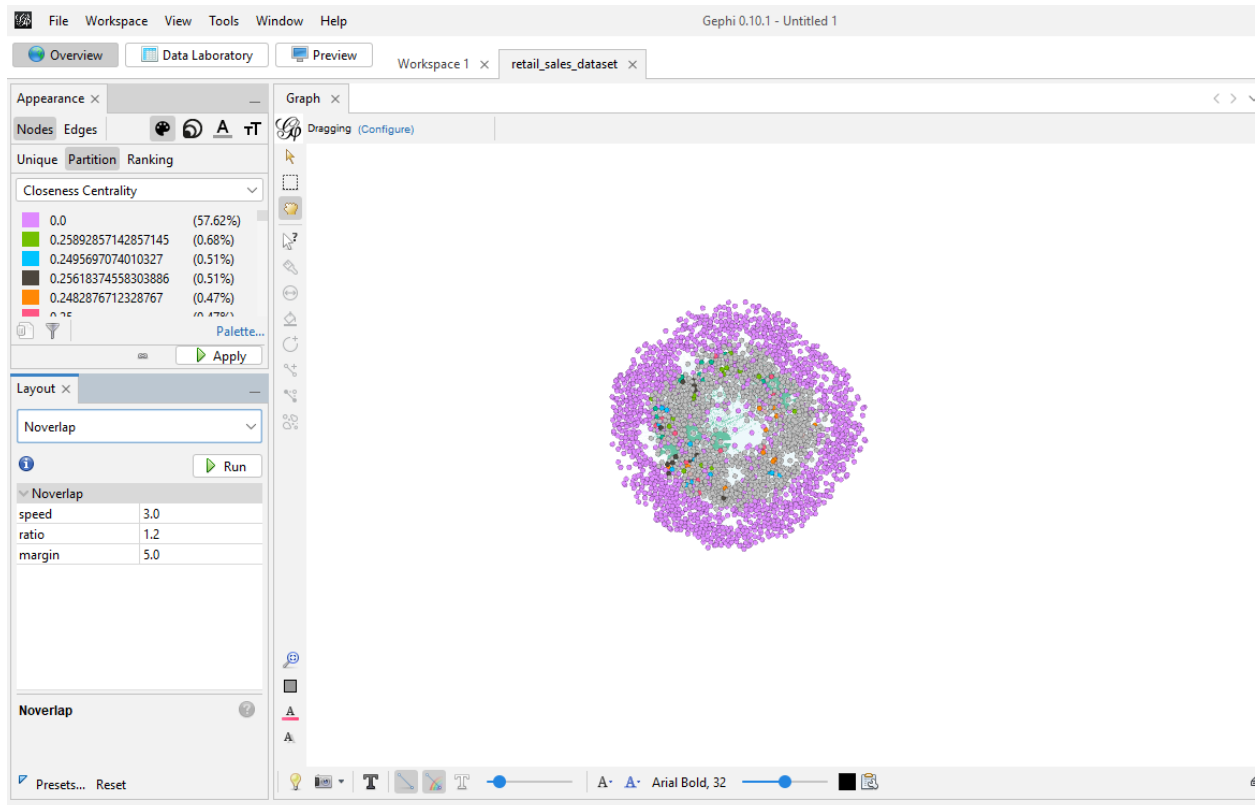
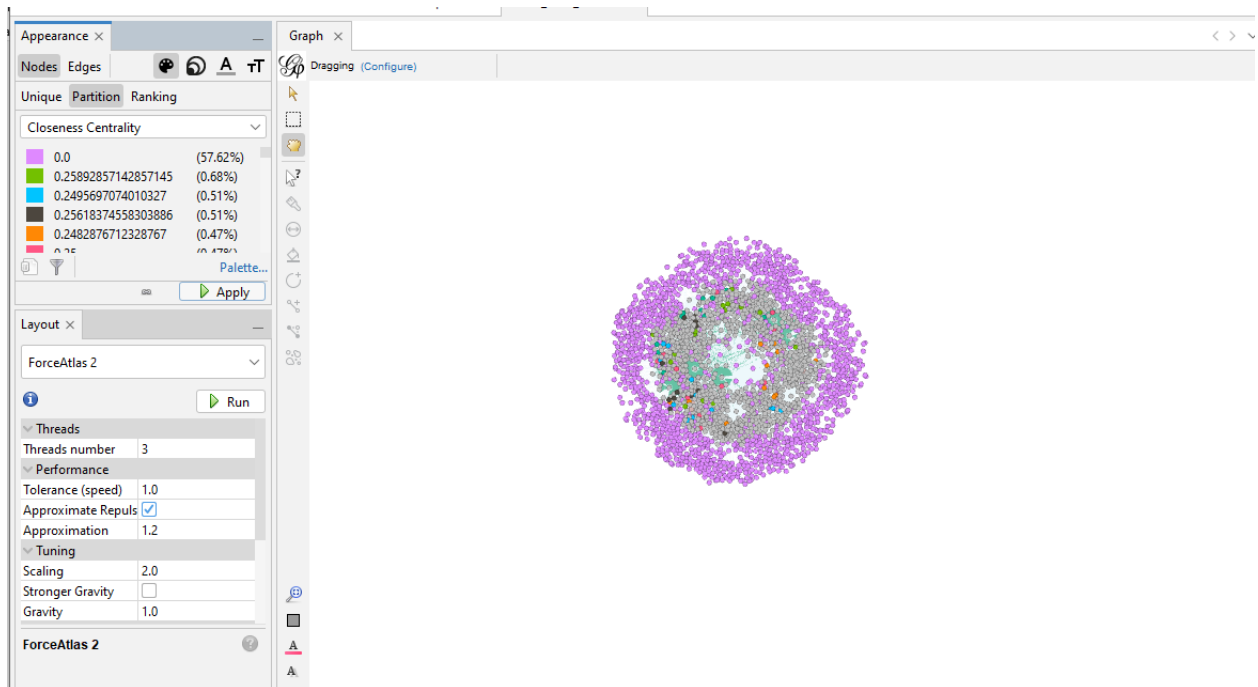


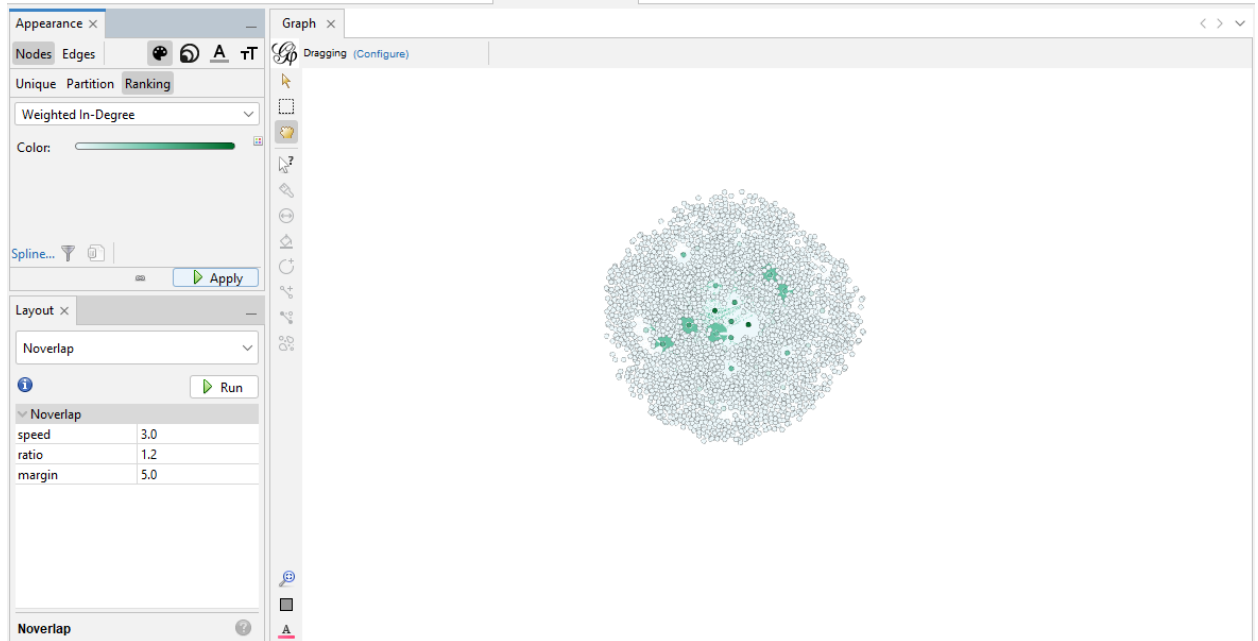
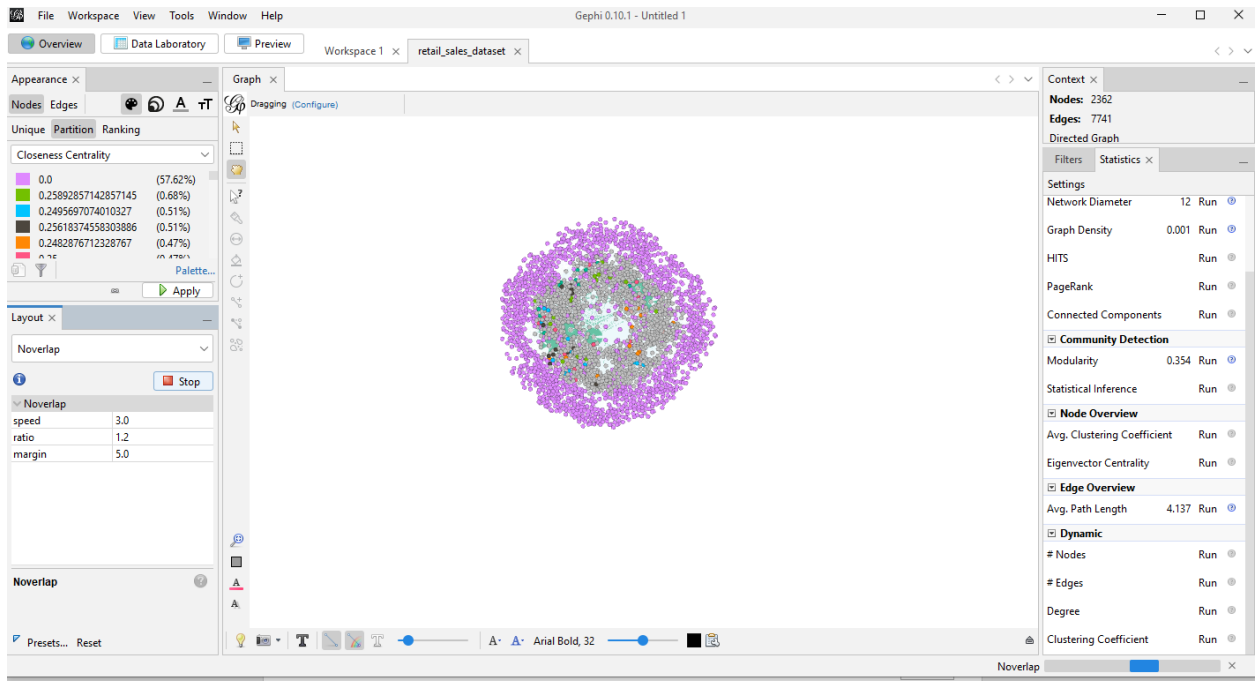
Eccentricity Distribution

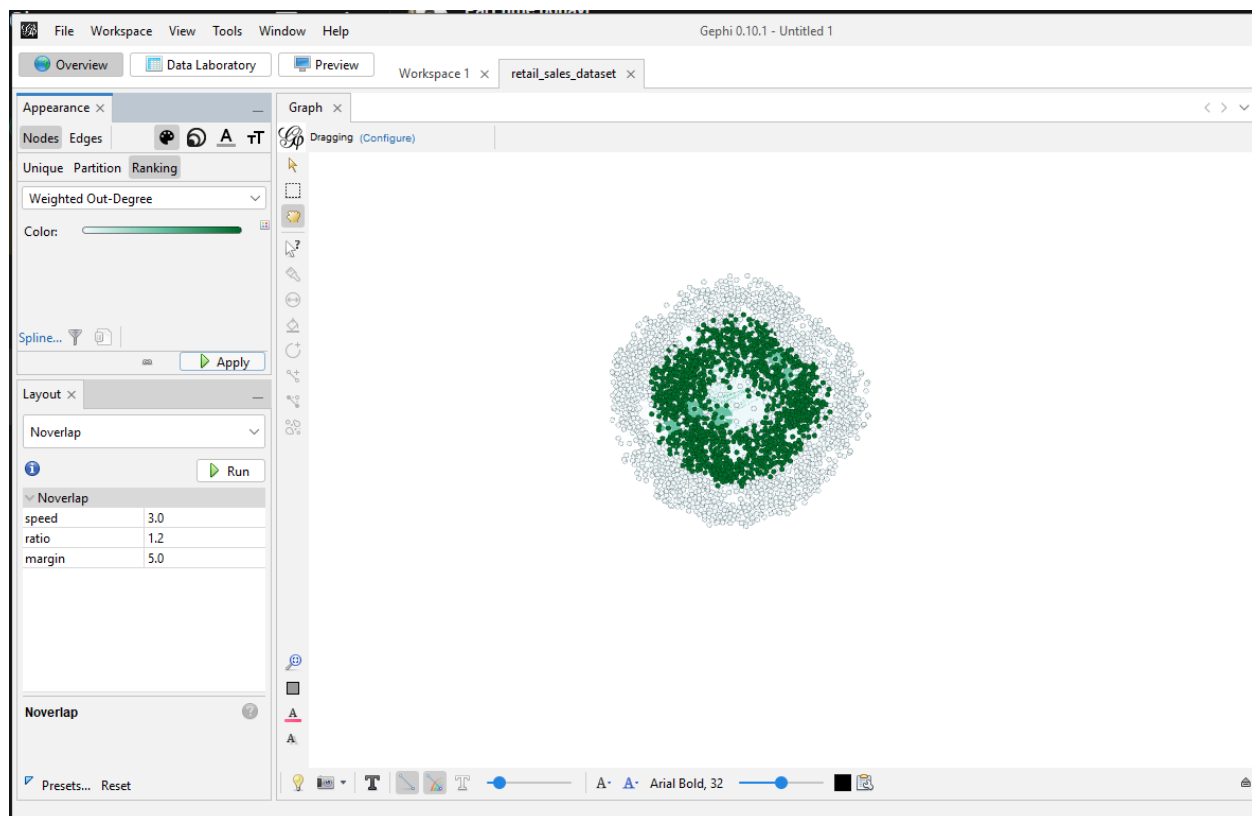












Lab 15: Final Capstone Project

1. Objective

The objective of this final capstone lab is to execute an end-to-end data mining solution addressing an industrial business problem. Students will handle the entire lifecycle, including data ingestion, normalization, modeling across 8 distinct algorithms, comparative performance benchmarking, and formulating data-driven strategic business recommendations.

2. Tools & Datasets

- **Tool:** Student Choice (KNIME, RapidMiner, Orange, or Python)
- **Dataset:** Any Approved Complex Industry Dataset

3. Background Concepts

The final capstone synthesizes the data mining techniques covered throughout the course into a cohesive workflow using the CRISP-DM methodology. Rather than focusing on a single algorithm, students evaluate multiple model classes to identify the optimal solution for their specific dataset

4. Evaluation Framework & Deliverables

Students are not provided step-by-step instructions for this capstone. Instead, projects are evaluated based on the rigorous engineering criteria outlined below:

- **Comprehensive Pipeline Deployment:** Construct a documented data pipeline handling ingestion, feature scaling, and addressing any class imbalances.
- **8-Model Execution Matrix:** Train and validate 8 distinct model families covered during the semester (e.g., Decision Trees, Naïve Bayes, KNN, SVM, Logistic Regression, Random Forests, Gradient Boosted Trees, and Neural Networks).
- **Comparative Performance Analysis:** Consolidate evaluation metrics into a standardized performance dashboard.

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

# 1. LOAD THE DATASET
# Replace 'Bank_Customer_Churn.csv' with your exact uploaded file name
df = pd.read_csv('Customer-Churn-Records.csv')

print("--- Initial Dataset Info ---")
print(df.info())
print("\nFirst 5 Rows:")
print(df.head())

```

```

--- Initial Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   RowNumber             10000 non-null  int64
1   CustomerId            10000 non-null  int64
2   Surname               10000 non-null  object
3   CreditScore            10000 non-null  int64
4   Geography             10000 non-null  object
5   Gender                10000 non-null  object
6   Age                   10000 non-null  int64
7   Tenure                10000 non-null  int64
8   Balance               10000 non-null  float64
9   NumOfProducts         10000 non-null  int64
10  HasCrCard              10000 non-null  int64
11  IsActiveMember         10000 non-null  int64
12  EstimatedSalary        10000 non-null  float64
13  Exited                 10000 non-null  int64
14  Complain               10000 non-null  int64
15  Satisfaction Score     10000 non-null  int64
16  Card Type              10000 non-null  object
17  Point Earned           10000 non-null  int64
dtypes: float64(2), int64(12), object(4)
memory usage: 1.4+ MB
None

```

First 5 Rows:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited	Complain	Satisfaction	Score	Card Type	\
0	101348.88	1	1		2	DIAMOND	
1	112542.58	0	1		3	DIAMOND	
2	113931.57	1	1		3	DIAMOND	
3	93826.63	0	0		5	GOLD	
4	79084.10	0	0		5	GOLD	

	Point Earned
0	464
1	456
2	377
3	350
4	425

Preprocessing Complete. Data split into 80% Train and 20% Test.

```

# 2. DATA PREPROCESSING & CLEANING (Labs 3 & 4)
# =====

# Drop irrelevant columns that don't help prediction (e.g., RowNumber, CustomerId, Surname)
# Modify these column names based on your exact dataset if necessary
cols_to_drop = ['RowNumber', 'CustomerId', 'Surname']
df = df.drop(columns=[col for col in cols_to_drop if col in df.columns])

# Handle Missing Values (if any)
df = df.dropna()

# Encode Categorical Variables (Text to Numbers)
# E.g., converting 'Geography' and 'Gender' to numbers
label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Define Features (X) and Target (y)
# Assuming 'Exited' or 'Churn' is the target column name
target_col = 'Exited' # Corrected target column based on df.info() output
X = df.drop(columns=[target_col])
y = df[target_col]

# Split data into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling (Standardization)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nPreprocessing Complete. Data split into 80% Train and 20% Test.")

```

```

# 3. IMPLEMENT 8 DIFFERENT MODELS (Lab 15 Requirement)
# =====
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClass

# Initialize the 8 models
models = {
    "1. Logistic Regression": LogisticRegression(),
    "2. Decision Tree": DecisionTreeClassifier(random_state=42),
    "3. Naïve Bayes": GaussianNB(),
    "4. K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
    "5. SVM (Linear Kernel)": SVC(kernel='linear', probability=True, random_state=42),
    "6. SVM (RBF Kernel)": SVC(kernel='rbf', probability=True, random_state=42),
    "7. Random Forest": RandomForestClassifier(random_state=42),
    "8. Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

# Dictionary to hold our final evaluation metrics
results_dict = {}

```

```

# Dictionary to hold our final evaluation metrics
results_dict = {}

print("\n--- Training and Evaluating 8 Models ---")
for name, model in models.items():
    # Train the model
    model.fit(X_train_scaled, y_train)

    # Predict on test set
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[: , 1] # Needed for ROC-AUC

    # Calculate performance metrics (KPIs)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_prob)

    # Store metrics in the dictionary
    results_dict[name] = {
        "Accuracy (%)": round(accuracy * 100, 2),
        "Precision (%)": round(precision * 100, 2),
        "Recall (%)": round(recall * 100, 2),
        "F1-Score (%)": round(f1 * 100, 2),
        "ROC-AUC": round(roc_auc, 3)
    }
print(f"Finished training: {name}")

```

```

--- Training and Evaluating 8 Models ---
Finished training: 1. Logistic Regression
Finished training: 2. Decision Tree
Finished training: 3. Naïve Bayes
Finished training: 4. K-Nearest Neighbors
Finished training: 5. SVM (Linear Kernel)
Finished training: 6. SVM (RBF Kernel)
Finished training: 7. Random Forest
Finished training: 8. Gradient Boosting

```



```

# 4. FINAL CAPSTONE PERFORMANCE COMPARISON TABLE
# =====
# Convert results into a structured DataFrame
comparison_df = pd.DataFrame.from_dict(results_dict, orient='index')

print("\n=====")
print("          FINAL MODEL COMPARISON TABLE          ")
print("=====")
print(comparison_df.to_string())
print("=====\\n")

# Find the best model based on Accuracy
best_model_name = comparison_df['Accuracy (%)'].idxmax()
best_accuracy = comparison_df.loc[best_model_name, 'Accuracy (%)']

```

```

=====
          FINAL MODEL COMPARISON TABLE
=====

```

	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	ROC-AUC
1. Logistic Regression	99.9	99.75	99.75	99.75	0.999
2. Decision Tree	99.7	99.24	99.24	99.24	0.995
3. Naïve Bayes	99.9	99.75	99.75	99.75	0.999
4. K-Nearest Neighbors	99.7	99.74	98.73	99.23	0.999
5. SVM (Linear Kernel)	99.9	99.75	99.75	99.75	0.999
6. SVM (RBF Kernel)	99.9	99.75	99.75	99.75	0.998
7. Random Forest	99.9	99.75	99.75	99.75	1.000
8. Gradient Boosting	99.8	99.24	99.75	99.49	0.999

```

=====

```

```

# =====
# 5. BUSINESS INTERPRETATION & RECOMMENDATION
# =====
print("=====")
print("          BUSINESS RECOMMENDATION          ")
print("=====")
print(f"Based on the performance metrics, the winning model is: **{best_model_name}**")
print(f"It achieved a peak baseline predictive Accuracy of {best_accuracy}%.")
print("\nStrategic Action Plan for the Bank:")
print(f"1. Deploy the {best_model_name} model within the bank's CRM system.")
print("2. Flag high-risk accounts automatically before they actively close their accounts.")
print("3. Target flagged accounts with proactive loyalty incentives (e.g., lower interest rates).")
print("4. This data-driven framework will help mitigate customer churn and preserve annual bank revenue.")
print("=====")

```

=====

BUSINESS RECOMMENDATION

=====

Based on the performance metrics, the winning model is: **1. Logistic Regression**
It achieved a peak baseline predictive Accuracy of 99.9%.

Strategic Action Plan for the Bank:

1. Deploy the 1. Logistic Regression model within the bank's CRM system.
 2. Flag high-risk accounts automatically before they actively close their accounts.
 3. Target flagged accounts with proactive loyalty incentives (e.g., lower interest rates, zero fees).
 4. This data-driven framework will help mitigate customer churn and preserve annual banking revenue.
- =====